DEEP AUTHENTICATOR : A METHOD FOR COMPRESSED DEEPFAKE DETECTION

by

Muhammad Zubair

Bachelor of Computer Science, National University of Computer and Emerging Science Lahore, Pakistan, 2019

A Thesis Submitted in Partial Fulfilment of
the Requirements for the Degree of

Master of Computer Science

In the Graduate Academic Unit of your GAU

Supervisor: Saqib Hakak, Ph.D., Computer Science

Examining Board: Francis Palma, Ph.D., Computer Science
Fatemeh Khoda Parast, Ph.D., Computer Science
Argyri Panezi, PhD., Faculty of Law

This thesis is accepted by the
Dean of Graduate Studies

THE UNIVERSITY OF NEW BRUNSWICK

Month, 2025

© Muhammad Zubair, 2025

Abstract

The rapid advancement of generative AI and social media has enabled the widespread dissemination of compressed DeepFake videos due to storage and bandwidth constraints. However, current DeepFake detection methods struggle with compressed videos, making their identification a critical challenge. We propose an efficient two-stream frame-level approach that first extracts I-Frames from input videos. One stream processes RGB information to detect semantic inconsistencies, while the other extracts noise features using Spatial Rich Model (SRM) filters. Both streams utilize deep convolutional neural networks, and our findings suggest that using more than three SRM filters enhances detection performance. Finally, stream fusion was applied. We evaluate our approach on large, diverse DeepFake datasets, including FaceForensics++, Celeb-DF-V1, and Celeb-DF-V2. Experimental results demonstrate that our method achieves state-of-the-art performance.

Dedication (if any)

This thesis is dedicated to my parents and siblings. Their love and support have been most important key factor in my journey of knowledge. Their faith in my abilities, coupled with their encouragement, has enabled me to fulfilling my dreams. My deepest gratitude goes out to them for being my steadfast pillars of strength and for their invaluable contributions to my life. Thank you for your emotional and financial support.

Acknowledgements

I sincerely thank my supervisor, Dr. Saqib Hakak, for his continuous support and guidance. He has taught and motivated me throughout my research journey at UNB. I would like to thank him for providing me a chance to do research. I would like to thank the University of New Brunswick, Faculty of Computer Science, and the Canadian Institute of CyberSecurity for the opportunity and support. I would also like to thank all my professors and mentors who taught me during my studies at UNB. I also thank NBIF and the Harrison McCain Foundation for providing initial funding for my research. I would also like to acknowledge OpenAI's ChatGPT and Grammarly, as during the preparation of this work, these tools helped enhance the writing quality of this work.

Table of Contents

Abstract ii

Dedication iii

Acknowledgments iv

Table of Contents v

List of Tables viii

List of Figures x

Abbreviations xiii

1 Introduction 1

1.1 Introduction 1

1.2 Problem Statement 5

1.3 Objective of the Study 6

1.4 Summary of Contributions 6

1.5 Thesis Organization 7

2 Background & Literature Review 8

2.1 Background 8

2.1.1 Technical Terms 8

2.1.2 Types of DeepFake Videos 10

2.1.3 DeepFake Generation 13
2.1.3.1 DeepFake Generation Models 13
2.1.3.2 *AutoEncoders* 14
2.1.3.3 *Generative Adversarial Networks* 17
2.1.3.4 *Diffusion Models* 19
2.2 Recent Advances 22
2.2.1 DeepFake Datasets 23
2.2.1.1 Compression Aware Datasets: 23
2.2.1.2 Non-Compression Datasets: 26
2.2.2 Compressed DeepFake Detection 29
2.2.2.1 Compression Aware Detection 31
2.2.2.2 Compression Centric Detection 33
2.3 Concluding Remarks 39

3 Proposed Architecture **41**
3.1 Data Preprocessing 41
3.2 Data Augmentation 43
3.3 RGB Stream 43
3.4 SRM Noise Stream 47
3.5 Fusion Box 49
3.6 Concluding Remarks: 50

4 Experiments **51**
4.1 Datasets 51
4.2 Experimental Setup 54
4.3 Evaluation Matrix 55
4.4 Network Selection for RGB Stream 56
4.4.1 RGB Stream Without Augmentation 56

4.4.2 RGB Stream with Augmentation 59
4.4.3 RGB Stream Visualization 61
4.5 Filters Selection for SRM Stream 62
4.6 Ensemble of Both Branches 67
4.7 Comparison with Other Studies 71
4.8 Concluding Remarks: 73

5 Conclusion & Future Works **74**
5.1 Conclusion 74
5.1.1 Future Work 75

Bibliography **109**

A Codes **110**

Vita

List of Tables

2.1 Summary of Manipulation Tools. 15

2.2 Summary of AutoEncoders. 17

2.3 Summary of GANs. 20

2.4 Summary of Diffusion Models. 22

2.5 Compression Aware Datasets 24

2.6 Non-Compression Datasets 30

2.7 Compression Aware Detection 33

2.8 Compression Resilience based Detection 37

2.9 Image Restoration based Detection 38

2.10 Compression features based Detection 39

3.1 Comparison of Different Deep Learning Models 44

4.1 Summary of Datasets 52

4.2 Summary of Compression and Storage of Datasets 52

4.3 Distribution of Videos for Training and Testing Phase with 80% and 20% Distribution with Compression level 23 and 40 54

4.4 Performance Comparison of Deep Learning Models without Augmentation on RGB Stream 57

4.5 RGB & Residual Streams Selected Hyperparameters 58

4.6 Performance Comparison of Deep Learning Models with Augmentation on RGB Stream 60

4.7 Performance Comparison of Deep Learning Models with Augmentation on 3 SRM Filters 63
4.8 Performance Comparison of Deep Learning Models with Augmentation on 9 SRM Filters 63
4.9 Performance Comparison of Deep Learning Models with Augmentation on 27 SRM Filters 64
4.10 Ensemble of RGB EfficientNet-B4 with SRM 3, SRM 9, and SRM 27. CLB represents CELEB-DF, DF represents DeepFakes, F2F represents Face2Face, FS represents FaceSwap, and NT represents NeuralTextures subsets of FaceForensics++ datasets. 68
4.11 Ensemble of EfficientNet-B7 with SRM 3, SRM 9, and SRM 27. CLB represents CELEB-DF, DF represents DeepFakes, F2F represents Face2Face, FS represents FaceSwap, and NT represents NeuralTextures subsets of FaceForensics++ datasets. 69
4.12 Ensemble of RGB ResNet-152 with SRM 3, SRM 9, and SRM 27. CLB represents CELEB-DF, DF represents DeepFakes, F2F represents Face2Face, FS represents FaceSwap, and NT represents NeuralTextures subsets of FaceForensics++ datasets. 70
4.13 Comparison with Other Studies 72

List of Figures

1.1 DeepFake Images Generated through an Online App [1]. 2

1.2 Distribution of DeepFake Attacks Across Different Industries, according to Sensity AI [2]. 3

1.3 The Left picture is real, and the middle and right pictures are fake. These samples are taken from the Celeb-DF-V2 dataset [3]. 4

2.1 Images for total face synthesis are collected from [4]. Images for attribute manipulation have been generated from FaceApp [5]. Images for motion transfer are collected from [6]. 11

2.2 DeepFake Generation PipeLine 14

2.3 DeepFake Generation Algorithms 14

2.4 AutoEncoder A with an image of Person A as input (top row). AutoEncoder B with an image of Person B as input (bottom row). . . . 16

2.5 DeepFake videos are created by using two networks. Two networks shared the same encoder but two different decoders in the training step (top step). An image of face A is encoded with a shared encoder and decoded with decoder B to create a DeepFake (bottom step). . . 16

2.6 CycleGAN for DeepFake Generation by Performing Gender Swap from Male to Female 18

2.7 Noising and De-noising Process for Creating FaceSwap Using Diffusion Model 21

2.8 DeepFake Classification 23

3.1 Overview of Proposed Architecture 42
3.2 Semantic Features in Forged Videos 44
3.3 Absence of Semantic Features in Forged Images 47
3.4 Collection of SRM Filters 49

4.1 Results on RGB Stream without Augmentation on with Different Backbone Models 58
4.2 Results on RGB Stream with Augmentation on with Different Backbone Models 59
4.3 GRAD CAM Visualization on MesoNet 61
4.4 GRAD CAM Visualization on Xception 61
4.5 GRAD CAM Visualization on ResNet-152 62
4.6 Results on 3 Filters on SRM Streams 63
4.7 Results on 9 Filters on SRM Streams 64
4.8 Results on 27 Filters on SRM Streams 64
4.9 Results on EfficientNet-B4 Different Number of Filters 65
4.10 Results on EfficientNet-B7 Different Number of Filters 65
4.11 Results on ResNet-152 Different Number of Filters 65
4.12 Comparison of SRM Noise in High Quality (HQ) and Low Quality (LQ) Videos. Each two consecutive rows indicates a noise feature from the same dataset. The order of the datasets is Celeb-DF-V2, FaceSwap, DeepFakes, and Face2Face. Green represents real, and red represents fake images. 66
4.13 Results comparison of RGB Stream, SRM stream, and Ensemble Results for ResNet-152 67
4.14 Comparison With Other Studies 71

A.1 Code Snippet to convert a video into H.264 and apply Compression on it 110
A.2 Code Snippet for Extracting I-Frames from a Video 111
A.3 Code Snippet for Face Detection and Resizing Image 112
A.4 Code Snippet for Training of ResNet-152 on RGB data Part 1 113
A.5 Code Snippet for Training of ResNet-152 on RGB data Part 2 114
A.6 Code Snippet for Training of ResNet-152 for RGB data Part 3 115
A.7 Code Snippet for Calling Function to Train ResNet-152 for Different DataSets on RGB data PART 1 116
A.8 Code Snippet for Calling Function to Test ResNet-152 for Different DataSets on RGB data Part 2 117
A.9 Code Snippet for Calling Function to Test ResNet-152 for Different DataSets on RGB data Part 3 118
A.10 Code Snippet for Defining SRM Layer Part 1 119
A.11 Code Snippet for Defining SRM Layer Part 2 120
A.12 Code Snippet for Defining SRM Layer Part 3 121
A.13 Code Snippet for Defining SRM Layer Part 4 122
A.14 Code Snippet for Defining SRM Layer Part 5 123
A.15 Code Snippet for Defining SRM Layer Part 6 124
A.16 Code Snippet for Defining SRM Layer Part 7 125
A.17 Code Snippet for Defining SRM Layer Part 8 126
A.18 Code Snippet for Defining SRM Layer Part 9 127
A.19 Code Snippet for Defining SRM Layer Part 10 128
A.20 Code Snippet for Defining ResNet-152 with SRM Layer Part 1 129
A.21 Code Snippet for Defining ResNet-152 with SRM Layer Part 2 130

List of Abbreviations

$FRUC$	Frame Rate Up-Conversion
$LLMs$	Large Language Models
$GANs$	Generative Adversarial Networks
GPT	Generative Pre-trained Transformer
USD	United States Dollar
RGB	RED, Green, Blue
HSV	Hue, Saturation, Value
$eKYC$	Electronic Know Your Customer
$3DMM$	3D Morphable Model
$DDPM$	Denoising Diffusion Probabilistic Mode
SRM	Spatial Rich Models
CNN	Convolution Neural Network
$FF++$	Face Forensics++
SR	Super Resolution
$FOMM$	First Order Motion Model
SVM	Support Vector Machine
$COVID-19$	Coronavirus 2019
$SSIM$	Structural Similarity Index Measure
GRU	Gated recurrent unit
DF	DeepFake
$F2F$	Face2Face
FS	FaceSwap
NT	NeuralTextures
HQ	High Quality
LQ	Low Quality

Chapter 1

Introduction

1.1 Introduction

DeepFake consists of two words *deep* and *fake* [7, 8, 9, 10]. Deep refers to *deep learning* [11], while *fake* can describe something counterfeit, fraudulent, or designed to deceive others. So, *DeepFake* is the process of using deep learning to create or generate content in the form of image, video, and audio that, in reality, does not exist [12]. Manipulation of images, videos, and audios is not a new phenomenon but has been investigated in the last few decades for forgeries such as copy-move forgery [13, 14], splicing, image inpainting [15], Frame Rate Up-Conversion (FRUC) [16], Face Swap [17] and Face reenactment [18]. The major difference is the use of deep learning-based generative artificial intelligence to achieve better and more realistic results.

Generative artificial intelligence based on generative models is being used to generate synthetic images, videos, and audio these days. Generative models learn by analyzing training examples. These models learn patterns and distribution from examples to produce facsimiles [19]. The current rapid progress in the development

of generative artificial intelligence models such as Large Language Models (LLMs), Transformers, Generative adversarial networks (GANs), Autoencoders, and Diffusion models is contributing to improved quality of synthetic content while maintaining time efficiency [20, 21]. DALL-E [22], Sora [23], Generative pre-trained Transformer (GPT) [24], and DeepSeek [25] are examples of prominent models in the field of generative artificial intelligence. Content generated through artificial intelligence is being used to facilitate multiple important aspects of human society, such as entertainment and education [26]. However, cybercriminals can exploit tools based on these state-of-the-art artificial intelligence models to spread content to facilitate their malicious activities, such as spreading fake news, committing financial fraud, and blackmailing.

Cybercriminals use social media platforms to spread synthetic content called DeepFake. Social media platforms such as YouTube, Facebook, Snapchat, and X, previously known as Twitter, not only facilitate the sharing of content with ease but also provide a substantial audience of more than 5 billion patrons [27]. DeepFake can be used to disrupt national and individual security by targeting someone by using DeepFake apps available online [1]. Due to the advancement of technology, only limited data is required to create DeepFake of a person; for example, only one image was required to create the DeepFake images presented in Fig.1.1.

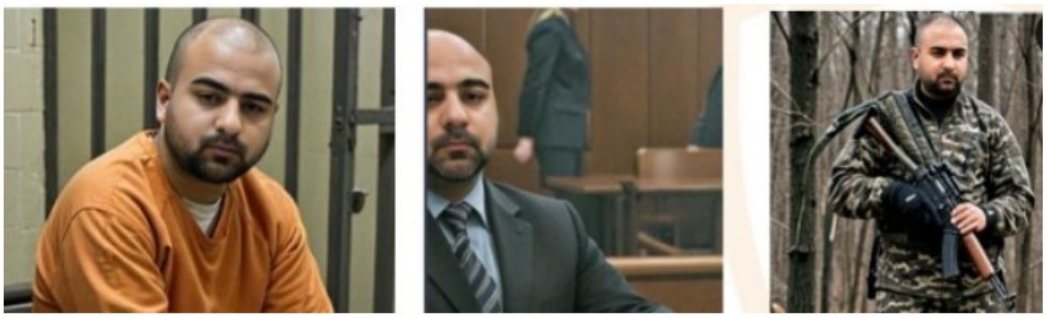


Figure 1.1: DeepFake Images Generated through an Online App [1].

DeepFake technology has previously been used to target individuals from different

backgrounds [2]. Almost 70% of these attacks are launched on individuals working in trading, retail, public subsidies, and health, as shown in Fig.1.2. One of the social engineering attacks that has been reported was on the chief executive officer of an energy firm in the United Kingdom using DeepFake. This attack resulted in a scam of 243,000 USD [28]. On social media, DeepFake of numerous famous personalities and world leaders can be found that were created with the intent of financial fraud, influencing elections or war, and damaging reputation. Well known, examples of celebrities and politicians who are being targeted by DeepFake cybercriminals are Emma Watson [29], Elon Musk [30], Donald Trump [31], Volodymyr Zelenskyi [32], Tom Hanks [33], and Joe Biden [34].

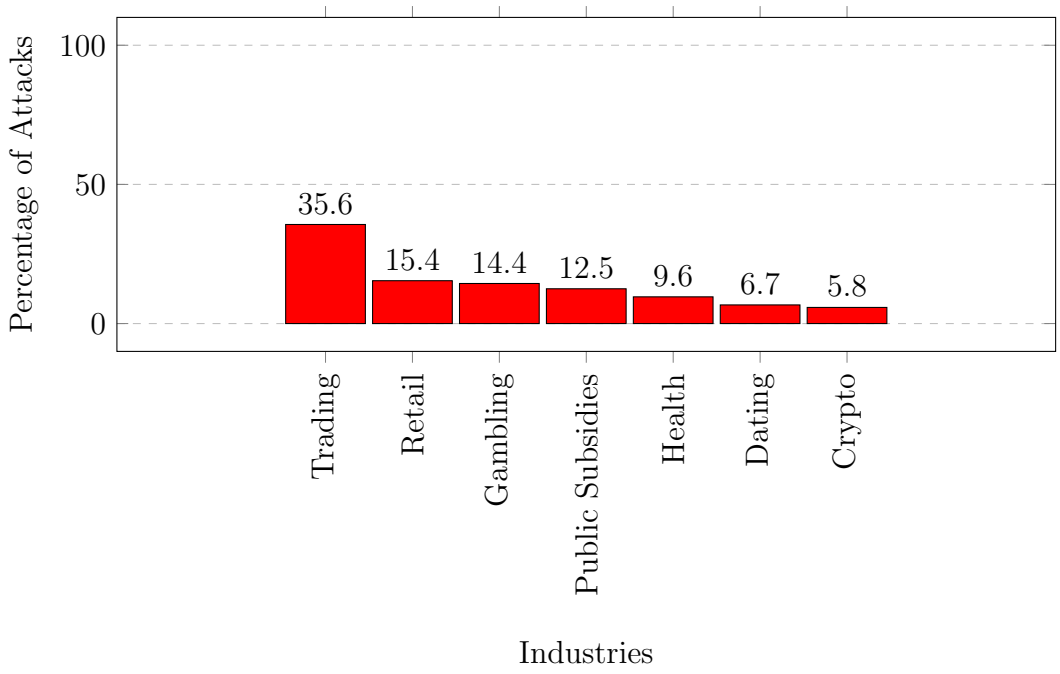


Figure 1.2: Distribution of DeepFake Attacks Across Different Industries, according to Sensity AI [2].

It is becoming difficult to detect DeepFake with blank eyes, as shown in Fig. 1.3, especially due to the progress of research and ease of generating high-quality DeepFake due to the availability of DeepFake generation tools over the internet. According to Cisco, more than 80% of the volume of internet consumer traffic is in video format



Figure 1.3: The Left picture is real, and the middle and right pictures are fake. These samples are taken from the Celeb-DF-V2 dataset [3].

[35, 36]. Nowadays, when a video is uploaded on social media, it undergoes compression to save bandwidth and to optimize storage [37, 36, 38]. Therefore, it is important to develop automatic DeepFake detection models for compressed DeepFake videos.

Several studies have used temporal features for DeepFake detection [39, 40, 41]. However, temporal feature-based studies are computationally expensive. Several studies have used spatial features to detect DeepFake in videos. Some spatial domain studies have used semantic features extracted directly from RGB (Red, Green, and Blue) colour model images for detection through deep learning [39, 42, 43]. However, these semantic RGB-based methods can be improved by considering diverse experimental settings like different compression levels, forgeries, and backbone networks. Several other studies have focused on noise or residual features extracted from RGB images [44, 45, 46]. These studies need improvement in training time efficiency and detection performance. For DeepFake detection, very few studies combined both semantic RGB and noise-based features [47] by considering the effectiveness of the hybrid approach in other image manipulation types [48]. This hybrid approach still needs to be extended by adding different compression levels, noise or residual extraction settings, and datasets. In this thesis, we present a hybrid approach based on semantic and residual features, which provides a more accurate solution than existing stud-

ies while maintaining scalability as well. This model is also evaluated on different compression levels to provide a comparison between high and low-quality videos.

1.2 Problem Statement

As discussed in the previous section, the proliferation of DeepFake videos across various social networks underscores the importance of detecting them. Nowadays, when a video is uploaded on social media, it undergoes compression to save bandwidth [37] and to optimize storage [38]. Due to low internet speed [36, 49], in many countries around the globe, it is time-consuming to upload high-quality videos on social media, which restricts the sharing of information in high quality. Therefore, it is important to develop automatic DeepFake detection models for compressed DeepFake videos.

Let $V = \{v_1, v_2, \ldots, v_n\}$ be a set of compressed video instances. Each $v \in V$ is represented as a tuple:

$$v = \langle C_v, y_v \rangle, \quad y_v \in \{0, 1\}$$

where:

- C_v is the content of v.
- y_v is the binary label of v, where 0 indicates a DeepFake video and 1 represents a real video.

Given an unseen compressed video v, its content C_v, the problem is to assign the label y_v.

1.3 Objective of the Study

This study aims to advance the field of compressed DeepFake detection by focusing on the following key objectives:

- Investigate current DeepFake detection methods.
- Develop a new model for detecting compressed DeepFake videos.
- Evaluate the performance of the proposed model against current state-of-the-art methods.

These objectives not only enhance the understanding of compressed DeepFake but also contribute to the development of practical solutions for their detection.

1.4 Summary of Contributions

This research effort focuses mainly on safeguarding social media users by identifying compressed DeepFake videos. This will be achieved by distinguishing between manipulated and real videos. In summary, this thesis has made the following contributions:

- We introduce a new DeepFake Classification.
- We propose a new dual-stream model that uses a fine-tuned two-dimensional convolution neural network (2D-CNN) model by utilizing semantic features and residual features for DeepFake detection.
- To enhance the model's credibility, we test the model on three publicly available datasets. The evaluation results show that our model can detect DeepFake

in compressed video more accurately than state-of-the-art models while maintaining scalability.

- We test the model on two compression levels to provide features and performance comparison analysis of the proposed model.

1.5 Thesis Organization

The structure of the remainder of this thesis is outlined as follows:

Chapter 2: Background & Literature Review provides a fundamental understanding of the topic, including a discussion of key technical terms, DeepFake types, generation methods, and tools. The literature review focuses on prior research conducted on DeepFake detection and datasets. This chapter offers a new comprehensive classification of DeepFake.

Chapter 3: Presents the motivation behind the proposed architecture and describing the architecture, including an explanation of each component of the architecture.

Chapter 4: Experiments & Results, explains the final results of the conducted research. This chapter covers the datasets, details of the experimental setup, and performance metrics.

Chapter 5: We conclude the thesis by summarizing the key contributions, challenges faced during research and provide future research directions.

Chapter 2

Background & Literature Review

In this chapter, we provide the necessary background to understand the domain of DeepFake, and then we provide an overview of related studies of DeepFake detection and datasets.

2.1 Background

In this section, we provide an overview of the background and related studies of DeepFake. Firstly, we discuss key technical terms related to DeepFake. Then, we explain the types of DeepFake. After that, we explain the generation mechanism and tools.

2.1.1 Technical Terms

Before delving deeper into DeepFake, it is important to understand the basics of this subject. Knowing the key technical terms related to this field is essential.

- **Video:** A visual medium that captures and presents moving images, often

accompanied by audio, for the purpose of communication, entertainment, or documentation [50].

- **Frame:** A single still image within a sequence of images that make up a video, capturing a specific moment in time [51].
- **Compression:** The technique of reducing video or image file size while trying to maintain visual quality for efficient storage, transmission, and playback [52].
- **Lossy Compression:** Lossy compression is a technique that reduces files by permanently removing certain information [53].
- **CODEC:** An abbreviation for coder-decoder, a codec refers to a software or hardware algorithm used to compress and decompress audio or video data [54].
- **Compression Factors:** Parameters in video encoding that balance file size and quality by adjusting the level of compression [55].
- **C0:** Videos that are compressed with a compression factor of 0 [55].
- **C23:** Videos that are compressed with a compression factor of 23 [55].
- **C40:** Videos that are compressed with a compression factor of 40 [55].
- **I-Frames:** A keyframe in video compression that contains complete information, serving as a reference for subsequent frames [55].
- **P-Frames:** A compressed video frame that predicts the differences from a preceding I-frame, improving video compression efficiency [56].
- **B-Frames:** A compressed video frame that predicts the differences from preceding and subsequent frames, maximizing video compression by exploiting temporal redundancy [55].

- **Electronic Know your Customer (eKYC):** It is a system used by organizations to electronically verify the identity of customers by using various processes like identity documents and face matching. These systems also motions related information, such as the head movement of a customer, for liveness detection [57].

- **Source Person:** The person or face that is used as a starting point or reference in creating a DeepFake. It is the original face that is replaced or manipulated [58].

- **Target Person:** The person or face that is desired to be shown in the final DeepFake video. It is the face that is overlaid or inserted onto the source face to create the altered video or image [58].

2.1.2 Types of DeepFake Videos

DeepFake videos can be created using different approaches and are categorized into the following types: (i)Facial Swapping, (ii)Facial Reenactment, (iii) Facial Attribute Manipulation, (iv) Total Face Synthesis, and (v) Motion Reenactment.

- **Facial Swapping:** Digital face swapping refers to the process of replacing the entire face of one person in an image or video with the face of another person, thereby altering the identity of the depicted individual [59, 60], as shown in Fig. 2.1. Face swapping poses a significant threat to electronic Know Your Customer (eKYC) services [57].

- **Facial Reenactment:** Facial reenactment involves modifying facial expressions and conveying emotions of image subjects through alterations. This process manipulates the visual content to depict different emotional states, ranging from joy to sadness or anger. This may range from making Mona Lisa frown

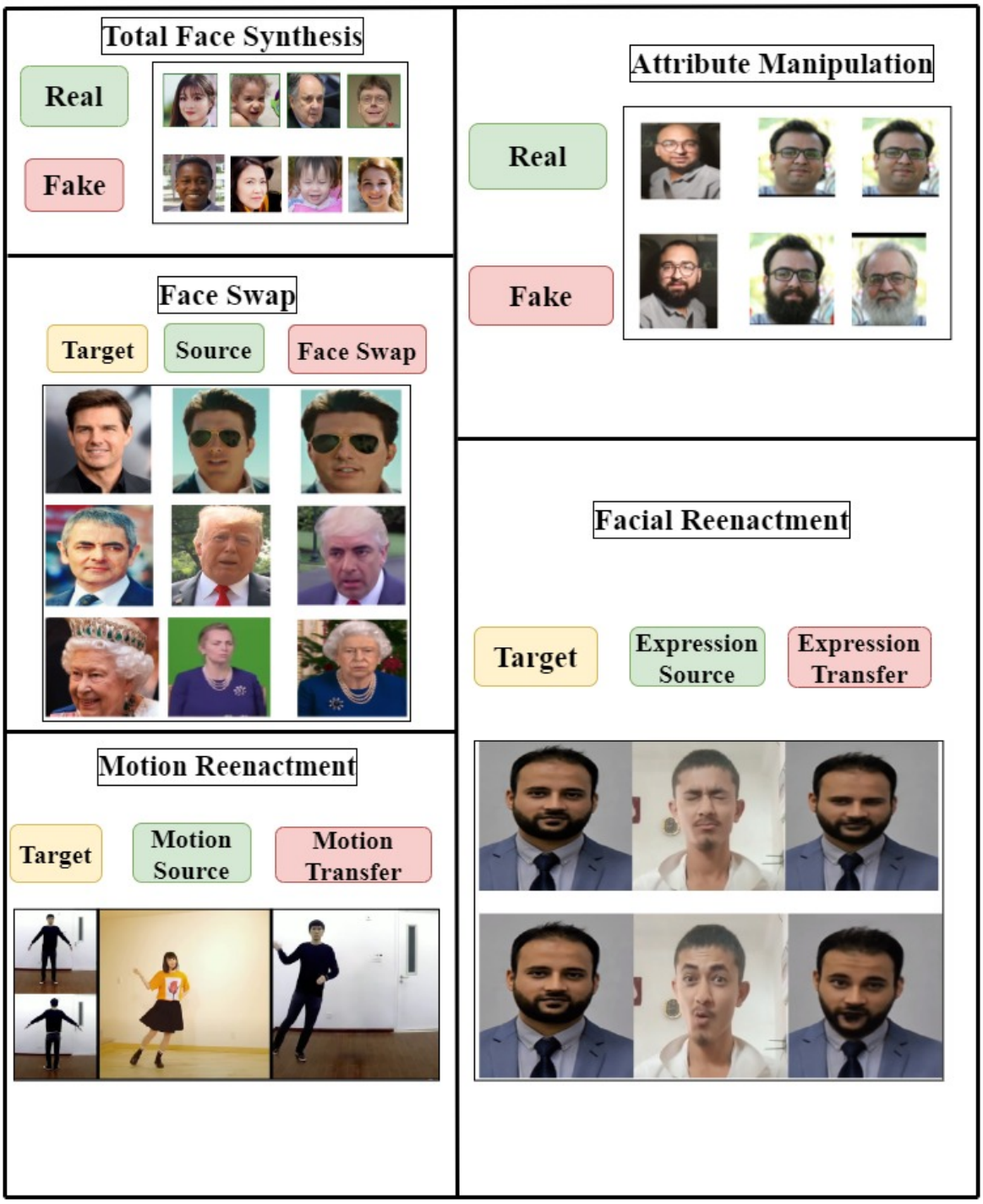


Figure 2.1: Images for total face synthesis are collected from [4]. Images for attribute manipulation have been generated from FaceApp [5]. Images for motion transfer are collected from [6].

to making her talk [59, 60]. One of the examples of these kinds of DeepFake is shown in Fig. 2.1. Face reenactment poses a significant threat to electronic

Know Your Customer (eKYC) services [57].

- **Facial Attribute Manipulation:** Facial Attribute Manipulation and targeted modification of selected visual attributes of the Image subject refers to the intentional alteration of specific characteristics of a person depicted in an image. These editable attributes encompass a range of visual elements, including the subject's gender, hair colour, skin colour, age, facial accessories, and use of cosmetics. This process involves selectively adjusting these attributes to create desired changes in the overall appearance of the subject in the image [61], as shown in Fig. 2.1.

- **Total Face Synthesis:** In total face synthesis, entirely non-existent face images are generated using deep learning techniques [62, 63]. These synthetic faces are created from scratch rather than being modified versions of images of real individuals. Advanced generative models, such as Generative Adversarial Networks (GANs), are often used to synthesize highly realistic facial images that do not correspond to any actual person [64]. This technique is commonly used in the creation of fictitious identities, digital avatars [65], and AI-generated media content, as shown in Fig. 2.1.

- **Motion Reenactment:** Motion reenactment refers to the process of replicating or mimicking the movements and actions of individuals or objects from existing video footage. It involves using deep learning algorithms and computer vision techniques to analyze and understand the motion patterns captured in the source material. Using this analysis, the algorithms can generate new video content that appears to depict the same or similar movements as the original footage [66, 67] as shown in Fig. 2.1.

2.1.3 DeepFake Generation

DeepFake videos that contain facial image manipulation are prepared through a pipeline consisting of five steps, as shown in Fig. 2.2. This pipeline has been modified as research has made progress in several research areas, such as deep learning. The first step is to select a source image or video and to find the target. Candidates can be selected either manually or by comparing faces. In the second step, the region of interest is selected from the images. In the first step, facial information can be shown in different formats, such as facial landmarks [68], Depth Map [69], and 3D Morphable model (3DMM) parameters [70]. In the fourth step, the facial features can be replaced in source images with target images by using deep learning or non-deep learning approaches. In the last step, the image of the region of interest is blended again with the original source image. Before deep learning, the replacement of a source face with a target face would eliminate all important features of a source face. Now, with the usage of deep learning approaches, important features of the expression of the source face can be maintained [64].

2.1.3.1 DeepFake Generation Models

This section defines the generative models used in the fourth step of DeepFake generation for the creation of DeepFake, as illustrated in Fig. 2.3. It also explores their working principles, variations, and advancements over time. The tools created by utilizing these models are available on the web, desktop, and mobile devices [55]. A brief summary of DeepFake generation tools can be found in Table 2.1

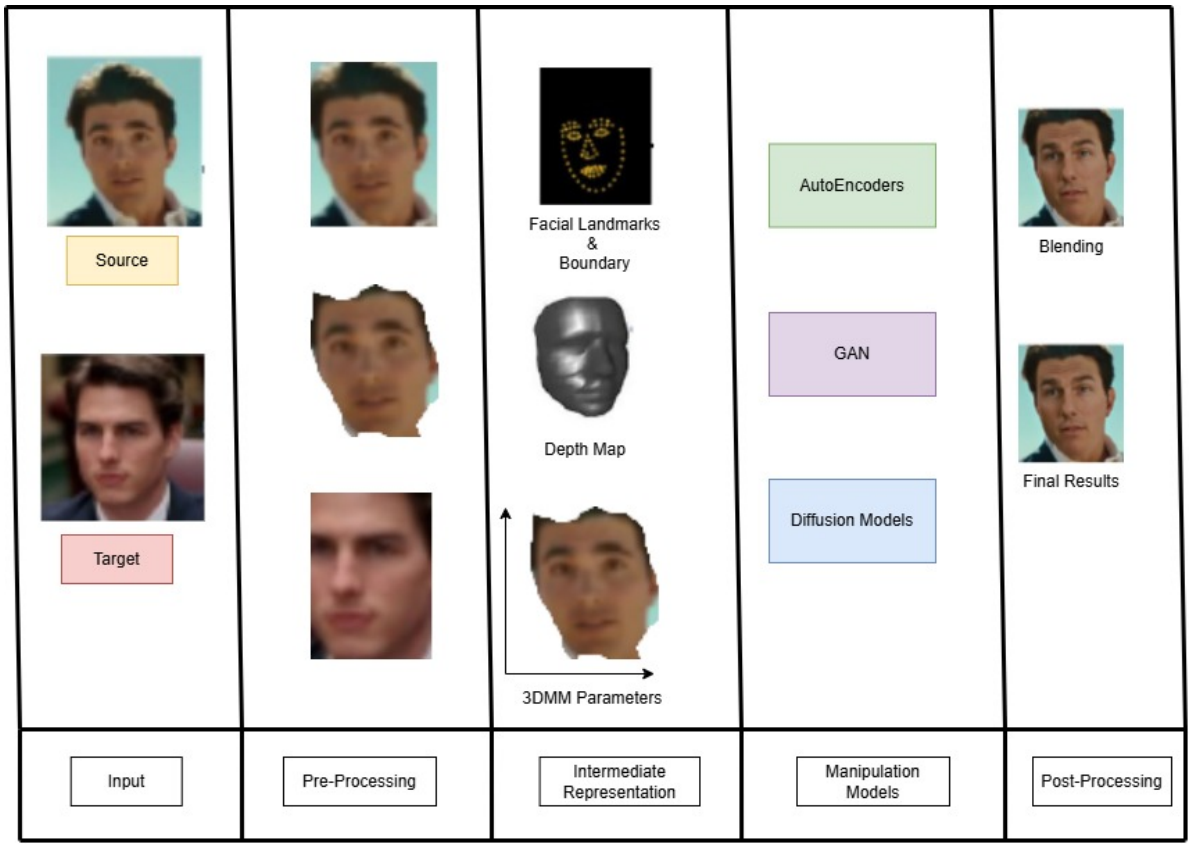


Figure 2.2: DeepFake Generation PipeLine

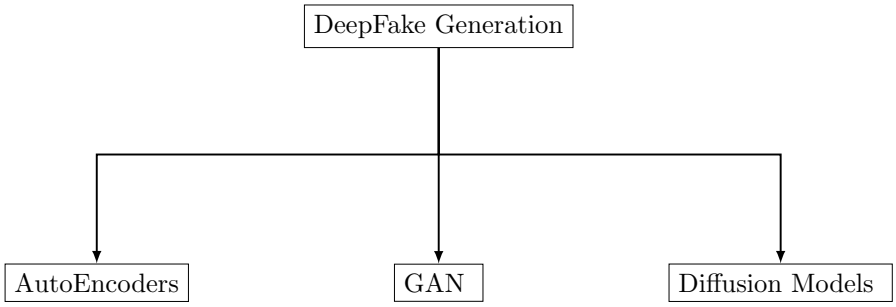


Figure 2.3: DeepFake Generation Algorithms

2.1.3.2 *AutoEncoders*

Autoencoders were first introduced in 1986 [98]. An autoencoder is a special type of neural network for unsupervised learning [99] that is trained to copy its input to its output. Basically, it acts as a dimensional reduction process. In autoencoder, given an image of a person, an autoencoder first encodes the image into a lower dimensional

Table 2.1: Summary of Manipulation Tools.

Name	**Generation Mechanism**	**Live Stream Capable**	**Identity Swap**	**Face Synthesis**	**Attribute Manipulation**	**Expression Transfer**	**Motion Transfer**
FaceApp-MobileApp [5]	Not Disclosed	✗	✓	✗	✓	✗	✗
FacePlay [71]	Not Disclosed	✗	✓	✗	✗	✗	✗
Reface [72]	Not Disclosed	✗	✓	✗	✓	✓	✗
Diff AI [73]	Not Disclosed	✗	✗	✗	✗	✗	✓
FaceHub [74]	Not Disclosed	✗	✓	✗	✗	✗	✗
DeepFaceLab [75]	AutoEncoders	✗	✓	✗	✗	✗	✗
FaceShifter [76]	GAN	✗	✓	✗	✗	✗	✗
FaceSwap [77]	AutoEncoders	✗	✓	✗	✗	✗	✗
DFaker [78]	Not Disclosed	✗	✓	✗	✗	✗	✗
FaceSwap-GAN [79]	GAN	✗	✓	✗	✗	✗	✗
DiscoFaceGAN [80]	GAN	✗	✓	✗	✗	✓	✗
StyleGAN [81]	GAN	✗	✗	✓	✗	✗	✗
StyleGAN2 [82]	GAN	✗	✗	✓	✗	✗	✗
StarGAN [83]	GAN	✗	✗	✗	✓	✗	✗
Face2Face [84]	Graphics	✗	✗	✗	✗	✓	✗
NeuralTextures [85]	GAN	✗	✗	✗	✗	✓	✗
Every Body Dance Now [86]	GAN	✗	✗	✗	✗	✗	✓
Impersonator++ [87]	GAN	✗	✗	✗	✗	✗	✓
ClipSwap [88]	GAN	✗	✓	✗	✗	✗	✗
DiffSwap [89]	Diffusion	✗	✓	✗	✗	✗	✗
DiffFace [90]	Diffusion	✗	✓	✗	✗	✗	✗
DeepFaceLive [91]	GAN	✓	✗	✗	✗	✗	✗
Deep-Live-Cam [92]	GAN	✓	✗	✗	✗	✗	✗
FaceFusion [93]	GAN	✗	✓	✗	✗	✗	✗
ComfyUI-InstaSwap [94]	GAN	✗	✓	✗	✗	✗	✗
LivePortrait [95]	GAN	✗	✗	✗	✗	✓	✗
Reshot [96]	Not Disclosed	✗	✗	✗	✗	✓	✗
DeepFake-Web [97]	Not Disclosed	✗	✓	✗	✗	✗	✗

latent representation, then decodes the latent representation back to an image as shown in Fig. 2.4. An autoencoder is made up of two networks called *encoder* and *decoder*, respectively. A shared encoder can be used to generate DeepFake between Person A and Person B by training an autoencoder with a dataset containing images of both individuals. The shared encoder extracts latent representations from images of Person A and Person B, ensuring a consistent encoding process. By combining the shared encoder with separate decoders for each person, DeepFake can be generated that swaps the appearance of Person A with Person B and vice versa [100], as shown in Fig. 2.5. Table 2.2 shows the main research work in the research field of autoencoders.

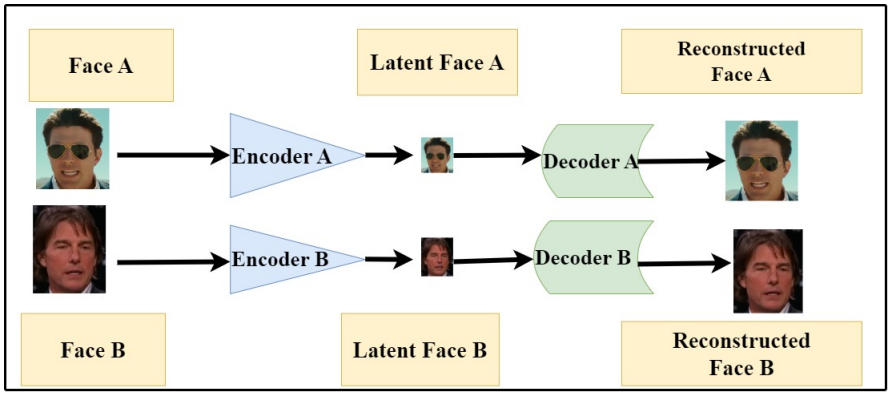


Figure 2.4: AutoEncoder A with an image of Person A as input (top row). AutoEncoder B with an image of Person B as input (bottom row).

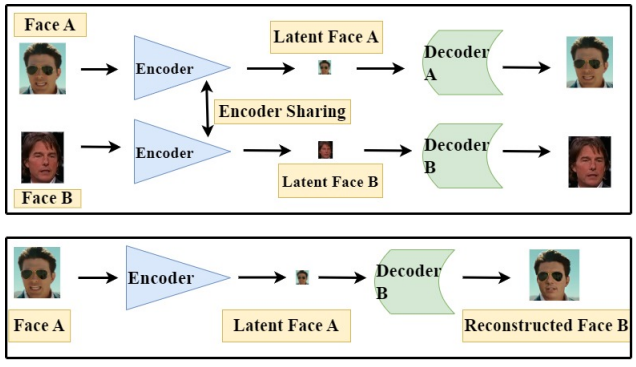


Figure 2.5: DeepFake videos are created by using two networks. Two networks shared the same encoder but two different decoders in the training step (top step). An image of face A is encoded with a shared encoder and decoded with decoder B to create a DeepFake (bottom step).

Table 2.2: Summary of AutoEncoders.

Reference	Type	Application	Research Contribution
[98], 1986	Basic AutoEncoders	Image Generation	This study introduced the concept of autoencoders.
[101], 2008	Denosing AutoEncoders	Image Generation	These autoencoders were built to generate clean images from noisy images.
[102], 2011	Sparse Autoencoder	Image Generation	These were introduced to improve the performance of base autoencoders, by use of regularization.
[103], 2011	Convolution AutoEncoders	Image Generation	These introduced convolution process to encode and decode data.
[104], 2011	Contractive AutoEncoders	Image Generation	These were developed to improve the performance of base autoencoders by improvement of the loss function.
[105], 2013	K-sparse Autoencoders	Image Generation	These were built to scale up sparsity, to handle large scale problems.
[106], 2013	Variational Autoencoder	Image Generation	This study introduced Gaussian distribution to learn the underlying distribution of data.
[107], 2015	Conditional Variational Autoencoder	Image Generation	This study improved variational autoencoders by introducing conditional variables.
[108], 2015	Adversarial Autoencoders	Image Generation	These were developed by combining the idea of autoencoders and using the adversarial training process of GAN.
[109], 2015	Variational Fair Autoencoder	Image Generation	These were developed to make sure that autoencoders are invariant to sensitive demographic information.
[110], 2016	Pixel Variational Autoencoder++	Image Generation	It was developed to combine the strengths of Pixel-CNN [111] and variational autoencoders.
[112], 2021	Kernal method-based Autoencoders	Image Generation	It is a type of autoencoder that employs kernel methods for non-linear feature extraction and data representation. It leverages kernel functions to implicitly map data into a higher-dimensional space, facilitating the capture of complex relationships and patterns within the data.
[113], 2022	Masked Autoencoders	Image Generation	Masked autoencoders used transformers, and they can generate full images by training only on parts of images.
[114], 2022	Videomae	Video Generation	This study is inspired by masked autoencoders to produce videos rather than images.

2.1.3.3 *Generative Adversarial Networks*

Generative Adversarial Networks (GANs) were first introduced in 2014 [115]. It is a deep learning architecture. A generative adversarial network is comprised of two deep neural networks. One neural network is called a generator, and the other is called a discriminator. In the process of creating fake images using GANs, the generator and discriminator networks collaborate and compete to improve the overall performance of the model. The generator contains layers that transform random noise or a given image into a realistic fake image. It aims to deceive the discriminator by capturing visual characteristics from the training data. The discriminator, consisting of layers

for feature extraction and classification, assesses the authenticity of the generated images [116].

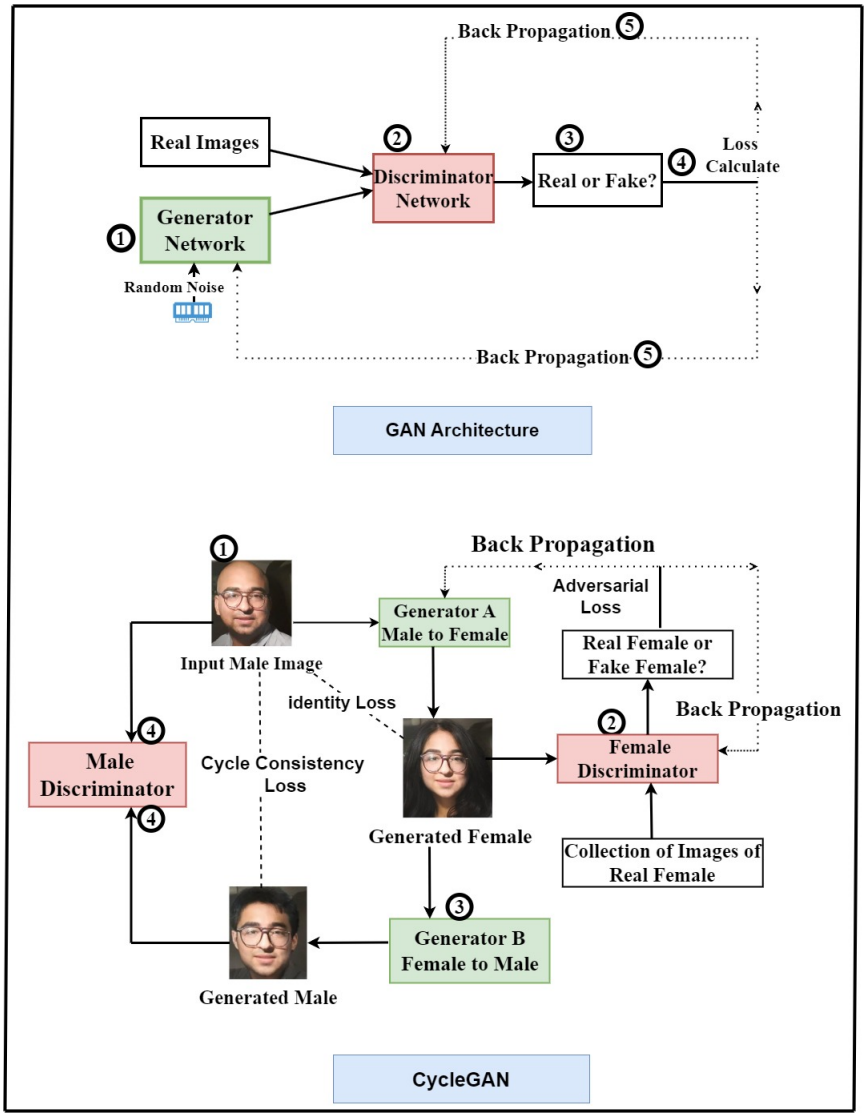


Figure 2.6: CycleGAN for DeepFake Generation by Performing Gender Swap from Male to Female

During training, random noise is fed into the generator, producing initial fake images. The discriminator provides feedback on their authenticity, and both networks update their weights using the back-propagation process. This adversarial interplay continues iteratively, refining the generator's ability to produce high-quality fake images and the discriminator's ability to differentiate real from fake [117]. Fig. 2.6

shows the process of transforming male images into female images using CycleGAN [118]. CycleGAN contains two generators and two discriminators. CycleGAN operates in three main steps: first, a picture of a male is converted into a picture of a female; second, a female discriminator evaluates the authenticity of the generated female image by comparing it with real female images, utilizing adversarial loss to ensure that the generated images are indistinguishable from real images. Additionally, identity loss is used to ensure that if an image is fed into the generator, the output retains certain properties of the original image, such as the background and other unchanged elements. This helps in preserving the consistency of these features during the transformation process. Third, the generated female image is then transformed back into a male image, which is compared to the original input male image to calculate the cycle consistency loss. The cycle consistency loss captures the intuition that if an image is translated from one domain to another and then back again, it should return to its original form. CycleGAN can also be used to convert a picture of a young person into a picture of an old person [119]. Table 2.3 summarizes the research progression in the field of GANs.

2.1.3.4 *Diffusion Models*

Diffusion models were first introduced in 2015 [137]. Diffusion models are based on the denoising Diffusion Probabilistic Model (DDPM), a parametric Markov chain that refines images by systematically eliminating noise. DDPM operates through two main processes: the forward process, where Gaussian noise is incrementally added to image data until it becomes random noise, and the reverse process, where noise is progressively removed in a step-by-step denoising procedure to generate high-quality samples [138]. The detailed process of DeepFake creation using DiffSwap [89] has been shown in Fig. 2.7.

Table 2.3: Summary of GANs.

Reference	Name	Application	Research Contribution
[115], 2014	Generative Adversarial Nets	Image Generation	Introduced GANs concept. This was the first study in this domain.
[120], 2016	DCGAN	Image Generation	This study introduced structural upgrades and hyper-parameter tuning for base architecture [115].
[121], 2017	WGAN	Image Generation	This study introduced a new loss function to make the previous models more stable.
[122], 2017	ProGAN	Image Generation	This study introduced a new training methodology to improve the performance of the network.
[123], 2017	Pix2Pix	Image-to-Image Translation	It is image-to-image-conditional GAN. It can translate images from one domain to another, given a paired image set in both domains.
[124], 2017	CycleGAN	Image-to-Image Translation	It is consist of two-generators and two discriminators. It does not require paired examples like Pix2Pix.
[125], 2017	DualGAN	Image-to-Image Translation	It belongs to the same family of GANs as CycleGAN, but DualGAN facilitates the simultaneous training of primal and dual GANs, enabling the utilization of a reconstruction error term for generating informative feedback signals.
[126], 2017	DiscoGAN	Image-to-Image Translation	Compare to CycleGAN, DiscoGAN has two reconstruction losses.
[127], 2017	StackGAN	Text-to-Image Generation	It divided the problem into subproblems. The first stage focuses on structure, and the second stage focuses on quality and fine details [128].
[129], 2018	PS2MAN	Image-to-Image Translation	It introduced a novel face photo-sketch synthesis framework based on GANs. It employed multi-adversarial networks with adversarial supervision provided to hidden layers. This approach improves synthesis results. It was the first detailed study of adversarial networks specifically for face photo-sketch synthesis.
[130], 2018	STACKGAN-V2	Text-to-Image Generation	It is enhancement of StackGAN. It improved the overall quality of output [128].
[131], 2019	DualAttn-GAN	Text-to-Image Generation	It had two attention modules, one for handling purely related to image and the second was related to creating a relation between text and image [128].
[81], 2019	StyleGAN	Image Generation	It improved the quality of images generated by ProGAN.
[132], 2020	DM-GAN	Text-to-Image Generation	In DM-GAN a memory-based architecture was introduced.
[133], 2021	Alias-Free GAN	Image Generation	It focused on texture property of Images.
[134], 2021	CSGAN	Image-to-Image Translation	It also belongs to the family of CycleGAN, DiscoGAN, and DualGAN. It introduced a new loss function to improve performance.
[135], 2022	DF-GAN	Text-to-Image Generation	DF-GAN enhanced text-to-image synthesis with a streamlined one-stage backbone network.
[136], 2022	LD-GAN	Text-to-Image Generation	This study proposed a novel framework for enhancing robustness against adversarial robustness in GANs.

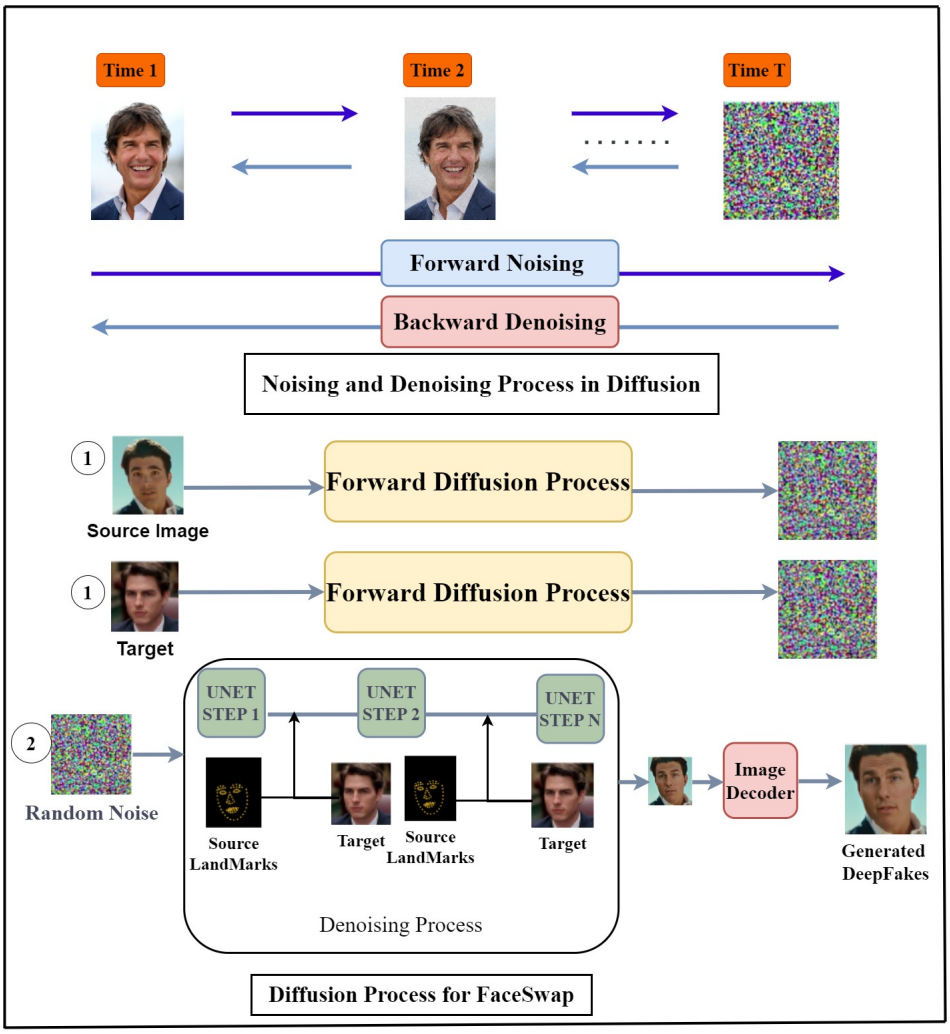


Figure 2.7: Noising and De-noising Process for Creating FaceSwap Using Diffusion Model

In the first step, the forward process is applied to images to understand data distribution. In the second step, noise is converted back to an image using the facial landmarks of the source face and the identity of the target face using U-NET [139]. Diffusion models can be used for image generation, image-to-image translation, text-to-image generation, and 3D data generation, as shown in Table 2.4.

Table 2.4: Summary of Diffusion Models.

Reference	Application	Research Contribution
[137], 2015	Image Generation	This is considered to be one of the first papers in this field. This study employed an iterative process to gradually dismantle and then reconstruct the structure within a data distribution.
[140], 2020	Image Generation	This is considered to be one of the groundbreaking studies in diffusion models. This research worked on improving the quality of images. This research was inspired by [137].
[141], 2021	Image Generation	This study was conducted to demonstrate that diffusion models can outperform GAN models in unconditional image synthesis.
[22], 2021	Text-to-Image Generation	The research introduces a simplified text-to-image generation (DALL-E). This method employed a transformer that handles text and image tokens as a unified data stream. This approach demonstrated competitive performance without the need for domain-specific modeling assumptions or auxiliary data, even under zero-shot evaluation scenarios.
[142], 2022	Text-to-Image Generation	This research introduced a diffusion model named Imagen. The research contribution of this work lies in harnessing large pre-trained language models such as T5 to effectively encode text for image synthesis.
[143], 2022	Text-to-Image Generation	In this study stable-diffusion was introduced. This study advances image synthesis with Latent Diffusion Models (LDMs), enhancing efficiency and quality through training in latent space and the incorporation of cross-attention layers. It significantly reduces computational demands while achieving competitive performance on various image-related tasks.
[144], 2023	Text-to-Image Generation	This study introduced a conditional control for the image generation process in the diffusion models.
[145], 2023	Text-to-Image & Image-to-Text Generation	This study introduced a multi-task network.
[146], 2023	3D Generation	In this study 3D data is being generated by using 2D images for training.
[147], 2023	Image-to-Image Translation	This study provided users with more control over the image editing process.

2.2 Recent Advances

This section explores recent developments in compressed DeepFake detection and datasets and provides a new taxonomy for these studies.

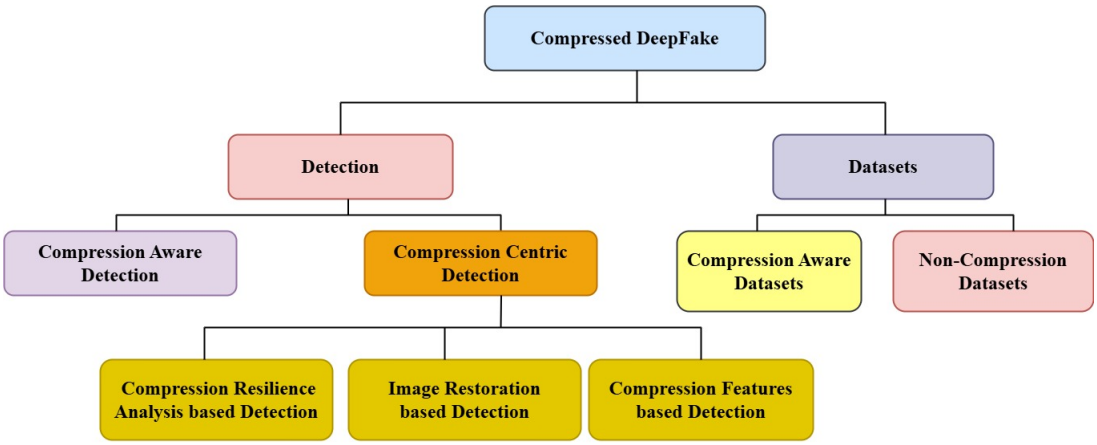


Figure 2.8: DeepFake Classification

2.2.1 DeepFake Datasets

This section focuses on the examination of datasets developed by researchers to assess DeepFake detection systems. The size and quality of these datasets play a crucial role in combating the presence of DeepFake videos in real-world scenarios. These datasets have been created from a diverse range of sources, tools, experimental setups, and demographic representations. Categorize DeepFake datasets they can be divided into two groups based on whether they provide information about compression levels, as shown in Fig. 2.8.

2.2.1.1 Compression Aware Datasets:

This section discusses datasets that explicitly provide information about the compression levels applied to their data. Compression aware datasets include *FaceForensics++*, *DeepFAKE MNIST+*, *Google & JigSaw-DFD*, *DF-Platter*, *eKYC-DF* as shown in Table 2.5.

The *FaceForensics++* [148] dataset is categorized into four subdivisions, named as DeepFakes, Face2Face, FaceSwap, and Neural Texture. Among these, Face2Face and Neural Texture subsets are designed for facial reenactment. DeepFakes and FaceSwap

Table 2.5: Compression Aware Datasets

Reference	**Contribution**	**Limitation**
FaceForensics++ [148]	Provides DeepFake videos of facial swapping and facial reenactment generated through autoencoders and GANs. Allows researchers to download videos of different compression levels independently.	Do not provide videos generated through diffusion models.
DeepFAKE MNIST+ [149]	Provide a large dataset of 20000 videos for DeepFake detection models.	Only contain facial swapping videos. Not publicly available. Do not provide videos generated through diffusion models.
DFD [150]	Provide a big dataset with three compression levels.	Only contains facial swapping videos. More than 200 GB is required to store the whole dataset. Do not provide videos generated through diffusion models.
eKYC-DF [57]	Tested on liveness detection system.	Not publicly available. Do not provide videos generated through diffusion models.
DFPlatter [151]	Provide a large, diverse multiface dataset based on autoencoders and the GAN method.	Did not utilize diffusion models. A large Storage of 417 GB is required to download and use the dataset. Subjects only belong to one country
eKYC-DF [57]	Tested on liveness detection system.	Not publicly available. Do not provide videos generated through diffusion models.

subsets are tailored for facial replacement. The dataset consists of 1,000 real videos sourced from YouTube and 4,000 fake videos generated through DeepFake generation tools. To ensure diversity, the videos were generated at three different compression levels: c0, c23, and c40. Additionally, this dataset encompasses videos with multiple faces, providing a comprehensive resource for DeepFake detection research. A total storage of 542 GB is required to store all videos available in different compression levels. To download videos with compression levels of 23 and 40, less than 50 GB is required.

DeepFAKE MNIST+ [149] dataset is built to test the robustness of the liveness

detection system. It contains 10,000 real and 10,000 fake videos. This dataset consists of forgery videos depicting 10 actions: blink, open mouth, yaw, nod, right head tilt, left head tilt, look up, and smile, surprise, and embarrassment. It also contains videos in three compression levels: 0, 23, and 40. It only contains videos that contain facial reenactments. The source videos for this dataset are collected from VoxCeleb1 [152]. The driving videos are collected from the ADFES dataset [153] and also recorded with the help of a volunteer. This dataset is not publicly available anymore.

DFD [150] is the dataset that was published by Google and Jigsaw. It has 3,068 fake and 363 real videos. It is available in three different image compression levels: 0, 23, and 40. A total of 28 actors, with their consent, were part of this dataset.

DFPlatter [151], is a comprehensive simulation of real-world deepfake scenarios, incorporating diverse deepfake variations, including those with different compression levels 0, 23, and 40. Annotated for various facial attributes, this dataset encompasses both low-resolution and high-resolution DeepFake, single-subject, and multiple-subject instances of Indian ethnicity. A rigorous benchmark analysis underscores the challenges in detecting low-resolution and multiple-subject DeepFake videos, aiming to bolster the capabilities of deepfake detection algorithms for real-world applications, particularly in scenarios involving compressed data. A total storage of 417 GB is required to download all videos of all compression levels. It contains 100 real videos and 5,040 fake videos. Real videos are sourced from YouTube.

eKYC-DF [57] dataset was also developed to test the robustness of the liveness detection system. The real videos for this dataset were collected from the DeeperForensics-1.0 [154] dataset. It contains 760 real videos and 228,000 fake videos. Fake videos were generated using three face-swapping systems, including SberSwap [155], SimSwap [156], and FaceDance [157]. This dataset contains three compression levels: 0,

23, and 40. This dataset is not publicly available.

2.2.1.2 Non-Compression Datasets:

This section focuses on datasets that do not offer explicit information regarding the compression levels used. Non-compression datasets include *FakeAVCELEB*, *DeepfakeTIMIT*, *DeeperForensics-1.0*, *WildDeepfake*, *Celeb-DF-V2*, *KoDF*, *ZoomDF*, *DeePhy*, *FFIW* ,*ToonTalker* and *DFDC* as shown in Table 2.6.

FakeAVCeleb [158] is a novel dataset featuring diverse real videos, integrating cloned voices and facial reenactment for deepfake generation. Leveraging advanced techniques like Faceswap [159], FSGAN [160], and SV2TTS [161], it facilitates the creation of both DeepFake videos and audios. Wav2Lip [162] further enhances the DeepFake videos based on the synthesized audio, enabling comprehensive training of detectors for both video and audio deepfake datasets.

DeepfakeTIMIT [163] consists of 320 real videos and 640 fake videos generated using the VidTIMIT database [164] with the help of a GAN-based face-swapping algorithm. This data set has videos of two qualities based on resolution: one is referred to as Low Quality (LQ) with a resolution of 64x64, and the second is High Quality (HQ) with a resolution of 128x128. These videos contain only 32 individuals.

A dataset called *DeeperForensics-1.0* [154] comprising 50,000 real and 10,000 fake videos. This data set only contains face-swapping videos. To prepare this dataset, services of 100 paid actors were taken to prepare 50,000 real videos. The videos of actors are recorded in the resolution of 1920x1080. Fake videos are generated by manipulating 1,000 videos collected in the FaceForensic data set with the help of recorded videos of actors and different distortion levels. Variational auto-encoder [165] acted as the backbone to prepare fake videos. A total storage of 284 GB is required to store all videos of this dataset.

A dataset called *WildDeepfake* [166] has been introduced to address the limitations of current DeepFake detection systems when it comes to detecting DeepFake on social media platforms. The researchers argue that existing detection systems are primarily validated on datasets generated by popular face swap applications, which do not accurately represent the diverse range of DeepFake found on social media. Furthermore, these datasets often exhibit obvious artefacts such as jitters, blurs, and other peculiar visual elements, whereas DeepFake encountered on social media are typically created using a combination of different software and techniques, with more effort and time invested in each video. To address these challenges, the researchers have collected a new data set comprising 707 real and 707 corresponding fake videos from the internet. The dataset contains a total of 7,314 face sequences, with 3,805 being real and 3,509 being fake, and it encompasses a vast number of 1,180,099 images.

The *CELEB-DF-V2* [3] dataset presents researchers with a valuable resource due to its large data size and high visual quality. Comprising 5,639 fake videos and containing a total of 2 million frames, the dataset also includes 890 real videos with a standardized duration of 13 seconds and a frame rate of 30 frames per second. The videos featuring 59 celebrities were sourced from YouTube [167] to ensure diversity across gender, age, and ethnic groups. To improve visual quality, various techniques were employed, including increasing the layers in encoders and decoders to achieve a resolution of 256x256. Colour transfer algorithms were implemented to enhance colour correlation, and a refined mask generation technique was used to better cover the donor face with the synthetic or target face. Additionally, temporal flickering was introduced through the correlation between facial landmarks. The evaluation of the CELEB-DF-V2 dataset employed the Mask-SSIM score as a primary parameter. While the Structural Similarity Index measure (SSIM) [168] is influenced by both changes in visual quality and changes in identity, it was found that visual quality

played a more decisive role in obtaining the best SSIM scores within this dataset. Notably, CELEB-DF-V2 poses significant challenges for many existing DeepFake detection algorithms. When compared to other datasets, the average detection accuracy for Celeb-DF was found to be 56.9 percent, while the accuracy for other datasets ranged from 64 percent to 80 percent.

The KoDF [169] stands as the most expansive dataset, boasting an impressive 2.3 terabytes in size. It was meticulously crafted, exclusively sourcing real videos from the Korean population. The creation process involved the integration of face-swapping and face-reenactment techniques, incorporating a sophisticated suite of six distinct models. Moreover, the development phase accounted for various perturbations, encompassing factors such as compression, hue, and other critical elements. The dataset comprises a rich assemblage of 62,000 authentic clips alongside an additional 175,000 simulated clips.

Zoom-DF [170] allows for the smooth inclusion of synthetic participant images in virtual meetings. Given the increased reliance on video conferencing platforms like Zoom [171], Webex [172], and Google Meet [173] due to the coronavirus 2019 (COVID-19) pandemic, the risk of unauthorized intrusion, known as Zoombombing, has become a pressing concern. This threat is further exacerbated when combined with deepfake technology, which can manipulate participants' images, voices, and movements, creating a challenge in discerning real from fake information during meetings. This dataset, customized for remote meetings, focuses on participant movement manipulation, using Avatarify [174] and the First Order Motion Model (FOMM) [175] to create 400 synthetic videos from images from the CelebA data set, employing the Avatarify Python tool.

The Deepfake Phylogeny dataset [176] encompasses 5,040 deepfake videos created through three distinct generation techniques. It includes 840 one-time swapped

DeepFake, 2,520 two-times swapped DeepFake videos, and 1,680 three-times swapped DeepFake, totalling over 30 GB. The dataset was crafted over 1,100 hours, employing 18 GPUs with a combined memory of 1,352 GB. Furthermore, the benchmark analysis conducted on the DeePhy dataset, utilizing six deepfake detection algorithms, emphasizes the importance of advancing research in deepfake model attribution and generalizing the process across various deepfake generation techniques.

FFIW [177] contain multiple faces per frame. It contains three people per frame on average. It contains 10,000 real and 10,000 fake videos. The average length of videos in this dataset is 12 seconds. The total length of this whole dataset is 33 hours.

ToonTalker [178] dataset focuses on cross-domain face reenactment, involving the synchronization of a cartoon image with the video of an actual individual and vice versa. To generate this dataset, a customized method was specifically devised, ensuring the successful alignment of these diverse visual domains.

DFDC [179] has 23,654 real and 104,500 fake videos. It only contains face-swapping videos. Real videos were recorded by 3,426 paid actors with H.264 compression format [180]. Fake videos were generated by using six tools. The resolution of this dataset is 256x256. A total storage of 472 GB is required to store all videos of this dataset.

2.2.2 Compressed DeepFake Detection

In this section, we briefly summarize the most recent works on compressed DeepFake detection into two main detection categories, as shown in Fig. 2.8. The classification is done at two levels. At the first level, the detection studies are sorted based on the focus of their research on compression. At the second level, studies are sorted based on the depth of their analysis of compression in DeepFake detection models.

Table 2.6: Non-Compression Datasets

Reference	Contribution	Limitation
FakeAVCeleb [158]	Provide a dataset for both audio and video DeepFake detection models.	Researchers did not collect real videos by themselves but relied on their own already-present dataset. Does not provide facial reenactment DeepFake videos. Did not utilize diffusion models.
DeepfakeTIMIT [163]	Provides a high-quality and low-quality dataset based on Image resolution.	Does not provide facial reenactment DeepFake videos. Contains videos with highly clear visual artefacts of DeepFake. Did not utilize diffusion models.
DeeperForensics-1.0 [154]	Provide a dataset containing 50,000 real videos prepared by 100 paid actors. It contains fake videos with 32 perturbations	A large Storage of 284 GB is required to download and use the dataset. Did not utilize diffusion models.
WildDeepfake [166]	Propose a dataset that contains fake video downloads from the internet instead of being created in the academic lab.	Videos are delivered to research in image form instead of video form. Did not utilize diffusion models.
CELEB-DF-V2 [3]	Provide a large challenging facial swapping dataset short videos.	Do not provide facial reenactment DeepFake videos. Did not utilize diffusion models.
KoDF [169]	It is a dataset that contains perturbations, encompassing factors such as compression and hue, etc.	Only contains subjects from one country. Provide a dataset of the size of 2.3 TB, which is not possible to use in a resource-constrained environment. Did not utilize diffusion models.
Zoom-DF [170]	It is developed to defend against DeepFake videos in live scenarios.	This dataset is not publicly available. Did not utilize diffusion models.
FFIW [177]	Contains multi-person DeepFake videos.	Only contains face-swapping videos. Did not utilize diffusion models.
Deepfake Phylogeny [176]	Provide a dataset prepared by applying multiple times DeepFake techniques.	Does not contain facial reenactment video. Did not utilize diffusion models.
ToonTalker [178]	Provide DeepFake videos with cross-domain scenarios.	Final output videos are of cartoon characters. Did not utilize diffusion models.
DFDC [179]	Dataset is created from videos that are being shared by the public.	This dataset is not challenging as it contains a lot of DeepFake traces because of no standard in video recording. Contains videos with highly clear visual artefacts of DeepFake. Did not utilize diffusion models.

2.2.2.1 Compression Aware Detection

In Compression-Aware Detection, we have categorized studies that conducted experiments on compressed videos but did not treat compression as a primary focus. While these studies incorporated compressed data into their evaluation, they did not explore compression as a core issue, nor did they attempt to solve compression-related challenges. Additionally, they provided limited or no analysis regarding how compression influenced the performance of detection models or whether detection features or performance were retained after compression. The summary of these studies is shown in Table 2.7.

The study carried out in [181], has introduced temporal optical flow features to detect DeepFake videos. Optical flow represents the movement of intensities in the image plane [182]. Deep convolution neural network ResNet-50 [183] was used as the backbone model. Experiments were conducted on different compression levels on the FaceForensics++ dataset. The accuracy of the model dropped by almost 20 percent when the compression level increased from 0 to 40.

The method proposed in [184] was based on spatial key point descriptors extracted from different regions of the face. Experiments were conducted on SIFT, SURF, FAST, BRIEF, ORB, and A-KAZE descriptors [185]. The study found that using the *FAST & BRIEF* feature descriptors yielded the most effective results as compared to other descriptors. In this study, a random forest model [186] was used for DeepFake detection. This method also suffered from a decrease in performance when subjected to higher compression levels. This approach is computationally less expensive than deep learning-based methods.

The study conducted in [187] focused on using the spatial depth [188] of the image calculated using a custom convolution neural network as a feature of the detection of DeepFake videos. This method also suffered from a decrease in performance when

subjected to higher compression levels. Swin transformer [189] was used as the classification model.

In [45], a novel image representation called Face X-ray was introduced to detect DeepFake videos. A face X-ray is a grayscale image specifically designed to reveal the boundary noise between the target and source face. The focus of their work was on addressing the blending step, which occurs during the face swap process and often leaves visible blending boundaries. In this approach, a neural network model called HRNET [190] was used. This approach was evaluated using the FaceForensics++ dataset. This method also suffered from a drop in performance due to compression.

In [191], a two-stream ensemble method was used to detect DeepFake videos by capturing spatial as well as temporal inconsistencies. The frame-level stream is used to learn the spatial inconsistencies from the phase component of the Fourier transform [192], while the temporal-level stream is used to extract temporal inconsistencies by calculating the temporal correlation feature on consecutive frames. The method was tested on various public datasets, including Celeb-DF and FaceForensics++. Experiments on various compressions led to a decrease in performance on different subsets of FaceForensics++.

In [193], RGB and high-frequency SRM noise features [194] are combined to classify videos using XceptionNet [195] and the attention module. The main feature of this study was to conduct a three-class classification instead of a binary classification. Analysis was conducted on FaceForensics and Celeb-DF-V2 datasets. A degradation of 13 percent in accuracy was experienced when the compression factor increased from 0 to 40.

Table 2.7: Compression Aware Detection

Reference	Contribution	Limitation
[181]	Temporal Optical-Flow features were adopted for the first time for the detection of DeepFake videos	Spatial Features were not Investigated Computationally Expensive because of using a large number of frames No analysis was provided of compression on optical flow features
[184]	Features Points descriptors were used to detect DeepFake videos	Temporal Analysis was not performed No analysis was provided of compression on features Used a large number of frames for training and testing
[187]	Introduced a custom convolution neural network for extracting depth feature	Depth feature extraction module is not tested on complex environments such as extreme illumination and face occlusion.
[45]	Image representation was introduced to focus on blending features introduced during face swap	No analysis was provided of compression on features Did not consider features introduced due to DeepFake generation models
[191]	Introduced Frequency based Non-RGB features	Computationally Expensive because of using 100 to 200 frames per video
[193]	Handled DeepFake detection as a multi-class problem instead of a binary-class problem	Did not customize noise-based image forensics features for DeepFake detection

2.2.2.2 Compression Centric Detection

The studies in this group focus on compression as a central aspect of their research. These studies are further divided into three groups. These studies aim to address the challenges introduced by video compression by providing a detailed analysis of the robustness of detection features under different compression settings, restoring video quality, and using properties of compressed videos

(i) Compression Resilience analysis based Detection

This group includes studies that conducted experiments on compressed videos and provides insights into whether detection features or performance were preserved after

compression or not, as shown in Table 2.8.

The work of [41] focused on the movement of the human face. This is a person-specific approach. Through this approach, it can be decided whether a video belongs to a specific person or not. Compared to other studies, this approach only used real videos for training. This method used a 3D Morphable model (3DMM) [196] to represent the human face in each frame of the video. The approach used VoxCeleb [197] dataset for training and DFD, DFDC, and Celeb-DF datasets for testing to perform cross-dataset analysis. This model was subjected to both low and high-quality videos. This model also showed degradation in performance on compressed videos.

In [198], the symmetrical nature of the face was utilized to detect DeepFake videos. The two-branch architecture has been used in this approach with DRN [183] as the backbone model. This approach was tested on FaceForensics++, Celeb-DF, DeepFake-TIMIT, DFD, and DFDC datasets. This method lacked generalization as it showed a performance drop of 4 to 40 percent on cross-dataset testing.

In [199], temporal geometrical features were calculated using the motion of facial muscles. The analysis was conducted in the spatial and temporal domains. Support vector machine (SVM) [200] has been used as a classification model for the spatial domain, while for the temporal domain, Gated recurrent unit (GRU) [201] has been used. Analysis was conducted on the FaceForensics++ and FaceShifter [76] datasets. Detection accuracy of 87 to 90 percent was recorded on different subsets of the FF++ dataset, while the area under the curve was between 92 to 97 percent. This study used 60 frames per video to train and test the DeepFake classification model. Therefore, it is computationally expensive.

The study conducted in [202] employed multilevel statistical analysis on the texture property of images to detect DeepFake videos. Experiments were conducted on colour

models of (Red, Green, Blue) RGB, (Hue, Saturation, Value) HSV, and YCbCr [203]. In YCbCr, Y represents luminance, and C represents chrominance. The R, G, B, Y, and V channels were found to provide the most useful information for investigating DeepFake videos based on the texture property of the image. Support Vector Machine (SVM) [200] was used to perform classification. It was observed that a performance decrease of 33 percent in accuracy occurred when the image compression level was changed from raw to c40 on the FaceForensics++ dataset. Additionally, experiments are conducted on the Celeb-DF dataset.

In the study [44], the focus was on utilizing the statistical properties of photo-response non-uniformity (PRNU) in spatial and spectral representation as a method for the detection of DeepFake videos. PRNU refers to the imaging sensor noise that can be considered a unique fingerprint of a camera. The PRNU method was evaluated on three datasets: FaceForensics++, DeepFake Detection, and DeepfakeTIMIT. This approach was also sensitive to compression. The performance of the PRNU detector varied between 88 percent and 63 percent across different compression settings and datasets.

In [204], an unsupervised method was introduced for detecting DeepFake videos. Two different noise methods were used, including Photo-Response Non-Uniformity (PRNU) and noise print [205]. In this approach, a two-step unsupervised learning-based approach has been used. In the first stage, the clustering was performed based on the source camera, and in the second step, clustering was performed to differentiate between real and fake videos. K-Mean [206] was used for the clustering. This approach has been tested on FaceForensics++, DFD, and DFDC. This approach also showed a decrease in performance under compression.

In the study [43], a combination of local and global deep features was employed to detect facial reenactment. Four local branches were trained on different facial

regions, while a fifth branch was trained on the full face, utilizing ResNet-18 [183] as the backbone network. Experiments were conducted on the Face2Face dataset across three image quality levels: c0, c23, and c40.

In [40], spatial-temporal fusion was achieved by the usage of different 3DCNN [207] networks, which included 3D ResNet [208], 3D ResNeXt and I3D [209]. Experiments were conducted on the FaceForensics++ dataset with a compression factor of 40. Models showed a detection rate of between 83 and 87 percent when trained and tested on the same subset dataset, but performance dropped below 75 percent in cross-subset detection.

In [210], two types of artefacts were used to classify DeepFake. The first one is *intrinsic artefacts*, which are introduced during the DeepFake generation process due to up-convolution and up-sampling caused by encoders and decoders. Second is *extrinsic artefacts*, which are introduced during the blending step in the DeepFake creation process. This method was evaluated on several datasets, including Celeb-DF, FaceForensics++, UADFV, DeepfakeTIMIT, DFD, and DFDC. To assess generalization, the model was trained on Celeb-DF and was tested on all other datasets. The average Area Under the Curve (AUC) is found to be 0.90, ranging from 0.99 to 0.81 across different datasets. The model was also trained on FaceForensics++ and tested on all other datasets, resulting in an average AUC of 0.90, ranging from 0.99 to 0.77. Furthermore, the impact of compression factors was also evaluated on the performance of this method. It was observed that as compression factors increase from 5 to 30, the AUC decreases from 0.99 to 0.88. Additionally, the method exhibits reduced performance when faced with noise, resizing, and Gaussian blurring.

(ii) Image Restoration based Detection

This group encompasses studies that focus on image restoration as a critical first

Table 2.8: Compression Resilience based Detection

Reference	Contribution	Limitation
[41]	Proposed a approach that only use real videos for training	It is Person specific approach
[198]	Introduced facial symmetry features to detect DeepFake	Computationally expensive because of using 50 frames per video
[199]	Introduced an approach to calculate facial movement of a face by using facial landmarks of a video in a video	Did not explore Spatial cues for detection of DeepFake videos
[202]	Introduced an approach to detect DeepFake videos by using multiple colour systems	Computationally expensive because of using 20 frames per video
[44]	Introduced statistical approach to detect DeepFake videos by using spatial and spectral representation of noise in image	This approach did not perform well as compared to Deep learning methods
[204]	Proposed a unsupervised approach for detect DeepFake	Computationally expensive because of using all frames in a video
[43]	Proposed an approach use of a multi-scale approach to detect facial reenactment DeepFake	Did not test the approach on Face-Swapping DeepFake videos
[40]	Proposed use to 3D-CNN for DeepFake detection	Computationally expensive because of using more than 200 frames per video
[210]	Proposed an approach to detect DeepFake by using traces left by DeepFake generation models and during the post-processing step of the face blending step of DeepFake generation	Computationally expensive because of using every tenth frame in a video

step before feature extraction and classification. By restoring the quality of compressed videos, these studies aim to enhance the effectiveness of subsequent video classification tasks, as shown in Table 2.9.

In [211], a Spatial Restore Detection Framework (SRDF) was introduced to improve the detection performance for low-quality videos. Depth [188] and restored texture [212] features were used in this study. Compression analysis were performed on the FaceForensics++ dataset. It was found that detection performance dropped by 7 percent on lower-quality videos as compared to high-quality videos.

In [213], an approach was proposed to improve the detection of low-quality DeepFake using super-resolution (SR) [214] as a preprocessing step. Extensive experiments

were conducted on the FaceForensics++ (c40) DeepFake dataset. Super-resolution preprocessing improved the performance of the DeepFake detection pipeline by 2 percent. Super-resolution also decreases the requirement for the number of frames to train the model by five times.

In [215], a method called DF-UDetector was proposed to detect compressed DeepFake videos. This study used a U-NET [139] based GAN to enhance image quality. Support vector machine (SVM) and Bayes [216] models were used for classification. The proposed method is evaluated on multiple video datasets of DFDC, Celeb-DF, and FaceForensics++. To evaluate the performance of the model under diverse conditions three attack settings, of blur, noise, and compression, named weak, middle, and strong, were used. This method also faced a degradation in performance on all attack settings.

Table 2.9: Image Restoration based Detection

Reference	**Contribution**	**Limitation**
[211]	A model was introduced to restore textures features of compressed images	Computationally expensive because of using 32 frames per videos Add the additional computational cost of restoring images
[213]	A model was proposed to improve the accuracy of DeepFake detection models by restoring images	Computationally expensive because of using 50 frames per video Add the additional computational cost of restoring images
[215]	GAN-based method was introduced to restore images	Did not explore temporal features Add the additional computational cost of restoring images

(iii) Compression Features based Detection

This group encompasses studies that focus on video compression-based detection, utilizing the properties of compressed videos as shown in Table 2.10.

In [42], two new networks, Meso-4 and MesoInception-4, were introduced for DeepFake classification. The performance difference between the two models was minimal,

with only a 1 percent to 2 percent variation. Experiments were conducted across various compression levels, revealing a performance drop due to compression. Image aggregation techniques were employed to improve decision accuracy. Additionally, time-efficient experiments were carried out using a storage scheme of compressed videos.

In [39], a new approach was proposed to ensemble artefacts present in spatial and temporal streams to detect DeepFake in compressed videos. In this study, the compression video storage scheme was used to calculate the features of both the frame level and the temporal level features. The frame level branch was trained on semantic features, and the temporal level branch was trained on cumulative motion information of sub-parts of videos calculated through an external library of Caviar [217].

Table 2.10: Compression features based Detection

Reference	Contribution	Limitation
[42]	Introduced a new shallow CNN to detect DeepFake	Did not provide a performance comparison with Deep CNN
[39]	Used storage scheme of compressed videos to calculate both spatial-temporal features	Computationally expensive because of using all frames of videos to calculate temporal features Did not explore non-semantic features for enhancing the performance of Spatial branch of the proposed model

2.3 Concluding Remarks

This chapter provides a foundational overview essential to understand the limitation of existing studies. We begin by introducing key terminologies related to DeepFake videos. Next, we explore different types of DeepFake, supported by relevant examples. Following this, we present a summary of the DeepFake generation process. This discussion first outlines the overall workflow before offering an in-depth review of

DeepFake generation models, incorporating insights from available generation tools and recent research advancements. In the last section, we provide a comprehensive literature review with a new taxonomy. In the literature review, we focused on the DeepFake detection method and datasets.

Through our literature review, we have identified several research limitations in deepfake detection and available datasets. Most existing datasets do not adequately address social media-based image perturbations, diversity in video sources, or the legal implications of using online videos. Additionally, these datasets struggle to remain up-to-date as new DeepFake generation tools emerge, and their maintenance is often neglected after their initial release. In terms of detection methods, computational expense remains a significant challenge. Most approaches are highly sensitive to compression artefacts, predominantly trained on autoencoders and GAN-based architectures, and fail to adapt to evolving DeepFake generation techniques. The lack of regular updates further limits their effectiveness against newly developed manipulation methods. In the next chapter, we will discuss the proposed architecture.

Chapter 3

Proposed Architecture

In this chapter, we explain the different components of our proposed architecture, as shown in Fig. 3.1. The algorithm is detailed in Algorithm 1 for training the models. Our architecture contains five components. The five components include Data (i) Preprocessing, (ii) Data Augmentation, (iii) RGB Stream, (iv) SRM Noise Stream, and (v) Fusion Box. We have used a binary cross-entropy loss function to train both of our streams. This loss function is usually used in binary classification problems with deep learning models [218]. Early stopping if no improvement was observed for 15 consecutive epochs. L1 represents a loss of RGB stream, and L2 represents the loss of the residual stream.

3.1 Data Preprocessing

Data preprocessing is the first step of our architecture. Data preprocessing ensures effective data sampling. Data preprocessing performs three major tasks in our detection pipeline: frame selection, detecting and extracting facial regions, and resizing frames. A standard video often contains 30 frames per second in the Celeb-DF-V2

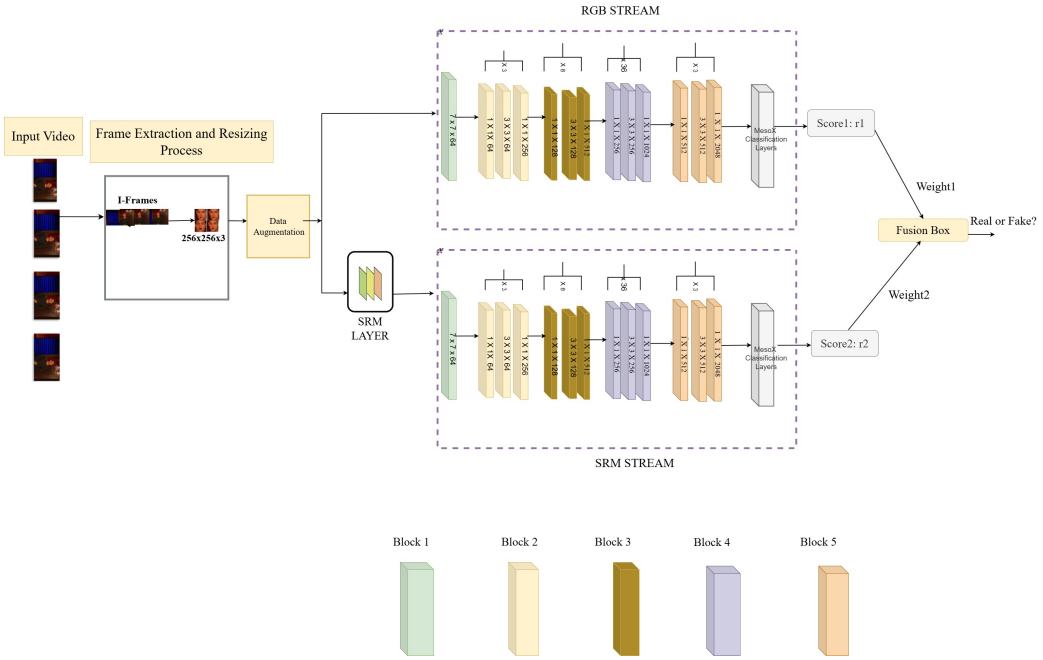


Figure 3.1: Overview of Proposed Architecture

Input:
Detected Facial Region in I-Frames
The noise features are extracted with SRM filters
The learning rate α_1 and α_2 are fixed at 0.00001 for S1 and S2, decayed by factor 0.1 every 20 epochs
The batch size b_1 and b_2 are fixed to 8 for S1 and S2
Output:
Trained RGB Stream θ_{S1}
Trained SRM stream θ_{S2}
while *Step* $<$ *Max Steps for S1 & and S2* **do**
  for $j = 1$ *to num iter* **do**
    $g_{\theta S_1} \leftarrow \nabla_{\theta S_1} \left(\frac{1}{b_1} \sum_{j=1}^{b_1} L_1 \right)$
    $\theta_{S_1} \leftarrow \theta_{S_1} + \alpha_{S_1} \cdot \text{Adam}(\theta_{S_1}, g_{\theta S_1})$
    $g_{\theta S_2} \leftarrow \nabla_{\theta S_2} \left(\frac{1}{b_2} \sum_{j=1}^{b_2} L_2 \right)$
    $\theta_{S_2} \leftarrow \theta_{S_2} + \alpha_{S_2} \cdot \text{Adam}(\theta_{S_2}, g_{\theta S_2})$
  end
end

Algorithm 1: The Procedure of Training the RGB Stream S1 and SRM Stream S2

dataset. From the Celeb-DF-V2 dataset, a 13-second standard video could potentially contain up to 390 frames. Sampling the large number of frames from each video in a resource-constrained environment will be directly proportional to training

time. To address this challenge, we have specifically opted for I-Frames to reduce the training time. I-frames also contain more spatial information; these frames in the video do not depend on any other frame in a video [39]. FFMPEG [219] was used for extracting I-Frames [220]. Only the facial region was extracted by using the dlib library [221] from each I-Frame because it is the only manipulated region in the whole frame. The I-frames that did not contain faces were dropped. In the last data preprocessing stage, facial regions were resized to dimensions of 256x256x3. This image dimension has been used in many previous studies [42, 39, 45].

3.2 Data Augmentation

After data preprocessing, we applied data augmentation to increase the model generalization. Data augmentation is a process of creating new data from already-present data. Data Augmentation helps to overcome overfitting by creating diversity in the dataset. Data Augmentation was found to be effective in multiple previous studies [39, 46], and we also found it to be effective through our own experiments. After experimenting with multiple augmentations, we applied data augmentation using flip, rotation, and zoom operations [222].

3.3 RGB Stream

The RGB Stream uses ResNet-152 as the backbone network for feature extraction, which uses a three-channel RGB image as input. This stream is used to learn semantic RGB artefacts, such as unnatural artefacts like blending boundary, distorted face which appears when a target face is blended with source face, and blurriness in the generated target face, as shown in Fig. 3.2. These features mainly appear during DeepFakes generation techniques that require fusing the target face to the

source face. Adam optimizer [223] with a 0.00001 learning rate is used in the RGB stream.

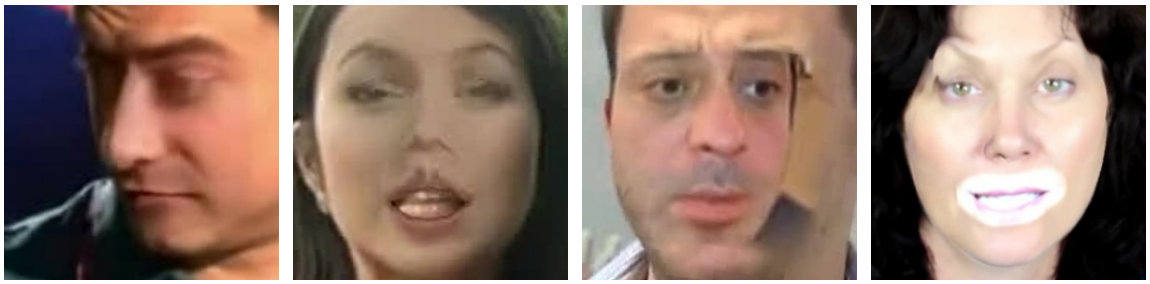


Figure 3.2: Semantic Features in Forged Videos

ResNet-152 was chosen after comparing its results to MesoNet [42], Xception [195], VGG-19 [224], EfficientNet-B4 [225] and EfficientNet-B7 [225]. A comparison of these Deep learning models is shown in Table 3.1. ResNet-152 contains five blocks of convolution operations [226]. These five blocks are used to extract features from images. At the end of ResNet-152, non-convolution layers of MesoNet were added.

Table 3.1: Comparison of Different Deep Learning Models

Ref, Relase Year	**DeepFake Specific**	**Pre-Trained**	**No. of Parameter Range**	**Best Top 1% Accuracy on ImageNet & Variant**
VGG [224], 2014	✗	✔	133 to 144 Million	VGG-19 71%
ResNet [183], 2015	✗	✔	10 to 60 Million	ResNet-152 77.8%
Xception [195], 2017	✗	✔	23 Million	Xception 79%
MesoNet [42], 2018	✔	✗	28000 Thousand	Not Applicable
EfficientNet [225], 2019	✗	✔	0.4 Million to 60 Million	EfficientNetB7 84.3%

ResNet was developed in 2015 to solve the vanishing gradient problem, which deeper networks often suffer, hindering convergence and making them challenging to train

effectively. ResNet's contribution lies in introducing residual connections, which allow information to bypass certain layers, enabling the network to learn residual functions. This architecture facilitates the training of extremely deep networks [227]. Structurally, ResNet consists of multiple residual blocks, each containing several convolutional layers and skip connections [183]. The skip connections enable the gradient to flow more effectively during backpropagation, mitigating the vanishing gradient problem [228]. ResNet has been used in many previous DeepFakes detection studies [43, 47]. ResNet has multiple variants. The number of parameters for ResNet lies between 10 and 60 million based on different variants of ResNet.

VGG architecture that was released in 2014 is notable for its emphasis on depth to improve image classification performance, with variants like VGG-11, VGG-13, VGG-16, and VGG-19 differing in layer count [224]. VGG networks consist of convolutional layers followed by max-pooling layers and fully connected layers. They use 3x3 convolutional filters for feature extraction and max-pooling for spatial downsampling, retaining crucial features. The Rectified Linear Unit (ReLU) activation function introduces non-linearity, aiding in modelling complex data relationships. VGG has parameters in the range of 133 to 144 million based on different variants. Despite its high performance, VGG's large number of parameters results in increased computational costs and potential overfitting. Nevertheless, its simplicity, versatility, and effectiveness make it a benchmark in computer vision research. It achieved a top-1 accuracy of 71% on the ImageNet dataset [229], which contains images of dimensions 224x224x3.

Xception [195] was released in 2017, it is a convolutional neural network with depthwise separable convolutions. The motivation behind Xception is to enhance computational efficiency while maintaining or improving performance in image classification tasks. Its key contribution lies in replacing the standard convolutional layers with depthwise separable convolutions, which separate spatial and channel-wise convo-

lutions. This architectural change significantly reduces the number of parameters and computational costs compared to traditional convolutional layers. Structurally, Xception is organized into modules consisting of depthwise separable convolutions, followed by skip connections and nonlinear activations. This design allows Xception to capture complex features efficiently while reducing computational complexity. The advantages of Xception include its ability to achieve competitive performance on image classification tasks with reduced computational resources and its versatility for deployment on resource-constrained devices. Xception outperformed other transfer learning networks like ResNet and MesoNet in previous studies [42, 47]. Xception uses 22.8 million parameters.

MesoNet was released in 2018. It has been used in several studies to examine spatial features [42, 39]. *MesoNet* [42] is a CNN capable of analyzing the mesoscopic level features of images. The level of different types of features has been explored in our experiment through visualization by comparing MesoNet with Xception. It was released in 2018. It is the only network that has been tested with I-Frames. It is a network specifically introduced to detect DeepFakes. It begins with a sequence of four layers of successive convolutions and pooling and is followed by a dense network with one hidden layer. To improve generalization, the convolutional layers use ReLU activation functions that introduce non-linearity. Batch Normalization is used to regularize the output and to prevent the vanishing gradient effect. Dropout is utilized to regularize and improve the robustness of the network. It is a shallow and lightweight network with 27,977 trainable parameters in total.

EfficientNet[225] was released in 2019. Eight models under the name of EfficientNet were proposed. These models are named EfficientNetB0 to EfficientNetB7. EfficientNetB7 achieved the best top-1 accuracy in all EfficientNet models. EfficientNet-B7 also outperformed ResNet-152, Xception, and VGG on the ImageNet dataset [230]. EfficientNet was proposed to scale the CNN by considering and applying the scaling

ratio of network depth, height, and resolution. EfficientNet-B4 top-1 accuracy was 82.9% with 19 million parameters, while EfficientNet-B7 top-1 accuracy was 84.3% with 66 million parameters.

3.4 SRM Noise Stream

Semantic RGB features are not always clearly visible in forged images to differentiate them from real images, as shown in Fig. 3.3. Image noise can be an important feature that can be used in combination with RGB features to classify real and fake images. Image noise features are micro-level features [42] that depend on the local relationship between image pixels and usually can not be easily seen with a blank eye. Noise features extracted through Spatial Rich Models (SRM) [194] proved to be effective in increasing accuracy when combined with RGB features to classify real and forged images.

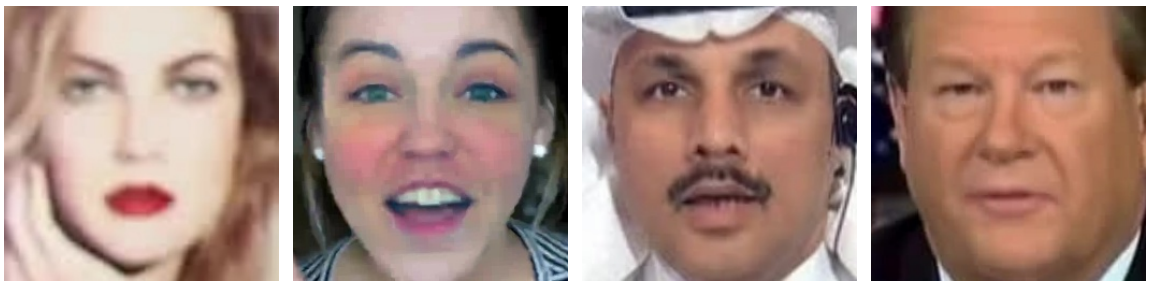


Figure 3.3: Absence of Semantic Features in Forged Images

SRM was originally created for steganalysis. Steganalysis is performed on images to detect whether an image is modified or not to hide a secret message inside of it. Originally, in the SRM approach, first, the residual or difference between neighbouring pixels is calculated. In the second step, statistical properties are calculated from the co-occurrence matrix to construct a feature vector for each image. In the last step, machine learning models are trained on calculated feature vectors to classify

images. Different filters are used to calculate the residual, as shown in Fig. 3.4. The residual or difference in image pixels is calculated using the following equations.

$$R_{ij} = \hat{X}_{ij}(N_{ij}) - cX_{ij} \tag{3.1}$$

where $c \in \mathbb{N}$ represents the residual order, N_{ij} denotes the local neighborhood of pixel X_{ij}. For example, in the first-order residual $R_{ij} = X_{i,j+1} - X_{ij}$, the central pixel X_{ij} is predicted as its immediate neighbour, $\hat{X}_{ij} = X_{i,j+1}$, while the predictor in the second-order residual $R_{ij} = X_{i,j-1} + X_{i,j+1} - 2X_{ij}$ assumes that the image is locally linear in the horizontal direction, $2\hat{X}_{ij} = (X_{i,j+1} + X_{i,j-1})$. Higher-order differences, as well as differences involving a larger neighborhood, correspond to more complicated patterns. After residual calculation, each residual is quantized and truncated.

$$R_{ij} \leftarrow \text{trunc}_T \left(\text{round} \left(\frac{R_{ij}}{q} \right) \right), \tag{3.2}$$

where q represents the quantization step. Quantization enhances the sensitivity of the residual to embedding changes occurring at spatial discontinuities in the image, such as edges and textures. Truncation, denoted by T, serves the construction of co-occurrence matrices in the next stages by limiting the dynamic range of the residual.

In the latest steganalysis [231] approaches, deep learning is used to learn features from residuals instead of using manual feature calculation from a co-occurrence matrix. In the SRM noise stream, each image is first passed through 27 high-pass SRM filters before passing through a deep learning model to learn noise features. It was found that initializing the convolution filters with filters used in the SRM approach and allowing these filters to be optimized according to data can improve accuracy on DeepFakes datasets [47] [232]. All the filters in the SRM stream are resized

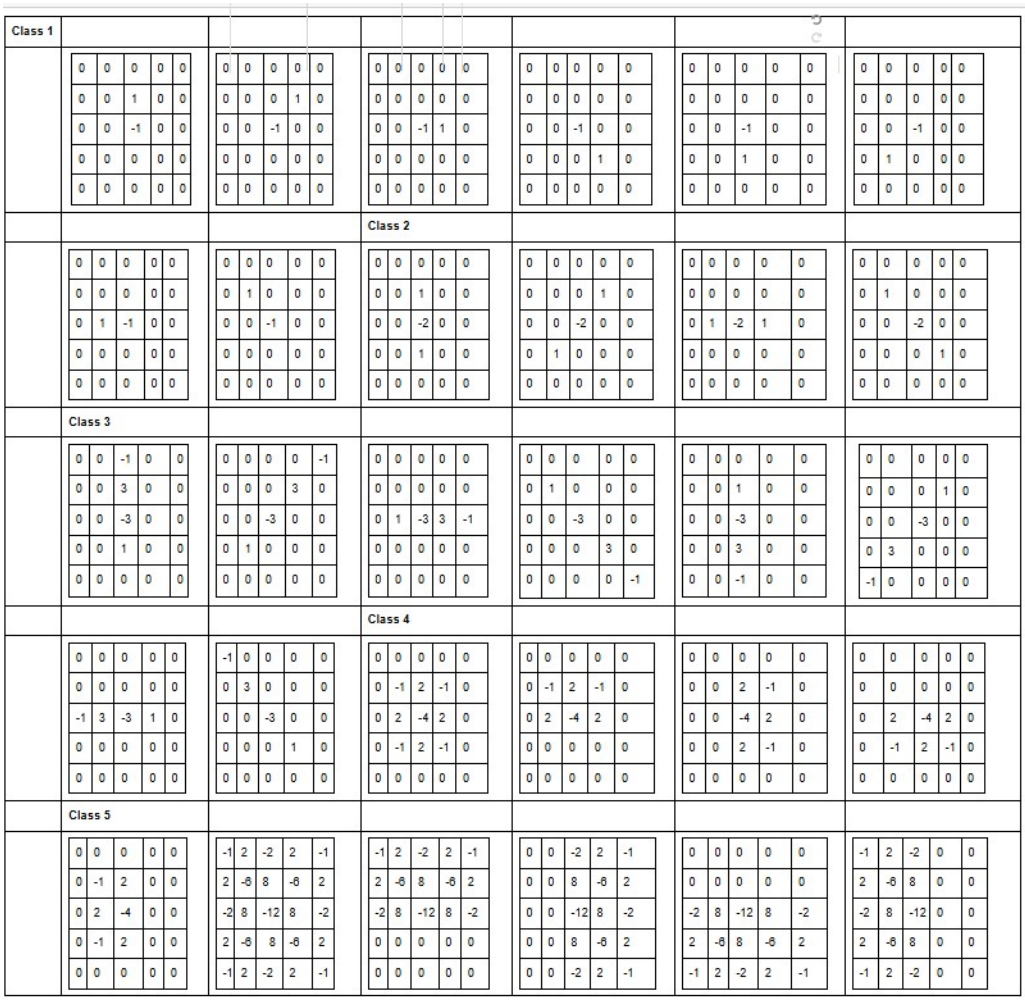


Figure 3.4: Collection of SRM Filters

to 5x5 to provide a standard dimension for the noise extraction process. Noise extracted through images will also pass through ResNet-152 in the SRM branch. Adam optimizer [223] with a 0.00001 learning rate is used in the SRM stream.

3.5 Fusion Box

The classification score of each video is acquired from both streams. For each stream, the last layer of network output scores for a video. The final score for a video is

calculated by combining the weighted score of both streams:

$$r = \text{Weight1} \times (r1) + \text{Weight2} \times (r2) \tag{3.3}$$

where r1 represents the output of the RGB stream, r2 refers to the result of the SRM stream, and r is the final score. Weight1 represents the weight of the RGB stream, and weight2 represents the weight of the SRM stream. The weight1 and weight2 are calculated using the weight selection experiment covered in the next chapter. Since the detection is a binary classification task, the model will output a value between 0 and 1, indicating the real frame probability. The frame is considered real if the final score is 0.5 or above the threshold; otherwise, it is labelled fake. A threshold of 0.5 has been used in several binary classification research studies [233, 234]. An average frame aggregation score is applied to assign a label to a video in testing.

3.6 Concluding Remarks:

In this chapter, we presented details of our DeepFake detection architecture. We presented that the first module of the model is responsible for preprocessing operations. In the second module, data augmentation is applied to increase the performance of our architecture. In the third and fourth modules, different types of features (RGB, Semantic) are extracted, and a score is assigned to each frame. In the last module, the fusion of both streams is performed. In the next chapter, we will discuss the experiments and results of the proposed framework.

Chapter 4

Experiments

This chapter presents details of the dataset, the implementation details of our proposed architecture, the evaluation matrix, and the results.

4.1 Datasets

In this work, we have utilized the three DeepFake datasets of [235] [236] of Celeb-DF-V1, CELEB-DF-V2 [237], and FaceForensics++ [238], as shown in Table 4.1. These datasets are diverse, large, and easy to access. These datasets do not contain audio.

The Celeb-DF dataset was chosen for its enhanced custom synthesis method, which guarantees comparatively high resolution and better color correlation, which makes it a challenging dataset. The Celeb-DF dataset has two versions: V1 and V2. In spite of the good quality of Celeb-DF, slight blurriness and colour mismatch can be found in fake videos. Celeb-DF only contains videos based on face-swapping. Fake videos in Celeb-DF are generated by using auto-encoders. This dataset does not contain any compression levels other than 0, as shown in Table 4.2.

Table 4.1: Summary of Datasets

DataSet	**Total Videos**	**Total Real Videos**	**Total Fake Videos**	**Manipulation Type**	**Audio**
Celeb-DF-V1	1,203	408	795	Facial Swapping	✗
Celeb-DF-V2	6,529	890	5,639	Facial Swapping	✗
FaceForensics++ (DeepFakes)	2,000	1,000	1,000	Facial Swapping	✗
FaceForensics++ (Face2Face)	2,000	1,000	1,000	Facial Reenact-ment	✗
FaceForensics++ (FaceSwap)	2,000	1,000	1,000	Facial Swapping	✗
FaceForensics++ (NeuralTextures)	2,000	1,000	1,000	Facial Reenact-ment	✗

Table 4.2: Summary of Compression and Storage of Datasets

DataSet	**Storage Size**	**Compression Levels Available Online**
Celeb-DF-V1	2.0 GB	0
Celeb-DF-V2	9.3 GB	0
FaceForensics++	542 GB	0, 23, 40

The FaceForensics++ dataset was selected for its diversity in types of forgery, which encompasses both facial swapping and facial reenactment, different compression levels, and the diverse tools used to produce fake videos. FaceForensics++ consists of four subsets of forged videos: DeepFakes (DF), Face2Face (F2F), FaceSwap (FS), and NeuralTextures (NT). DeepFake and FaceSwap are based on face-swapping techniques, while Face2Face and NeuralTextures consist of videos created using facial reenactment. The major drawback of the FaceForensics++ dataset is that it has clear visual artefacts, like distorted faces and blurriness in some of the fake videos.

We have utilized the H.264 codec [180] to compress videos of Celeb-DF-V1 and Celeb-DF-V2 datasets, as it is the default codec for the FaceForensics++ dataset

and is widely used on social media networks like YouTube, Snapchat, Facebook, and Telegram for video compression [239, 240, 241, 242]. H.264 is a video codec, an algorithm specially designed to compress video files to enhance transmission and storage efficiency. Its popularity stems from its fast coding speed, cost-effectiveness, and extensive hardware support [243].

We have used two compression levels in our study for both datasets by employing a compression factor. The compression factor parameter can range between values 0 and 51, where 0 is lossless, and 51 is the lossiest. This selection of different compression was aimed to introduce diversity and expedite the process, leveraging the pre-existing compression levels available in the FaceForensics++ dataset. Compression factor parameter values between 18 and 28 are most commonly used to reduce bit rates when visual quality is important [244]. A compression level of 0 was not used as it requires a storage size of more than 500 GB.

To ensure a balanced dataset, an equal number of positive and negative samples were selected from each dataset, as indicated in Table 4.3. In the Celeb-DF-V1 dataset, 408 real and 408 fake videos were randomly selected to achieve balance, as originally, this dataset consisted of a total of 408 real and 795 fake videos. Similarly, from the Celeb-DF-V2 dataset, 890 real and 890 fake videos were sampled randomly for experimentation as this dataset, in total, contains 890 real videos and over 5000 fake videos. In FaceForensics++ datasets, four sets were prepared, and each set contained 1000 real and 1000 fake videos from each forgery type. Some videos were discarded as dlib library was not able to detect faces in its frames. Table 4.3 also shows the scalability of our solution, as the average I-frames in each dataset were less than 3. To split the dataset for training and testing, almost 80% of the videos were allocated for training and validation, while the remaining 20% were reserved for testing purposes. 80-20 distribution has been used in many previous studies of DeepFake detection [39, 245].

Table 4.3: Distribution of Videos for Training and Testing Phase with 80% and 20% Distribution with Compression level 23 and 40

Dataset	**Total Videos**	**Total I-Frames**	**Average I-Frames**	**Real Videos (Train)**	**Fake Videos (Train)**	**Real Videos (Test)**	**Fake Videos (Test)**
Celeb-DF-V1 (C23)	816	1,746	2.1	327	327	81	81
Celeb-DF-V1 (C40)	816	1,724	2.1	327	327	81	81
Celeb-DF-V2 (C23)	1,780	3,725	2.09	712	712	177	177
Celeb-DF-V2 (C40)	1,780	3,674	2.06	712	712	177	177
DeepFakes (C23)	1,998	5,148	2.5	800	800	199	199
DeepFakes (C40)	1,998	5,122	2.5	800	800	199	199
Face2Face (C23)	1,998	5,155	2.5	800	800	199	199
Face2Face (C40)	1,998	5,147	2.5	800	800	199	199
FaceSwap (C23)	1,998	4,769	2.3	800	800	199	199
FaceSwap (C40)	1,998	4,754	2.3	800	800	199	199
NeuralTextures (C23)	1,998	4,771	2.3	800	800	199	199
NeuralTextures (C40)	1,998	4,758	2.3	800	800	199	199

4.2 Experimental Setup

The proposed architecture was implemented using the TensorFlow framework [246] with version 2.17.1 (See Appendix A). Experiments were conducted using L4 GPU

from Google Colab [247] with 22.5 GB of GPU RAM and 53 GB of system RAM.

4.3 Evaluation Matrix

The confusion matrix is commonly used to evaluate the performance of machine learning models trained to perform binary classification [248]. The confusion matrix classifies predictions into four categories: true positive (TP), true negative (TN), false positive (FP), and false negative (FN). The confusion matrix is used to calculate the multiple evaluation matrix of precision, recall, F1 score, and accuracy. DeepFake videos are considered positive, and real videos are the negative class.

- **Accuracy:** Accuracy represents the ratio of correct predictions to total predictions for a machine learning model.

$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{TN} + \text{FP} + \text{FN}} \tag{4.1}$$

- **Precision:** It is an important evaluation matrix that is very important for spam detection classifiers. It shows how many instances classified as positive are actually positive.

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}} \tag{4.2}$$

- **Recall:** It shows the ratio of true positive instances detected out of all the actual positive instances.

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}} \tag{4.3}$$

- **F1-Score:** The F1 score is a metric used to evaluate the performance of a classification model by combining both precision and recall into a single value.

It is the harmonic mean of precision and recall.

$$\text{F1-Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \tag{4.4}$$

- **Frames:** The number of per-video frames utilized as input samples for training the deepfake detection model.

4.4 Network Selection for RGB Stream

To select the backbone network for the RGB stream, we performed a comparative experiment with MesoNet [42], Xception [195], VGG-19 [224], EfficientNet-B4 [225], EfficientNet-B7 [225] and ResNet-152 [183]. MesoNet was selected as a shallow network and was introduced for the first time to detect DeepFake videos. Other networks were selected because of their comparative performance on the ImageNet dataset [229]. Other than MesoNet, all other deep learning networks are pre-trained. Pre-trained models were fine-tuned on our datasets. All models were trained using k-fold cross-validation [249] with k=5 to avoid overfitting . k=5 is a common value on k-fold cross-validation [250].

4.4.1 RGB Stream Without Augmentation

All deep networks outperformed shallow MesoNet on both high-quality and low-quality videos, as shown in Table 4.4 and Fig. 4.1. The performance gap between shallow and deep networks was between 2% and 11% across different datasets.

The learning rate, optimizer, and batch size for different networks were selected after hyperparameter tuning, as shown in Table 4.5.

Table 4.4: Performance Comparison of Deep Learning Models without Augmentation on RGB Stream

Dataset	**Meso Net**	**Xception**	**Efficient Net (B4)**	**Efficient Net (B7)**	**VGG-19**	**ResNet-152**
Celeb-DF-V1 (C23)	81.48	87.65	89.21	**92.59**	88.27	86.42
Celeb-DF-V1 (C40)	77.16	80.25	79.01	**82.72**	79.63	79.01
Celeb-DF-V2 (C23)	89.27	85.59	90.4	**91.53**	90.96	90.68
Celeb-DF-V2 (C40)	**79.38**	74.86	73.73	74.29	71.71	71.19
DeepFakes (C23)	94.97	94.47	**97.99**	**97.99**	96.23	**97.99**
DeepFakes (C40)	90.95	86.93	92.46	92.71	87.69	**92.96**
Face2Face (C23)	92.46	94.97	**97.49**	96.98	96.73	96.48
Face2Face (C40)	78.14	74.87	76.88	77.14	**80.4**	78.39
FaceSwap (C23)	93.97	95.98	97.99	**98.74**	96.98	97.24
FaceSwap (C40)	77.89	76.63	82.16	**86.43**	75.13	79.65
NeuralTextures (C23)	69.01	75.63	**78.89**	77.64	77.14	78.14
NeuralTextures (C40)	56.03	62.31	67.09	67.09	**69.6**	65.33

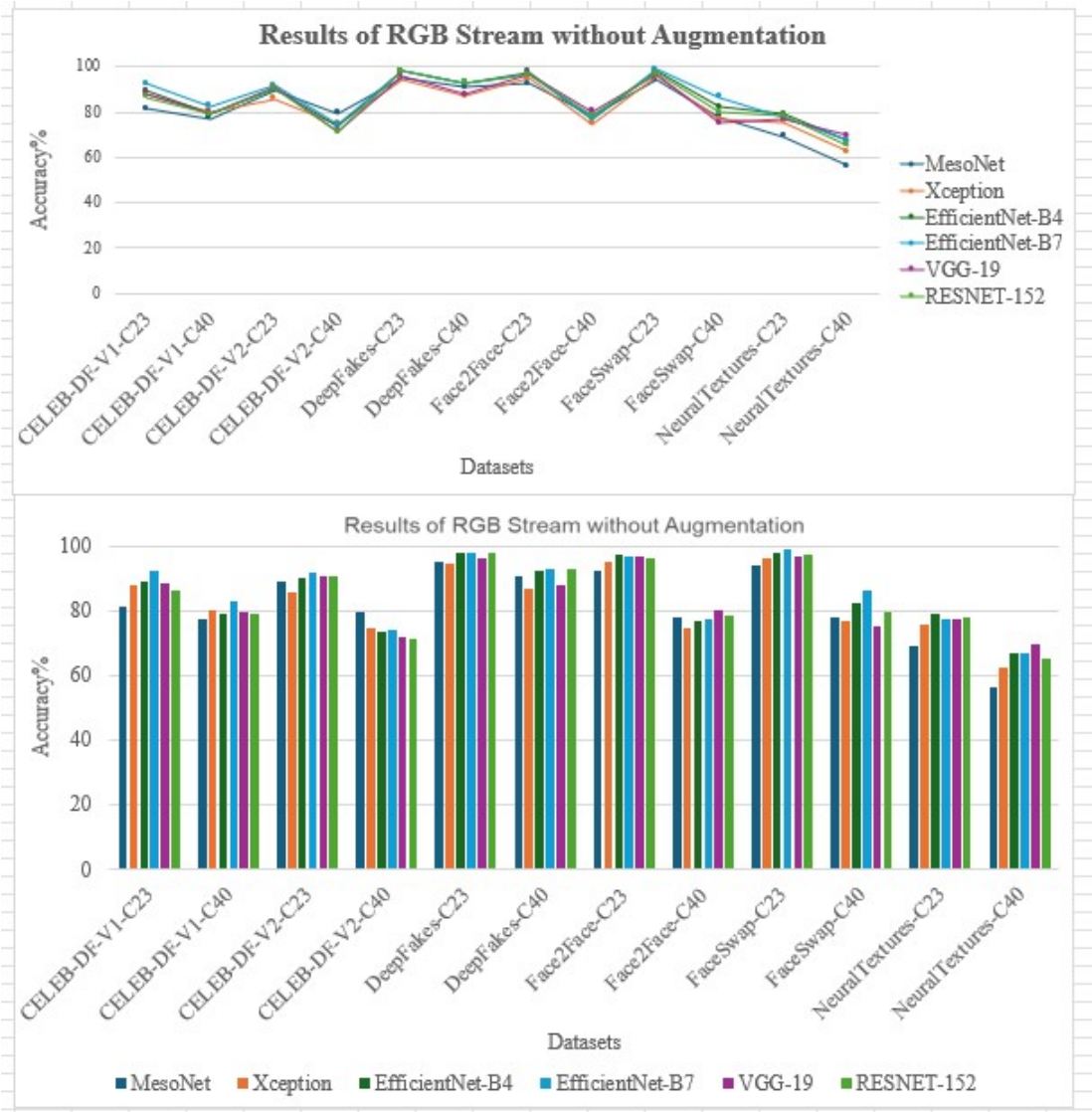


Figure 4.1: Results on RGB Stream without Augmentation on with Different Backbone Models

Table 4.5: RGB & Residual Streams Selected Hyperparameters

Model	**Optimizer**	**Initial Learning Rate**	**Batch Size**
MesoNet	Adam	0.001	8
Xception	Adam	0.00001	32
EfficientNet (B4)	Adam	0.00001	8
EfficientNet (B7)	Adam	0.00001	8
VGG-19	Adam	0.00001	32
ResNet-152	Adam	0.00001	8

4.4.2 RGB Stream with Augmentation

Data augmentation was applied during the training process to increase the performance of models and to model overfitting [251]. Data Augmentation increased the performance of deep learning models in most datasets, as shown in Table 4.6 and Fig. 4.2. EfficientNet-B4, EfficientNet-B7, and ResNet-152 outperformed other deep learning models, as shown in Table 4.6.



Figure 4.2: Results on RGB Stream with Augmentation on with Different Backbone Models

Table 4.6: Performance Comparison of Deep Learning Models with Augmentation on RGB Stream

Dataset	**Meso Net**	**Xception**	**Efficient Net (B4)**	**Efficient Net (B7)**	**VGG-19**	**ResNet-152**
CELEB-DF-V1 (C23)	73.46	89.51	91.36	92.59	88.27	**93.21**
CELEB-DF-V1 (C40)	72.84	77.16	79.01	85.19	74.69	**88.89**
CELEB-DF-V2 (C23)	83.62	92.09	92.09	**94.07**	92.94	93.79
CELEB-DF-V2 (C40)	75.42	79.38	76.55	78.25	**81.07**	77.97
DeepFakes (C23)	94.97	97.24	**98.99**	98.49	98.24	98.24
DeepFakes (C40)	87.94	91.96	93.22	**93.97**	90.45	93.47
Face2Face (C23)	79.9	96.73	**98.49**	98.24	97.99	97.99
Face2Face (C40)	73.87	82.41	82.41	78.39	78.64	**82.61**
FaceSwap (C23)	75.88	97.24	98.49	**98.74**	96.98	98.24
FaceSwap (C40)	69.6	85.68	**89.45**	88.94	77.14	89.2
NeuralTextures (C23)	72.61	81.66	81.66	**82.41**	77.89	**82.41**
NeuralTextures (C40)	63.57	**68.09**	67.84	67.84	66.08	67.09

4.4.3 RGB Stream Visualization

Gradient-weighted Class Activation Mapping [252] was used to understand the working of MesoNet, Xception, and ResNet-152. Gradient-weighted Class Activation Mapping (Grad-CAM) is a technique for visual explanations for the decisions of CNN-based models. It helps us to understand which areas of the input image are important to predict a particular class by providing a heat map as output. Grad-CAM is generated by computing the gradient of the model's output with respect to the activation map or feature map in the last convolution layer. Fig. 4.3, Fig. 4.4, and Fig. 4.5 demonstrate the difference between the grad cam visualization of MesoNet, Xception, and ResNet-152. Fig. 4.3, Fig. 4.4, and Fig. 4.5 clearly demonstrate the features all models are focusing on. The scale of the region of MesoNet is comparatively narrow as compared to Xception and ResNet-152.



Figure 4.3: GRAD CAM Visualization on MesoNet



Figure 4.4: GRAD CAM Visualization on Xception



Figure 4.5: GRAD CAM Visualization on ResNet-152

4.5 Filters Selection for SRM Stream

We performed experiments to assess the impact of the number of filters in the SRM stream. Table 4.7, 4.8, 4.9 and Fig. 4.6, Fig. 4.7, Fig. 4.8 present the comparison of performance of different models with the 3, 9, and 27 filters. We used these divisions of filters to find the effect of the number of filters on the SRM stream. Three filters are used in studies that have previously been using SRM filters for DeepFake detection [47]. We have divided the total SRM filters into low, medium, and high numbers of filters, like 3, 9, and 27, to evaluate the effect of increasing the number of filters on the SRM stream.

The results demonstrate a considerable increase in performance with a higher number of SRM filters on face-swapping datasets generated using auto-encoders like the Celeb-DF dataset and FaceSwap, compared to the graphics-based DeepFakes facial swapping subset and the Face2Face facial reenactment subset of FaceForensics++. In these experiments, EfficientNet-B7 and ResNet-152 performed better than EfficientNet-B4. Fig. 4.12 also visually shows that the difference in SRM noise feature between real and fake is more prominent in the facial swapping dataset as compared to the facial reenactment dataset. Fig. 4.9, 4.10, and 4.11 demonstrate that increasing the number of filters can increase the performance of the SRM branch.

Table 4.7: Performance Comparison of Deep Learning Models with Augmentation on 3 SRM Filters

Dataset	**EfficientNet (B4)**	**EfficientNet (B7)**	**ResNet-152**
CELEB-DF-V1-C23	85.19	88.89	**90.74**
CELEB-DF-V1-C40	74.07	**80.86**	80.25
CELEB-DF-V2-C23	90.40	92.37	**95.20**
CELEB-DF-V2-C40	75.42	75.71	**81.92**
DeepFakes-C23	**98.49**	97.99	96.73
DeepFakes-C40	88.19	90.95	**91.21**
Face2Face-C23	**97.49**	96.98	97.24
Face2Face-C40	72.61	75.88	**79.15**
FaceSwap-C23	97.24	**97.74**	96.73
FaceSwap-C40	79.90	82.91	**86.43**
NeuralTextures-C23	76.38	**77.14**	75.63
NeuralTextures-C40	65.58	65.83	**66.33**

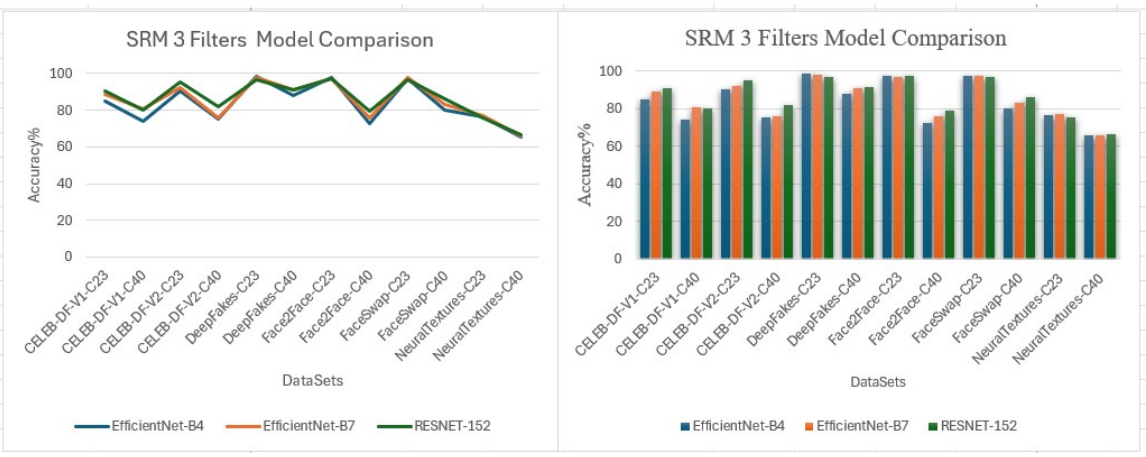


Figure 4.6: Results on 3 Filters on SRM Streams

Table 4.8: Performance Comparison of Deep Learning Models with Augmentation on 9 SRM Filters

Dataset	**EfficientNet (B4)**	**EfficientNet (B7)**	**ResNet-152**
CELEB-DF-V1-C23	86.42	**93.21**	92.59
CELEB-DF-V1-C40	74.69	80.25	**83.95**
CELEB-DF-V2-C23	88.70	**92.94**	**92.94**
CELEB-DF-V2-C40	76.55	**79.10**	78.81
DeepFakes-C23	97.24	**97.99**	97.74
DeepFakes-C40	91.96	**94.47**	91.46
Face2Face-C23	**97.99**	97.49	97.74
Face2Face-C40	77.89	**82.41**	80.65
FaceSwap-C23	97.24	**97.74**	97.24
FaceSwap-C40	86.68	88.94	**90.20**
NeuralTextures-C23	80.15	**81.16**	77.39
NeuralTextures-C40	66.83	**68.84**	67.09

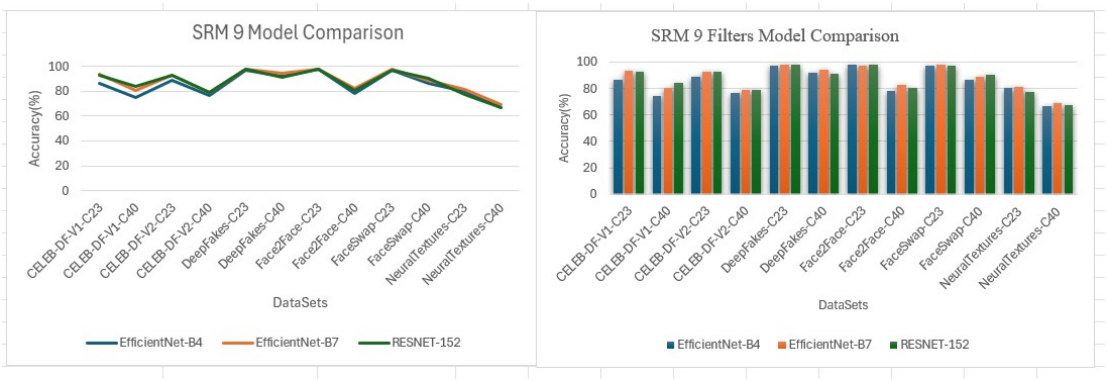


Figure 4.7: Results on 9 Filters on SRM Streams

Table 4.9: Performance Comparison of Deep Learning Models with Augmentation on 27 SRM Filters

Dataset	**EfficientNet (B4)**	**EfficientNet (B7)**	**ResNet-152**
CELEB-DF-V1-C23	90.74	**93.21**	**93.21**
CELEB-DF-V1-C40	76.54	**80.25**	**80.25**
CELEB-DF-V2-C23	91.24	94.35	**94.63**
CELEB-DF-V2-C40	78.25	**80.23**	78.81
DeepFakes-C23	98.24	**98.49**	97.99
DeepFakes-C40	90.7	**94.47**	93.22
Face2Face-C23	98.24	97.49	**98.49**
Face2Face-C40	79.9	80.9	**83.17**
FaceSwap-C23	97.74	**98.74**	96.98
FaceSwap-C40	86.18	**89.7**	89.7
NeuralTextures-C23	78.64	**81.41**	77.39
NeuralTextures-C40	66.58	63.82	**67.09**



Figure 4.8: Results on 27 Filters on SRM Streams

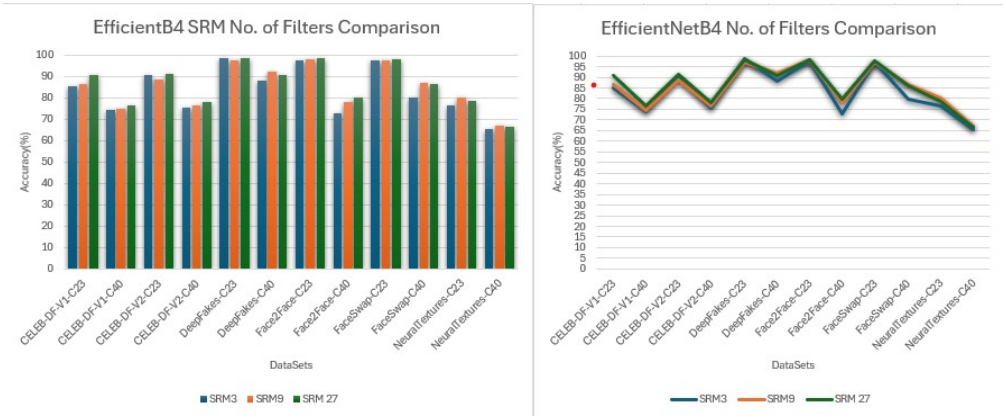


Figure 4.9: Results on EfficientNet-B4 Different Number of Filters

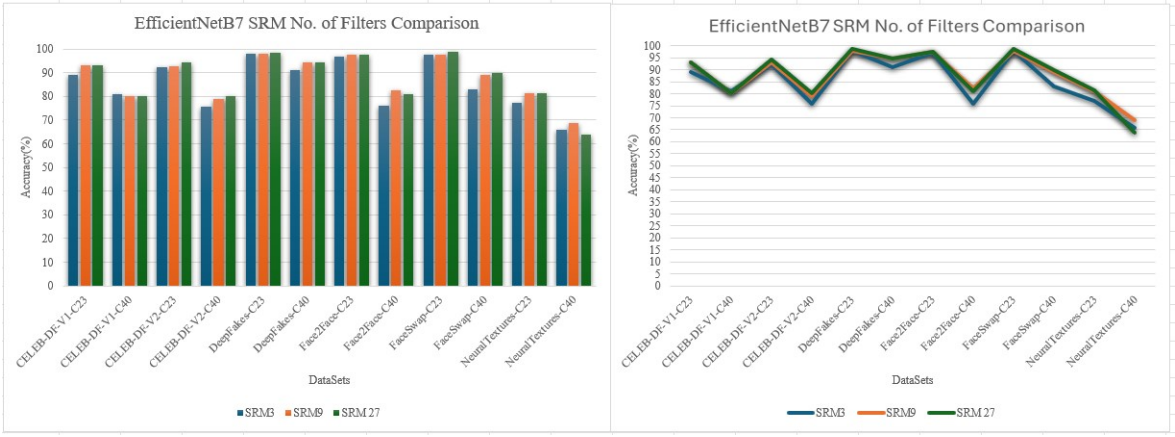


Figure 4.10: Results on EfficientNet-B7 Different Number of Filters

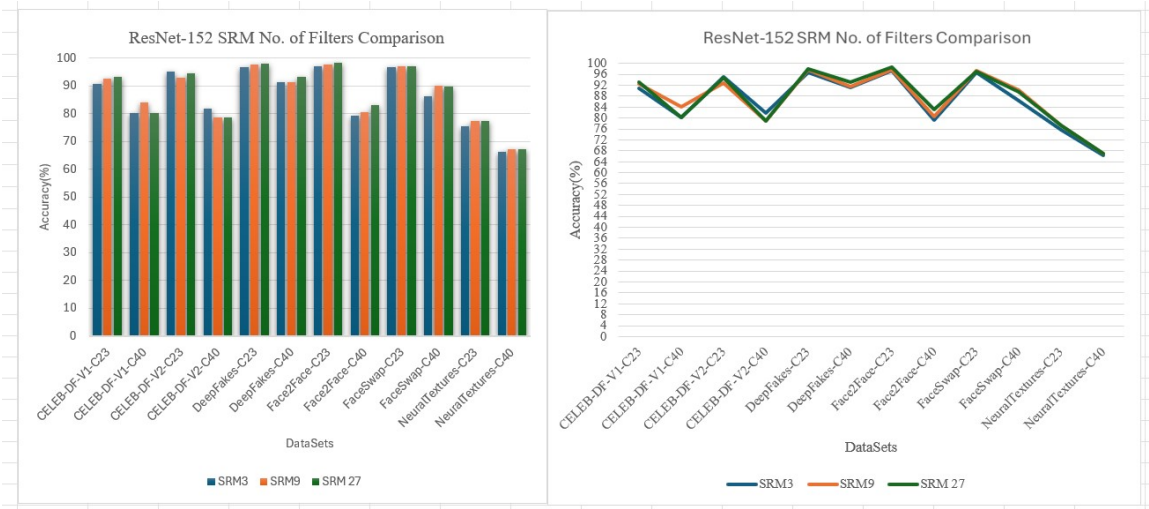


Figure 4.11: Results on ResNet-152 Different Number of Filters

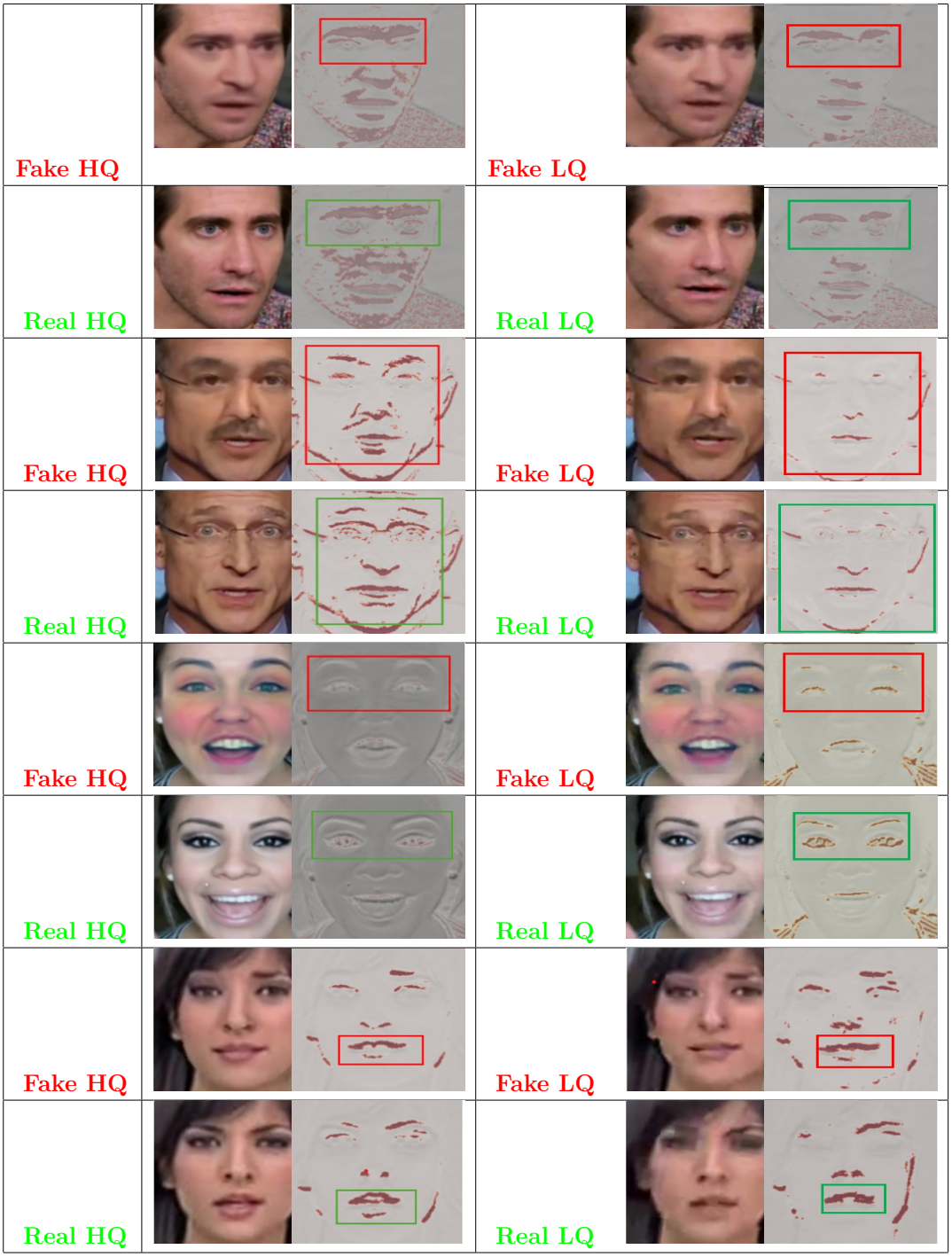


Figure 4.12: Comparison of SRM Noise in High Quality (HQ) and Low Quality (LQ) Videos. Each two consecutive rows indicates a noise feature from the same dataset. The order of the datasets is Celeb-DF-V2, FaceSwap, DeepFakes, and Face2Face. Green represents real, and red represents fake images.

4.6 Ensemble of Both Branches

The output of the RGB stream and the SRM stream with 27 filters will exhibit different properties, as the former focuses on semantic RGB information while the latter targets noise features. On the few datasets, the RGB stream performs better, whereas on others, the SRM stream outperforms, as shown in Figure 4.13. To take advantage of both types of features, we employed a weighted fusion approach, which means assigning different weights to each branch as demonstrated in Table 4.10, Table 4.11, and Table 4.12. We discovered that assigning a 0.6 weight to the RGB branch and 0.4 to the SRM branch yields the best average results across various datasets. Additionally, we found that the performance gap between the two branches is smaller in high-quality videos compared to low-quality videos, and the ensemble proved to be more effective on low-quality videos (C40) compared to high-quality videos (C23).



Figure 4.13: Results comparison of RGB Stream, SRM stream, and Ensemble Results for ResNet-152

Table 4.10: Ensemble of RGB EfficientNet-B4 with SRM 3, SRM 9, and SRM 27. CLB represents CELEB-DF, DF represents DeepFakes, F2F represents Face2Face, FS represents FaceSwap, and NT represents NeuralTextures subsets of FaceForensics++ datasets.

Model	W1	W2	CLB (C23)	CLB (C40)	CLB V2 (C23)	CLB V2 (40)	DF (C23)	DF (C40)	F2F (C23)	F2F (C40)	FS (C23)	FS (C40)	NT (C23)	NT (C40)
SRM3	0.1	0.9	84.57	72.84	90.68	75.71	98.49	89.7	97.49	73.87	97.24	80.4	77.89	66.83
	0.2	0.8	85.8	73.46	90.96	75.99	98.74	92.21	97.49	74.62	97.74	81.41	78.64	67.34
	0.3	0.7	87.04	74.07	92.94	75.99	98.99	92.46	97.74	75.63	97.99	83.42	78.14	67.09
	0.4	0.6	87.65	73.46	93.5	76.27	98.99	92.71	97.74	77.64	98.24	86.68	79.15	66.33
	0.5	0.5	87.65	74.07	**94.35**	**76.84**	98.99	93.47	98.24	81.16	98.49	87.44	79.15	67.09
	0.6	0.4	88.89	75.31	93.79	76.27	**99.25**	93.47	**98.74**	81.16	98.49	87.44	80.15	67.09
	0.7	0.3	**90.74**	75.31	93.79	75.42	98.99	93.97	98.49	80.9	**98.74**	87.69	79.65	67.59
	0.8	0.2	**90.74**	76.54	93.5	75.99	98.99	**94.22**	98.49	80.4	**98.74**	88.44	80.9	67.34
	0.9	0.1	91.36	**78.4**	92.66	76.27	98.99	**94.22**	98.24	**81.41**	**98.74**	**89.45**	**81.16**	**68.09**
SRM9	0.1	0.9	87.04	74.07	89.55	77.68	97.74	92.21	97.99	79.9	97.49	88.19	80.15	66.33
	0.2	0.8	87.04	75.31	90.11	**78.53**	97.99	91.96	97.99	80.15	98.24	88.69	80.9	67.34
	0.3	0.7	89.51	76.54	92.09	78.25	97.99	93.22	97.99	79.65	98.49	88.94	81.91	68.09
	0.4	0.6	90.12	76.54	92.94	78.25	98.24	92.96	97.99	79.9	98.49	89.7	81.66	68.09
	0.5	0.5	**90.74**	77.78	93.5	77.68	98.49	93.47	**98.24**	80.9	98.49	89.45	81.66	**68.59**
	0.6	0.4	**90.74**	76.54	93.22	77.12	98.49	93.47	**98.24**	81.16	**98.74**	**90.2**	82.41	67.34
	0.7	0.3	**90.74**	77.78	93.5	77.12	98.49	**94.22**	**98.24**	**82.91**	**98.74**	89.95	**82.91**	67.09
	0.8	0.2	**90.74**	**78.4**	**94.07**	77.12	**98.99**	93.97	**98.24**	82.41	**98.74**	89.95	**82.91**	67.84
	0.9	0.1	**90.74**	**78.4**	92.94	77.12	**98.99**	93.97	**98.24**	81.91	**98.74**	**90.2**	82.16	68.09
SMR27	0.1	0.9	90.74	76.54	91.24	79.1	98.74	91.71	98.24	80.4	97.74	86.93	78.89	67.34
	0.2	0.8	89.51	77.16	92.09	**79.38**	98.74	92.21	98.24	81.41	97.74	87.69	79.9	68.84
	0.3	0.7	90.12	76.54	92.66	78.81	98.99	93.47	98.49	82.41	97.99	88.69	80.4	68.34
	0.4	0.6	**91.98**	75.93	92.66	78.81	98.99	92.71	98.49	83.17	98.49	89.2	81.16	**69.1**
	0.5	0.5	91.36	74.69	92.66	78.81	**99.25**	92.71	98.49	**83.67**	98.49	**90.2**	81.66	67.59
	0.6	0.4	**91.98**	74.69	92.66	78.53	**99.25**	93.72	98.49	**83.67**	98.49	89.95	81.16	68.09
	0.7	0.3	**91.98**	75.31	92.66	77.97	**99.25**	93.72	98.24	83.92	98.49	**90.2**	81.41	67.84
	0.8	0.2	**91.98**	76.54	**93.22**	77.4	**99.25**	**93.97**	**98.74**	82.41	98.49	89.95	81.91	68.09
	0.9	0.1	91.36	**77.78**	92.37	77.12	**99.25**	**93.97**	98.49	82.91	**98.74**	89.7	**82.41**	68.34

Table 4.11: Ensemble of EfficientNet-B7 with SRM 3, SRM 9, and SRM 27. CLB represents CELEB-DF, DF represents DeepFakes, F2F represents Face2Face, FS represents FaceSwap, and NT represents NeuralTextures subsets of FaceForensics++ datasets.

Model	W1	W2	CLB (C23) V1	CLB (C40) V1	CLB (C23) V2	CLB (C40) V2	DF (C23)	DF (C40)	F2F (C23)	F2F (C40)	FS (C23)	FS (C40)	NT (C23)	NT (C40)
SRM3	0.1	0.9	90.12	82.1	93.22	76.84	98.49	91.96	97.49	75.63	98.49	83.92	78.14	67.09
	0.2	0.8	90.74	83.33	93.22	76.27	98.49	93.22	97.99	77.64	98.49	84.67	79.15	66.83
	0.3	0.7	91.36	83.33	93.79	76.84	98.74	92.96	**98.24**	77.14	**98.74**	86.68	80.9	66.83
	0.4	0.6	90.74	82.1	94.63	77.4	98.74	93.47	**98.24**	78.64	**98.74**	87.19	82.66	67.09
	0.5	0.5	90.74	82.72	**95.2**	**79.1**	**99.25**	93.72	**98.24**	78.89	**98.74**	88.44	83.42	68.34
	0.6	0.4	91.98	80.86	94.92	79.1	98.99	93.72	**98.24**	79.9	**98.74**	89.2	**83.67**	68.84
	0.7	0.3	92.59	82.72	**95.2**	78.53	**99.25**	93.72	**98.24**	**80.15**	**98.74**	**89.7**	**83.67**	**69.6**
	0.8	0.2	**93.21**	83.95	**95.2**	78.53	98.99	93.97	**98.24**	**80.15**	**98.74**	**89.7**	82.66	69.35
	0.9	0.1	92.59	**85.19**	94.63	78.53	98.74	**94.22**	**98.24**	79.15	**98.74**	89.2	82.41	67.59
SRM9	0.1	0.9	93.21	80.25	93.5	79.66	97.74	94.22	97.99	81.91	97.99	88.44	82.41	68.09
	0.2	0.8	**93.83**	81.48	92.94	79.38	97.49	94.72	97.99	**82.16**	97.99	88.19	81.91	67.84
	0.3	0.7	**93.83**	80.86	92.94	80.51	97.74	**95.23**	97.99	81.41	98.24	88.19	81.16	67.09
	0.4	0.6	93.21	81.48	93.5	81.64	98.49	94.97	97.99	81.16	98.49	88.19	81.66	68.59
	0.5	0.5	92.59	82.1	94.07	**81.92**	98.49	94.72	97.99	80.9	**98.99**	**89.2**	81.16	69.1
	0.6	0.4	93.21	84.57	94.07	80.79	**98.74**	94.47	97.99	80.4	98.74	**89.2**	82.16	69.6
	0.7	0.3	92.59	83.95	94.35	80.51	**98.74**	94.22	97.99	79.15	98.74	**89.2**	82.41	**70.85**
	0.8	0.2	93.21	85.8	**94.92**	79.38	98.49	94.22	**98.24**	78.39	98.74	**89.2**	**82.66**	69.1
	0.9	0.1	92.59	**85.19**	94.35	77.68	98.49	93.97	97.99	78.64	98.74	**89.2**	82.41	68.59
SMR27	0.1	0.9	92.59	80.86	94.63	**81.64**	98.49	94.47	97.74	**81.16**	98.74	89.95	81.66	64.32
	0.2	0.8	92.59	81.48	94.92	81.07	98.49	94.72	97.99	80.4	**98.99**	90.2	81.41	63.82
	0.3	0.7	92.59	82.72	**95.48**	80.23	98.49	94.97	97.99	79.9	**98.99**	**90.45**	81.66	64.32
	0.4	0.6	**93.21**	84.57	94.92	80.79	98.49	94.97	**98.24**	79.9	**98.99**	89.95	82.66	64.82
	0.5	0.5	92.59	83.95	94.92	79.94	**98.74**	94.97	**98.24**	79.9	98.74	88.94	**82.91**	66.83
	0.6	0.4	92.59	83.95	94.92	80.23	**98.74**	**95.23**	**98.24**	79.65	98.74	88.69	82.16	**68.09**
	0.7	0.3	92.59	85.8	94.92	79.66	**98.74**	94.97	**98.24**	79.9	98.74	88.69	82.66	67.84
	0.8	0.2	92.59	**86.42**	94.63	79.1	98.49	94.97	97.99	79.15	98.74	89.2	82.66	67.59
	0.9	0.1	91.98	85.19	94.63	78.53	**98.74**	94.47	**98.24**	79.15	98.74	89.7	82.16	67.84

Table 4.12: Ensemble of RGB ResNet-152 with SRM 3, SRM 9, and SRM 27. CLB represents CELEB-DF, DF represents DeepFakes, F2F represents Face2Face, FS represents FaceSwap, and NT represents NeuralTextures subsets of FaceForensics++ datasets.

Model	W1	W2	CLB (C23)	CLB (C40)	CLB V1	CLB V2	CLB (40)	DF (C23)	DF (C40)	F2F (C23)	F2F (C40)	FS (C23)	FS (C40)	NT (C23)	NT (C40)
SRM3	0.1	0.9	91.36	80.25	96.05	81.92		97.49	91.96	97.49	79.9	97.49	88.44	76.38	67.84
	0.2	0.8	91.98	81.48	96.33	82.49		97.99	92.46	97.49	81.91	98.24	88.19	76.88	69.1
	0.3	0.7	92.59	83.33	96.61	**82.49**		98.49	92.71	97.74	82.41	98.49	88.44	77.89	68.34
	0.4	0.6	92.59	83.95	**96.89**	**82.49**		98.74	92.96	**98.24**	82.66	98.49	88.94	80.15	68.84
	0.5	0.5	92.59	87.65	96.61	81.92		**99.25**	93.47	**98.24**	**82.91**	98.24	89.45	81.66	69.35
	0.6	0.4	92.59	88.27	96.05	81.07		**99.25**	93.47	**98.24**	81.91	98.24	**90.7**	82.16	**69.85**
	0.7	0.3	**94.44**	88.27	95.48	81.36		98.99	93.22	**98.24**	82.41	**98.74**	**90.7**	81.91	67.84
	0.8	0.2	93.83	**88.89**	95.2	79.38		98.99	93.22	97.99	81.66	**98.74**	90.2	**82.41**	68.34
	0.9	0.1	93.21	**88.89**	94.63	78.81		98.74	**93.72**	**98.24**	82.16	**98.74**	89.45	**82.41**	67.09
SRM9	0.1	0.9	93.83	84.57	94.07	79.66		97.99	92.46	97.74	81.66	97.49	91.21	78.39	66.83
	0.2	0.8	**94.44**	85.8	94.92	79.66		97.99	92.21	97.74	82.41	97.74	91.71	80.4	65.33
	0.3	0.7	**94.44**	86.42	94.92	80.79		98.49	92.21	**98.24**	82.66	97.74	91.21	82.16	65.83
	0.4	0.6	93.83	88.89	95.48	**81.64**		98.49	92.96	**98.24**	**84.17**	97.99	91.21	82.41	66.83
	0.5	0.5	93.21	88.89	**96.05**	81.36		**98.74**	93.22	98.49	83.92	97.99	91.21	83.17	65.83
	0.6	0.4	93.21	**89.51**	95.48	**81.64**		**98.74**	**93.97**	**98.24**	83.92	97.99	**91.46**	83.67	65.58
	0.7	0.3	**94.44**	88.27	94.63	80.51		**98.74**	93.72	**98.24**	83.17	97.99	91.21	**84.42**	68.09
	0.8	0.2	93.83	88.89	94.63	79.38		98.49	93.97	**98.24**	82.91	98.49	90.7	**84.42**	**68.34**
	0.9	0.1	93.83	89.51	94.35	79.1		98.49	93.97	**98.24**	82.16	**98.74**	89.95	82.66	67.59
SMR27	0.1	0.9	92.59	80.86	94.92	79.94		98.49	93.97	98.24	83.92	96.98	89.7	78.14	68.34
	0.2	0.8	91.98	81.48	95.2	79.66		98.49	94.22	98.49	84.17	97.74	89.95	78.39	68.59
	0.3	0.7	92.59	82.1	95.2	**80.23**		98.74	94.72	98.49	84.17	97.49	90.2	79.9	**69.35**
	0.4	0.6	92.59	84.57	94.92	**80.23**		98.74	94.97	97.99	85.43	97.49	90.4	80.9	**69.35**
	0.5	0.5	93.21	85.19	**95.76**	79.38		**99.5**	94.97	97.99	**86.93**	98.49	91.71	**82.41**	68.84
	0.6	0.4	93.21	88.89	95.48	79.94		99.25	**95.23**	98.24	86.18	98.49	**91.96**	82.16	69.1
	0.7	0.3	**93.83**	88.89	95.2	79.66		98.74	94.72	**98.24**	85.43	98.49	91.21	82.16	68.34
	0.8	0.2	**93.83**	88.89	94.35	79.1		98.74	94.47	**98.24**	84.42	98.49	90.7	**82.41**	68.09
	0.9	0.1	**93.83**	**89.51**	94.63	78.81		98.74	93.97	**98.24**	82.66	**98.74**	90.45	81.66	67.84

4.7 Comparison with Other Studies

To compare our results, we have selected recent works that have considered compressed DeepFakes and are quite related to our study in terms of noise residual and backbone networks. In this study, the extensive performance matrix was provided as compared to existing studies. We provided the evaluation matrix of accuracy, f1, the precision, recall, and number of frames used. Deep Authenticator has outperformed all other studies in terms of scalability. The proposed method outperformed PRNU [44] and HFXception [253], which focus only on spatial analysis. The performance of PRNU [44] was quite low, which is a clear indication of the low performance of the non-deep learning methods in this domain. PRNU [44] use camera noise as features. The proposed solution and HFXception [253] both use RGB and Noise features with deep networks as a backbone network. The major difference between both architectures is the noise extraction process. The performance difference between our proposed method and HFXception [253] is comparatively low compared to PRNU [44]. The Dual Stream approach [39], which focused on both spatial and temporal features, outperformed our method on the NeuralTextures subset dataset of FaceForensics++, which indicates the importance of temporal features in the detection process of DeepFakes.

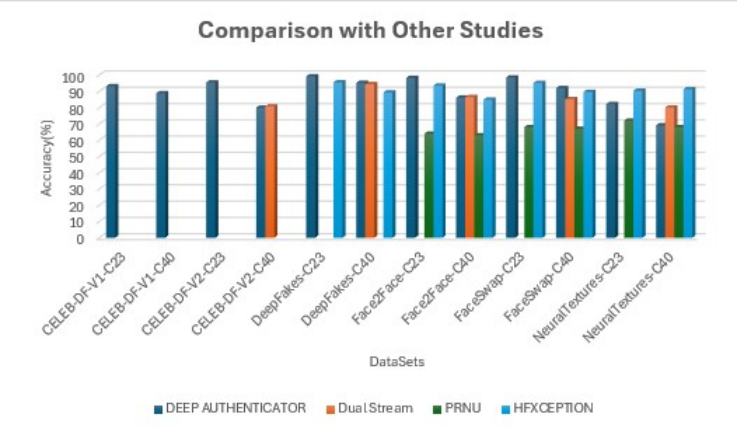


Figure 4.14: Comparison With Other Studies

Table 4.13: Comparison with Other Studies

Dataset	**Model**	**Acc (%)**	**Prec (%)**	**Recall (%)**	**F1(%)**	**Frames**
Celeb-DF-V1(C23)	**Dual-Stream[39]**	X	X	X	X	All
	PRNU[44]	X	X	X	X	20
	HFXception[253]	X	X	X	X	50
	Ours	**93.21**	88.89	98.77	93.57	3
Celeb-DF-V1(C40)	**Dual-Stream[39]**	X	X	X	X	All
	PRNU[44]	X	X	X	X	20
	HFXception[253]	X	X	X	X	50
	Ours	**88.89**	87.06	91.36	89.16	3
Celeb-DF-V2(C23)	**Dual-Stream[39]**	X	X	X	X	All
	PRNU[44]	X	X	X	X	20
	HFXception[253]	X	X	X	X	50
	Ours	**95.48**	94.97	96.05	95.51	3
Celeb-DF-V2(C40)	**Dual-Stream[39]**	**80.74**	X	X	X	All
	PRNU[44]	X	X	X	X	20
	HFXception[253]	X	X	X	X	50
	Ours	79.94	79.78	80.23	80.0	3
DeepFakes(C23)	**Dual-Stream[39]**	X	X	X	X	All
	PRNU[44]	X	X	X	X	20
	HFXception[253]	95.52	X	X	X	50
	Ours	**99.25**	100.0	98.49	99.24	3
DeepFakes(C40)	**Dual-Stream[39]**	94.46	X	X	X	All
	PRNU[44]	X	X	X	X	20
	HFXception[253]	89.28	X	X	X	50
	Ours	**95.23**	95.45	94.97	95.21	3
Face2Face(C23)	**Dual-Stream[39]**	X	X	X	X	All
	PRNU[44]	64	X	X	X	20
	HFXception[253]	93.49	X	X	X	50
	Ours	**98.24**	98.98	97.49	98.23	3
Face2Face(C40)	**Dual-Stream[39]**	**86.48**	X	X	X	All
	PRNU[44]	63	X	X	X	20
	HFXception[253]	84.85	X	X	X	50
	Ours	86.18	87.89	83.92	85.86	4
FaceSwap(C23)	**Dual-Stream[39]**	X	X	X	X	All
	PRNU[44]	68	X	X	X	20
	HFXception[253]	95.16	X	X	X	50
	Ours	**98.49**	100.0	96.98	98.47	3
FaceSwap(C40)	**Dual-Stream[39]**	85.27	X	X	X	All
	PRNU[44]	67	X	X	X	20
	HFXception[253]	89.6	X	X	X	50
	Ours	**91.96**	94.65	88.94	91.71	3
NeuralTextures(C23)	**Dual-Stream[39]**	X	X	X	X	All
	PRNU[44]	72	X	X	X	20
	HFXception[253]	**90.35**	X	X	X	50
	Ours	82.16	88.1	74.37	80.65	3
NeuralTextures(C40)	**Dual-Stream[39]**	80.05	X	X	X	All
	PRNU[44]	68	X	X	X	20
	HFXception[253]	**91.32**	X	X	X	50
	Ours	69.1	69.0	69.35	69.17	3

4.8 Concluding Remarks:

This chapter started by providing an overview of the dataset used and the experimental setup to obtain the results. The chapter also explained the evaluation metrics used and the model parameters of both branches of the proposed model. The evaluation results of the proposed model are presented in detail, and a comparison with the other models is provided. The results of the experiments demonstrated that the proposed model is effective at detecting DeepFakes. Experiments also show the effectiveness of using a deep learning model, data augmentation, and an enhanced number of filters. In the next chapter, we present our conclusion and discuss our future works.

Chapter 5

Conclusion & Future Works

5.1 Conclusion

The rapid progress in generative artificial intelligence and ease of access to social media have made it easy for cybercriminals to spread DeepFake videos for their malicious purposes. Many studies focus on the DeepFake detection problem, but limited works have aimed to detect DeepFake in compressed low-quality videos. In this thesis, We first review previous research on compressed DeepFake detection, providing a detailed survey of the different detections. As a solution to this problem, in this thesis, we have proposed a new model that uses semantics and residual features for detecting DeepFake videos. Compared to most of the existing studies, our model utilizes the storage structure of compressed videos. We also have provided extensive experiments on residual features for enhancing detection performance. We evaluated our method on three available datasets: Celeb-DF-V1, Celeb-DF-V2, and FaceForensics++. Our ensemble model outperformed baseline models in terms of performance. For future work, we aim to develop an efficient temporal feature extraction module to enhance the model's ability to capture temporal inconsistencies. We are also can

build a dataset consisting of DeepFake generated using diffusion models to enhance the evaluation of detection methods. Future work can also be enhanced by providing a biased analysis of DeepFake detection models based on factors such as gender, age, and skin color. Cross-compression level analysis can also be used to evaluate the model's performance.

5.1.1 Future Work

- For DeepFake detection, we aim to develop an efficient temporal feature extraction module to enhance the model's ability to capture temporal inconsistencies.
- Build a dataset consisting of DeepFake generated using diffusion models to enhance the evaluation of detection methods.
- Conduct experiments to analyze potential biases in DeepFake Detection models based on factors such as gender, age, and skin color.
- Conduct cross-compression level analysis to evaluate the impact of different compression levels on DeepFake detection performance.

Bibliography

[1] Introducing the Deepfake Sandbox - CivAI — civai.org. `https://civai.org/deepfake-sandbox`. [Accessed 11-02-2025].

[2] Reports - Sensity AI — sensity.ai. `https://sensity.ai/reports/`. [Accessed 17-02-2025].

[3] Yuezun Li, Xin Yang, Pu Sun, Honggang Qi, and Siwei Lyu. Celeb-df: A large-scale challenging dataset for deepfake forensics. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 3207–3216, 2020.

[4] Which Face Is Real? — whichfaceisreal.com. `https://www.whichfaceisreal.com/`. [Accessed 11-11-2023].

[5] FaceApp: Face Editor — faceapp.com. `https://www.faceapp.com/`. [Accessed 11-11-2023].

[6] impersonator_plus_plus — iperdance.github.io. `https://iperdance.github.io/work/impersonator-plus-plus.html`. [Accessed 07-03-2025].

[7] Yunes Al-Dhabi and Shuang Zhang. Deepfake video detection by combining convolutional neural network (cnn) and recurrent neural network (rnn). In *2021 IEEE International Conference on Computer Science, Artificial Intelligence and Electronic Engineering (CSAIEE)*, pages 236–241. IEEE, 2021.

[8] Zhang Hongmeng, Zhu Zhiqiang, Sun Lei, Mao Xiuqing, and Wang Yuehan. A detection method for deepfake hard compressed videos based on super-resolution reconstruction using cnn. In *Proceedings of the 2020 4th High Performance Computing and Cluster Technologies Conference & 2020 3rd International Conference on Big Data and Artificial Intelligence*, pages 98–103, 2020.

[9] Amna-Saga Humidan, Lili Nurliyana Abdullah, and Alfian Abdul Halin. Detection of compressed deepfake video drawbacks and technical developments. In *2022 5th International Conference on Signal Processing and Information Security (ICSPIS)*, pages 11–16. IEEE, 2022.

[10] Niteesh Kumar, P Pranav, Vishal Nirney, and V Geetha. Deepfake image detection using cnns and transfer learning. In *2021 International Conference on Computing, Communication and Green Engineering (CCGE)*, pages 1–6. IEEE, 2021.

[11] Yinglong Li. Research and application of deep learning in image recognition. In *2022 IEEE 2nd International Conference on Power, Electronics and Computer Applications (ICPECA)*, pages 994–999. IEEE, 2022.

[12] Wenyuan Yang, Xiaoyu Zhou, Zhikai Chen, Bofei Guo, Zhongjie Ba, Zhihua Xia, Xiaochun Cao, and Kui Ren. Avoid-df: Audio-visual joint learning for detecting deepfake. *IEEE Transactions on Information Forensics and Security*, 18:2015–2029, 2023.

[13] Yanfen Gan, Junliu Zhong, and Chiman Vong. A novel copy-move forgery detection algorithm via feature label matching and hierarchical segmentation filtering. *Information Processing & Management*, 59(1):102783, 2022.

[14] Muhammad Imran, Zulfiqar Ali, Sheikh Tahir Bakhsh, and Sheeraz Akram. Blind detection of copy-move forgery in digital audio forensics. *IEEE Access*, 5:12843–12855, 2017.

[15] Luisa Verdoliva. Media forensics and deepfakes: an overview. *IEEE Journal of Selected Topics in Signal Processing*, 14(5):910–932, 2020.

[16] Staffy Kingra, Naveen Aggarwal, and Nirmal Kaur. Lbpnet: Exploiting texture descriptor for deepfake detection. *Forensic Science International: Digital Investigation*, 42:301452, 2022.

[17] Yuan Lin, Shengjin Wang, Qian Lin, and Feng Tang. Face swapping under large pose variations: A 3d model based approach. In *2012 IEEE International Conference on Multimedia and Expo*, pages 333–338. IEEE, 2012.

[18] Supasorn Suwajanakorn, Steven M Seitz, and Ira Kemelmacher-Shlizerman. What makes tom hanks look like tom hanks. In *Proceedings of the IEEE international conference on computer vision*, pages 3952–3960, 2015.

[19] Mladan Jovanovic and Mark Campbell. Generative artificial intelligence: Trends and prospects. *Computer*, 55(10):107–112, 2022.

[20] Yiqing Hua, Shuo Niu, Jie Cai, Lydia B Chilton, Hendrik Heuer, and Donghee Yvette Wohn. Generative ai in user-generated content. In *Extended Abstracts of the CHI Conference on Human Factors in Computing Systems*, pages 1–7, 2024.

[21] Adrian Schuckart. Genai and prompt engineering: A progressive framework for empowering the workforce. In *Proceedings of the 29th European Conference on Pattern Languages of Programs, People, and Practices*, pages 1–8, 2024.

[22] Aditya Ramesh, Mikhail Pavlov, Gabriel Goh, Scott Gray, Chelsea Voss, Alec Radford, Mark Chen, and Ilya Sutskever. Zero-shot text-to-image generation. In *International conference on machine learning*, pages 8821–8831. Pmlr, 2021.

[23] Yixin Liu, Kai Zhang, Yuan Li, Zhiling Yan, Chujie Gao, Ruoxi Chen, Zhengqing Yuan, Yue Huang, Hanchi Sun, Jianfeng Gao, et al. Sora: A review on background, technology, limitations, and opportunities of large vision models. *arXiv preprint arXiv:2402.17177*, 2024.

[24] Alec Radford. Improving language understanding by generative pre-training. 2018.

[25] Xiao Bi, Deli Chen, Guanting Chen, Shanhuang Chen, Damai Dai, Chengqi Deng, Honghui Ding, Kai Dong, Qiushi Du, Zhe Fu, et al. Deepseek llm: Scaling open-source language models with longtermism. *arXiv preprint arXiv:2401.02954*, 2024.

[26] Vinay Chamola, Gaurang Bansal, Tridib Kumar Das, Vikas Hassija, Siva Sai, Jiacheng Wang, Sherali Zeadally, Amir Hussain, Fei Richard Yu, Mohsen Guizani, et al. Beyond reality: The pivotal role of generative ai in the metaverse. *IEEE Internet of Things Magazine*, 7(4):126–135, 2024.

[27] Giulia Bertazzini, Daniele Baracchi, Dasara Shullani, Massimo Iuliani, and Alessandro Piva. A codec-based approach for video life-cycle characterization in social networks. In *ICASSP 2024-2024 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 4790–4794. IEEE, 2024.

[28] Murtaza Ahmed Siddiqi, Wooguil Pak, and Moquddam A Siddiqi. A study on the psychology of social engineering-based cyberattacks and existing countermeasures. *Applied Sciences*, 12(12):6042, 2022.

[29] Deepfakes porn has serious consequences — bbc.com. https://www.bbc.com/news/technology-42912529. [Accessed 06-01-2025].

[30] - YouTube — youtube.com. https://www.youtube.com/watch?v=dRgUk2jEmeM. [Accessed 06-01-2025].

[31] - YouTube — youtube.com. https://www.youtube.com/watch?v=7ZVdb86upR4. [Accessed 06-01-2025].

[32] - YouTube — youtube.com. https://www.youtube.com/watch?v=X17yrEV5sl4. [Accessed 06-01-2025].

[33] - YouTube — youtube.com. https://www.youtube.com/watch?v=Q_KeJW41SqA. [Accessed 06-01-2025].

[34] - YouTube — youtube.com. https://www.youtube.com/watch?v=1bJwFxpF3tE. [Accessed 06-01-2025].

[35] Yi Zhu, Sharath Chandra Guntuku, Weisi Lin, Gheorghita Ghinea, and Judith A Redi. Measuring individual video qoe: A survey, and proposal for future directions using social media. *ACM Transactions on Multimedia Computing, Communications, and Applications (TOMM)*, 14(2s):1–24, 2018.

[36] Zeina Sinno and Alan Conrad Bovik. Large-scale study of perceptual video quality. *IEEE Transactions on Image Processing*, 28(2):612–627, 2018.

[37] Irene Amerini, Aris Anagnostopoulos, Luca Maiano, and Lorenzo Ricciardi Celsi. Learning double-compression video fingerprints left from social-media platforms. In *ICASSP 2021-2021 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 2530–2534. IEEE, 2021.

[38] Natalia Molinero Mingorance. The cost of uploading a video to social media explained: the inefficiency of current video compression algorithms. *Authorea Preprints*, 2023.

[39] Juan Hu, Xin Liao, Wei Wang, and Zheng Qin. Detecting compressed deepfake videos in social networks using frame-temporality two-stream convolutional network. *IEEE Transactions on Circuits and Systems for Video Technology*, 32(3):1089–1102, 2021.

[40] Yaohui Wang and Antitza Dantcheva. A video is worth more than 1000 lies. comparing 3dcnn approaches for detecting deepfakes. In *2020 15Th IEEE international conference on automatic face and gesture recognition (FG 2020)*, pages 515–519. IEEE, 2020.

[41] Davide Cozzolino, Andreas Rössler, Justus Thies, Matthias Nießner, and Luisa Verdoliva. Id-reveal: Identity-aware deepfake video detection. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 15108–15117, 2021.

[42] Darius Afchar, Vincent Nozick, Junichi Yamagishi, and Isao Echizen. Mesonet: a compact facial video forgery detection network. In *2018 IEEE international workshop on information forensics and security (WIFS)*, pages 1–7. IEEE, 2018.

[43] Prabhat Kumar, Mayank Vatsa, and Richa Singh. Detecting face2face facial reenactment in videos. In *Proceedings of the IEEE/CVF winter conference on applications of computer vision*, pages 2589–2597, 2020.

[44] Florian Lugstein, Simon Baier, Gregor Bachinger, and Andreas Uhl. Prnu-based deepfake detection. In *Proceedings of the 2021 ACM workshop on information hiding and multimedia security*, pages 7–12, 2021.

[45] Lingzhi Li, Jianmin Bao, Ting Zhang, Hao Yang, Dong Chen, Fang Wen, and Baining Guo. Face x-ray for more general face forgery detection. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 5001–5010, 2020.

[46] Xu Chang, Jian Wu, Tongfeng Yang, and Guorui Feng. Deepfake face image detection based on improved vgg convolutional neural network. In *2020 39th chinese control conference (CCC)*, pages 7252–7256. IEEE, 2020.

[47] Bing Han, Xiaoguang Han, Hua Zhang, Jingzhi Li, and Xiaochun Cao. Fighting fake news: two stream network for deepfake detection via learnable srm. *IEEE Transactions on Biometrics, Behavior, and Identity Science*, 3(3):320–331, 2021.

[48] Peng Zhou, Xintong Han, Vlad I Morariu, and Larry S Davis. Learning rich features for image manipulation detection. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 1053–1061, 2018.

[49] Internet Speeds by Country 2024 — worldpopulationreview.com. `https://worldpopulationreview.com/country-rankings/internet-speeds-by-country`. [Accessed 06-01-2025].

[50] Heru Chakra Setiawan, Widyo Nugroho, and Habib Abdur Rofi. The importance of video as learning media according to principle of media production "visuals". *Interdiciplinary Journal and Hummanity (INJURITY)*, 1(3):92–97, 2022.

[51] Eric Dubois. The sampling and reconstruction of time-varying imagery with application in video systems. *Proceedings of the IEEE*, 73(4):502–522, 1985.

[52] Majid Rabbani and Paul W Jones. *Digital image compression techniques*, volume 7. SPIE press, 1991.

[53] Matt Poyser, Amir Atapour-Abarghouei, and Toby P Breckon. On the impact of lossy image and video compression on the performance of deep convolutional neural network architectures. In *2020 25th international conference on pattern recognition (ICPR)*, pages 2830–2837. IEEE, 2021.

[54] Trinh Man Hoang and Jinjia Zhou. Recent trending on learning based video compression: A survey. *Cognitive Robotics*, 1:145–158, 2021.

[55] Muhammad Zubair and Saqib Hakak. Exploring the landscape of compressed deepfakes: Generation, dataset and detection. *Neurocomputing*, 619:129116, 2025.

[56] John Li, Shehab Sarar Ahmed, and Deepak Nair. Framecorr: Adaptive, autoencoder-based neural compression for video reconstruction in resource and timing constrained network settings. *arXiv preprint arXiv:2409.02453*, 2024.

[57] Hichem Felouat, Huy H Nguyen, Trung-Nghia Le, Junichi Yamagishi, and Isao Echizen. ekyc-df: A large-scale deepfake dataset for developing and evaluating ekyc systems. *IEEE Access*, 2024.

[58] Adrienne De Ruiter. The distinct wrong of deepfakes. *Philosophy & Technology*, 34(4):1311–1332, 2021.

[59] Brandon Khoo, Raphaël C-W Phan, and Chern-Hong Lim. Deepfake attribution: On the source identification of artificially generated images. *Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery*, 12(3):e1438, 2022.

[60] Bo Peng, Hongxing Fan, Wei Wang, Jing Dong, and Siwei Lyu. A unified framework for high fidelity face swap and expression reenactment. *IEEE Transactions on Circuits and Systems for Video Technology*, 32(6):3673–3684, 2021.

[61] Qi Guo, Ce Zhu, Zhiqiang Xia, Zhengtao Wang, and Yipeng Liu. Attribute-controlled face photo synthesis from simple line drawing. In *2017 IEEE international conference on image processing (ICIP)*, pages 2946–2950. IEEE, 2017.

[62] Achhardeep Kaur, Azadeh Noori Hoshyar, Vidya Saikrishna, Selena Firmin, and Feng Xia. Deepfake video detection: challenges and opportunities. *Artificial Intelligence Review*, 57(6):1–47, 2024.

[63] Ruben Tolosana, Ruben Vera-Rodriguez, Julian Fierrez, Aythami Morales, and Javier Ortega-Garcia. Deepfakes and beyond: A survey of face manipulation and fake detection. *Information Fusion*, 64:131–148, 2020.

[64] Momina Masood, Mariam Nawaz, Khalid Mahmood Malik, Ali Javed, Aun Irtaza, and Hafiz Malik. Deepfakes generation and detection: State-of-the-art, open challenges, countermeasures, and way forward. *Applied intelligence*, 53(4):3974–4026, 2023.

[65] Ilkka Kaate, Joni Salminen, Soon-Gyo Jung, Nina Rizun, Aleksandra Revina, and Bernard J Jansen. Getting emotional enough: Analyzing emotional diversity in deepfake avatars. In *Proceedings of the 13th Nordic Conference on Human-Computer Interaction*, pages 1–12, 2024.

[66] Lingjie Liu, Weipeng Xu, Michael Zollhoefer, Hyeongwoo Kim, Florian Bernard, Marc Habermann, Wenping Wang, and Christian Theobalt. Neural rendering and reenactment of human actor videos. *ACM Transactions on Graphics (TOG)*, 38(5):1–14, 2019.

[67] Aliaksandr Siarohin, Enver Sangineto, Stéphane Lathuiliere, and Nicu Sebe. Deformable gans for pose-based human image generation. In *Proceedings of the*

IEEE conference on computer vision and pattern recognition, pages 3408–3416, 2018.

[68] Yue Wu and Qiang Ji. Facial landmark detection: A literature survey. *International Journal of Computer Vision*, 127(2):115–142, 2019.

[69] Lulu Chen, Hong Wei, and James Ferryman. A survey of human motion analysis using depth imagery. *Pattern Recognition Letters*, 34(15):1995–2006, 2013.

[70] Bernhard Egger, William AP Smith, Ayush Tewari, Stefanie Wuhrer, Michael Zollhoefer, Thabo Beeler, Florian Bernard, Timo Bolkart, Adam Kortylewski, Sami Romdhani, et al. 3d morphable face models—past, present, and future. *ACM Transactions on Graphics (ToG)*, 39(5):1–38, 2020.

[71] FacePlay - AI Filter&Face Swap - Apps on Google Play — play.google.com. `https://play.google.com/store/apps/details?id=com.ai.face.play&hl=en`. [Accessed 16-09-2024].

[72] Reface: Face Swap AI Photo App - Apps on Google Play — play.google.com. `https://play.google.com/store/apps/details?id=video.reface.app&hl=en_CA`. [Accessed 16-09-2024].

[73] Diff AI - Animate Anyone Dance - Apps on Google Play — play.google.com. `https://play.google.com/store/apps/details?id=ai.dream.diff.www&hl=en_CA`. [Accessed 16-09-2024].

[74] FaceHub -AI Photo&Face Swap - Apps on Google Play — play.google.com. `https://play.google.com/store/apps/details?id=com.facehub.deepswap&hl=en_CA`. [Accessed 15-09-2024].

[75] Kunlin Liu, Ivan Perov, Daiheng Gao, Nikolay Chervoniy, Wenbo Zhou, and Weiming Zhang. Deepfacelab: Integrated, flexible and extensible face-swapping framework. *Pattern Recognition*, 141:109628, 2023.

[76] Lingzhi Li, Jianmin Bao, Hao Yang, Dong Chen, and Fang Wen. Faceshifter: Towards high fidelity and occlusion aware face swapping. *arXiv preprint arXiv:1912.13457*, 2019.

[77] Welcome - Faceswap — faceswap.dev. `https://faceswap.dev/`. [Accessed 03-08-2024].

[78] GitHub - dfaker/df: Larger resolution face masked, weirdly warped, deepfake, — github.com. `https://github.com/dfaker/df`. [Accessed 03-08-2024].

[79] GitHub - shaoanlu/faceswap-GAN: A denoising autoencoder + adversarial losses and attention mechanisms for face swapping. — github.com. `https://github.com/shaoanlu/faceswap-GAN`. [Accessed 03-08-2024].

[80] Yu Deng, Jiaolong Yang, Dong Chen, Fang Wen, and Xin Tong. Disentangled and controllable face image generation via 3d imitative-contrastive learning. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 5154–5163, 2020.

[81] Tero Karras, Samuli Laine, and Timo Aila. A style-based generator architecture for generative adversarial networks. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 4401–4410, 2019.

[82] Tero Karras, Samuli Laine, Miika Aittala, Janne Hellsten, Jaakko Lehtinen, and Timo Aila. Analyzing and improving the image quality of stylegan. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 8110–8119, 2020.

[83] Yunjey Choi, Minje Choi, Munyoung Kim, Jung-Woo Ha, Sunghun Kim, and Jaegul Choo. Stargan: Unified generative adversarial networks for multi-domain image-to-image translation. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 8789–8797, 2018.

[84] Justus Thies, Michael Zollhofer, Marc Stamminger, Christian Theobalt, and Matthias Nießner. Face2face: Real-time face capture and reenactment of rgb videos. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 2387–2395, 2016.

[85] Justus Thies, Michael Zollhöfer, and Matthias Nießner. Deferred neural rendering: Image synthesis using neural textures. *Acm Transactions on Graphics (TOG)*, 38(4):1–12, 2019.

[86] Caroline Chan, Shiry Ginosar, Tinghui Zhou, and Alexei A Efros. Everybody dance now. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 5933–5942, 2019.

[87] Wen Liu, Zhixin Piao, Zhi Tu, Wenhan Luo, Lin Ma, and Shenghua Gao. Liquid warping gan with attention: A unified framework for human image synthesis. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 44(9):5114–5132, 2021.

[88] Phyo Thet Yee, Sudeepta Mishra, and Abhinav Dhall. Clipswap: Towards high fidelity face swapping via attributes and clip-informed loss. In *2024 IEEE 18th International Conference on Automatic Face and Gesture Recognition (FG)*, pages 1–10. IEEE, 2024.

[89] Wenliang Zhao, Yongming Rao, Weikang Shi, Zuyan Liu, Jie Zhou, and Jiwen Lu. Diffswap: High-fidelity and controllable face swapping via 3d-aware masked

diffusion. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 8568–8577, 2023.

[90] Kihong Kim, Yunho Kim, Seokju Cho, Junyoung Seo, Jisu Nam, Kychul Lee, Seungryong Kim, and KwangHee Lee. Diffface: Diffusion-based face swapping with facial guidance. *arXiv preprint arXiv:2212.13344*, 2022.

[91] GitHub - iperov/DeepFaceLive: Real-time face swap for PC streaming or video calls — github.com. `https://github.com/iperov/DeepFaceLive`. [Accessed 12-09-2024].

[92] GitHub - hacksider/Deep-Live-Cam: real time face swap and one-click video deepfake with only a single image — github.com. `https://github.com/hacksider/Deep-Live-Cam`. [Accessed 12-09-2024].

[93] GitHub - hassan-sd/facefusion-unlocked: Next generation face swapper and enhancer - no NSFW checker — github.com. `https://github.com/hassan-sd/facefusion-unlocked`. [Accessed 12-09-2024].

[94] GitHub - titogabus/ComfyUI-InstaSwap — github.com. `https://github.com/titogabus/ComfyUI-InstaSwap`. [Accessed 12-09-2024].

[95] Jianzhu Guo, Dingyun Zhang, Xiaoqiang Liu, Zhizhou Zhong, Yuan Zhang, Pengfei Wan, and Di Zhang. Liveportrait: Efficient portrait animation with stitching and retargeting control. *arXiv preprint arXiv:2407.03168*, 2024.

[96] Reshot AI - The best AI photo editor for professional faceshots — reshot.ai. `https://www.reshot.ai/`. [Accessed 12-09-2024].

[97] Make Your Own Deepfakes [Online App] — deepfakesweb.com. `https://deepfakesweb.com/`. [Accessed 03-08-2024].

[98] David E Rumelhart, Geoffrey E Hinton, and Ronald J Williams. Learning representations by back-propagating errors. *nature*, 323(6088):533–536, 1986.

[99] Pengzhi Li, Yan Pei, and Jianqiang Li. A comprehensive survey on design and application of autoencoder in deep learning. *Applied Soft Computing*, page 110176, 2023.

[100] Artem A Maksutov, Viacheslav O Morozov, Aleksander A Lavrenov, and Alexander S Smirnov. Methods of deepfake detection based on machine learning. In *2020 IEEE conference of russian young researchers in electrical and electronic engineering (EIConRus)*, pages 408–411. IEEE, 2020.

[101] Pascal Vincent, Hugo Larochelle, Yoshua Bengio, and Pierre-Antoine Manzagol. Extracting and composing robust features with denoising autoencoders. In *Proceedings of the 25th international conference on Machine learning*, pages 1096–1103, 2008.

[102] Andrew Ng et al. Sparse autoencoder. *CS294A Lecture notes*, 72(2011):1–19, 2011.

[103] Jonathan Masci, Ueli Meier, Dan Cireşan, and Jürgen Schmidhuber. Stacked convolutional auto-encoders for hierarchical feature extraction. In *Artificial Neural Networks and Machine Learning–ICANN 2011: 21st International Conference on Artificial Neural Networks, Espoo, Finland, June 14-17, 2011, Proceedings, Part I 21*, pages 52–59. Springer, 2011.

[104] Salah Rifai, Pascal Vincent, Xavier Muller, Xavier Glorot, and Yoshua Bengio. Contractive auto-encoders: Explicit invariance during feature extraction. In *Proceedings of the 28th international conference on international conference on machine learning*, pages 833–840, 2011.

[105] Alireza Makhzani and Brendan Frey. K-sparse autoencoders. *arXiv preprint arXiv:1312.5663*, 2013.

[106] Diederik P Kingma and Max Welling. Auto-encoding variational bayes. *arXiv preprint arXiv:1312.6114*, 2013.

[107] Kihyuk Sohn, Honglak Lee, and Xinchen Yan. Learning structured output representation using deep conditional generative models. *Advances in neural information processing systems*, 28, 2015.

[108] Alireza Makhzani, Jonathon Shlens, Navdeep Jaitly, Ian Goodfellow, and Brendan Frey. Adversarial autoencoders. *arXiv preprint arXiv:1511.05644*, 2015.

[109] Christos Louizos, Kevin Swersky, Yujia Li, Max Welling, and Richard Zemel. The variational fair autoencoder. *arXiv preprint arXiv:1511.00830*, 2015.

[110] Ishaan Gulrajani, Kundan Kumar, Faruk Ahmed, Adrien Ali Taiga, Francesco Visin, David Vazquez, and Aaron Courville. Pixelvae: A latent variable model for natural images. *arXiv preprint arXiv:1611.05013*, 2016.

[111] Aäron Van Den Oord, Nal Kalchbrenner, and Koray Kavukcuoglu. Pixel recurrent neural networks. In *International conference on machine learning*, pages 1747–1756. PMLR, 2016.

[112] Jinzhou Bao, Bo Zhao, and Ping Guo. Kernel-based autoencoders for large-scale representation learning. In *Proceedings of the 7th International Conference on Robotics and Artificial Intelligence*, pages 112–117, 2021.

[113] Kaiming He, Xinlei Chen, Saining Xie, Yanghao Li, Piotr Dollár, and Ross Girshick. Masked autoencoders are scalable vision learners. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 16000–16009, 2022.

[114] Zhan Tong, Yibing Song, Jue Wang, and Limin Wang. Videomae: Masked autoencoders are data-efficient learners for self-supervised video pre-training. *Advances in neural information processing systems*, 35:10078–10093, 2022.

[115] Ian Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. Generative adversarial nets. *Advances in neural information processing systems*, 27, 2014.

[116] Samuel Henrique Silva, Mazal Bethany, Alexis Megan Votto, Ian Henry Scarff, Nicole Beebe, and Peyman Najafirad. Deepfake forensics analysis: An explainable hierarchical ensemble of weakly supervised models. *Forensic Science International: Synergy*, 4:100217, 2022.

[117] Olaoluwa Adigun and Bart Kosko. Training generative adversarial networks with bidirectional backpropagation. In *2018 17th IEEE international conference on machine learning and applications (ICMLA)*, pages 1178–1185. IEEE, 2018.

[118] GitHub - nipunmanral/Cross-Domain-Image-Translation-Using-CycleGAN: CycleGAN based neural network architecture to change the gender of a person's face — github.com. `https://github.com/nipunmanral/Cross-Domain-Image-Translation-Using-CycleGAN?tab=readme-ov-file`. [Accessed 07-08-2024].

[119] Vaibhav Parate, Arka Dey, Manas Bhandarkar, Saadiya Dafedar, and Satishkumar Chavan. Age progression and regression using cyclegan. In *2022 IEEE 3rd Global Conference for Advancement in Technology (GCAT)*, pages 1–6. IEEE, 2022.

[120] Alec Radford, Luke Metz, and Soumith Chintala. Unsupervised representation learning with deep convolutional generative adversarial networks. *arXiv preprint arXiv:1511.06434*, 2015.

[121] Martin Arjovsky, Soumith Chintala, and Léon Bottou. Wasserstein generative adversarial networks. In *International conference on machine learning*, pages 214–223. PMLR, 2017.

[122] Tero Karras, Timo Aila, Samuli Laine, and Jaakko Lehtinen. Progressive growing of gans for improved quality, stability, and variation. *arXiv preprint arXiv:1710.10196*, 2017.

[123] Phillip Isola, Jun-Yan Zhu, Tinghui Zhou, and Alexei A Efros. Image-to-image translation with conditional adversarial networks. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 1125–1134, 2017.

[124] Casey Chu, Andrey Zhmoginov, and Mark Sandler. Cyclegan, a master of steganography. *arXiv preprint arXiv:1712.02950*, 2017.

[125] Zili Yi, Hao Zhang, Ping Tan, and Minglun Gong. Dualgan: Unsupervised dual learning for image-to-image translation. In *Proceedings of the IEEE international conference on computer vision*, pages 2849–2857, 2017.

[126] Taeksoo Kim, Moonsu Cha, Hyunsoo Kim, Jung Kwon Lee, and Jiwon Kim. Learning to discover cross-domain relations with generative adversarial networks. In *International conference on machine learning*, pages 1857–1865. PMLR, 2017.

[127] Han Zhang, Tao Xu, Hongsheng Li, Shaoting Zhang, Xiaogang Wang, Xiaolei Huang, and Dimitris N Metaxas. Stackgan: Text to photo-realistic image synthesis with stacked generative adversarial networks. In *Proceedings of the IEEE international conference on computer vision*, pages 5907–5915, 2017.

[128] Guillermo Iglesias, Edgar Talavera, and Alberto Díaz-Álvarez. A survey on gans for computer vision: Recent research, analysis and taxonomy. *Computer Science Review*, 48:100553, 2023.

[129] Lidan Wang, Vishwanath Sindagi, and Vishal Patel. High-quality facial photo-sketch synthesis using multi-adversarial networks. In *2018 13th IEEE international conference on automatic face & gesture recognition (FG 2018)*, pages 83–90. IEEE, 2018.

[130] Han Zhang, Tao Xu, Hongsheng Li, Shaoting Zhang, Xiaogang Wang, Xiaolei Huang, and Dimitris N Metaxas. Stackgan++: Realistic image synthesis with stacked generative adversarial networks. *IEEE transactions on pattern analysis and machine intelligence*, 41(8):1947–1962, 2018.

[131] Yali Cai, Xiaoru Wang, Zhihong Yu, Fu Li, Peirong Xu, Yueli Li, and Lixian Li. Dualattn-gan: Text to image synthesis with dual attentional generative adversarial network. *IEEE Access*, 7:183706–183716, 2019.

[132] Zhangling Chen, Ce Wang, Huaming Wu, Kun Shang, and Jun Wang. Dmgan: Discriminative metric-based generative adversarial networks. *Knowledge-Based Systems*, 192:105370, 2020.

[133] Tero Karras, Miika Aittala, Samuli Laine, Erik Härkönen, Janne Hellsten, Jaakko Lehtinen, and Timo Aila. Alias-free generative adversarial networks. *Advances in Neural Information Processing Systems*, 34:852–863, 2021.

[134] Kancharagunta Kishan Babu and Shiv Ram Dubey. Csgan: Cyclic-synthesized generative adversarial networks for image-to-image transformation. *Expert Systems with Applications*, 169:114431, 2021.

[135] Ming Tao, Hao Tang, Fei Wu, Xiao-Yuan Jing, Bing-Kun Bao, and Changsheng Xu. Df-gan: A simple and effective baseline for text-to-image synthesis. In

Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, pages 16515–16525, 2022.

[136] Qi Liang, Qiang Li, and Weizhi Nie. Ld-gan: Learning perturbations for adversarial defense based on gan structure. *Signal Processing: Image Communication*, 103:116659, 2022.

[137] Jascha Sohl-Dickstein, Eric Weiss, Niru Maheswaranathan, and Surya Ganguli. Deep unsupervised learning using nonequilibrium thermodynamics. In *International conference on machine learning*, pages 2256–2265. PMLR, 2015.

[138] Jun Li, Chenyang Zhang, Wei Zhu, and Yawei Ren. A comprehensive survey of image generation models based on deep learning. *Annals of Data Science*, pages 1–30, 2024.

[139] Olaf Ronneberger, Philipp Fischer, and Thomas Brox. U-net: Convolutional networks for biomedical image segmentation. In *Medical image computing and computer-assisted intervention–MICCAI 2015: 18th international conference, Munich, Germany, October 5-9, 2015, proceedings, part III 18*, pages 234–241. Springer, 2015.

[140] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. *Advances in neural information processing systems*, 33:6840–6851, 2020.

[141] Prafulla Dhariwal and Alexander Nichol. Diffusion models beat gans on image synthesis. *Advances in neural information processing systems*, 34:8780–8794, 2021.

[142] Chitwan Saharia, William Chan, Saurabh Saxena, Lala Li, Jay Whang, Emily L Denton, Kamyar Ghasemipour, Raphael Gontijo Lopes, Burcu Karagol Ayan, Tim Salimans, et al. Photorealistic text-to-image diffusion

models with deep language understanding. *Advances in Neural Information Processing Systems*, 35:36479–36494, 2022.

[143] Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Björn Ommer. High-resolution image synthesis with latent diffusion models. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 10684–10695, 2022.

[144] Lvmin Zhang, Anyi Rao, and Maneesh Agrawala. Adding conditional control to text-to-image diffusion models. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 3836–3847, 2023.

[145] Xingqian Xu, Zhangyang Wang, Gong Zhang, Kai Wang, and Humphrey Shi. Versatile diffusion: Text, images and variations all in one diffusion model. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 7754–7765, 2023.

[146] Animesh Karnewar, Andrea Vedaldi, David Novotny, and Niloy J Mitra. Holodiffusion: Training a 3d diffusion model using 2d images. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 18423–18433, 2023.

[147] Gaurav Parmar, Krishna Kumar Singh, Richard Zhang, Yijun Li, Jingwan Lu, and Jun-Yan Zhu. Zero-shot image-to-image translation. In *ACM SIGGRAPH 2023 Conference Proceedings*, pages 1–11, 2023.

[148] Andreas Rössler, Davide Cozzolino, Luisa Verdoliva, Christian Riess, Justus Thies, and Matthias Nießner. FaceForensics++: Learning to detect manipulated facial images. In *International Conference on Computer Vision (ICCV)*, 2019.

[149] Jiajun Huang, Xueyu Wang, Bo Du, Pei Du, and Chang Xu. Deepfake mnist+: a deepfake facial animation dataset. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 1973–1982, 2021.

[150] FaceForensics/dataset/README.md at master · ondyari/FaceForensics — github.com. `https://github.com/ondyari/FaceForensics/blob/master/dataset/README.md`. [Accessed 14-Jul-2023].

[151] Kartik Narayan, Harsh Agarwal, Kartik Thakral, Surbhi Mittal, Mayank Vatsa, and Richa Singh. Df-platter: Multi-face heterogeneous deepfake dataset. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 9739–9748, 2023.

[152] Arsha Nagrani, Joon Son Chung, Weidi Xie, and Andrew Zisserman. Voxceleb: Large-scale speaker verification in the wild. *Computer Speech & Language*, 60:101027, 2020.

[153] Job Van Der Schalk, Skyler T Hawk, Agneta H Fischer, and Bertjan Doosje. Moving faces, looking places: validation of the amsterdam dynamic facial expression set (adfes). *Emotion*, 11(4):907, 2011.

[154] Liming Jiang, Ren Li, Wayne Wu, Chen Qian, and Chen Change Loy. Deeperforensics-1.0: A large-scale dataset for real-world face forgery detection. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 2889–2898, 2020.

[155] Alexander Groshev, Anastasia Maltseva, Daniil Chesakov, Andrey Kuznetsov, and Denis Dimitrov. Ghost—a new face swap approach for image and video domains. *IEEE Access*, 10:83452–83462, 2022.

[156] Renwang Chen, Xuanhong Chen, Bingbing Ni, and Yanhao Ge. Simswap: An efficient framework for high fidelity face swapping. In *Proceedings of the 28th ACM international conference on multimedia*, pages 2003–2011, 2020.

[157] Felix Rosberg, Eren Erdal Aksoy, Fernando Alonso-Fernandez, and Cristofer Englund. Facedancer: Pose-and occlusion-aware high fidelity face swapping. In *Proceedings of the IEEE/CVF winter conference on applications of computer vision*, pages 3454–3463, 2023.

[158] Hasam Khalid, Shahroz Tariq, Minha Kim, and Simon S Woo. Fakeavceleb: A novel audio-video multimodal deepfake dataset. *arXiv preprint arXiv:2108.05080*, 2021.

[159] Iryna Korshunova, Wenzhe Shi, Joni Dambre, and Lucas Theis. Fast face-swap using convolutional neural networks. In *Proceedings of the IEEE international conference on computer vision*, pages 3677–3685, 2017.

[160] Yuval Nirkin, Yosi Keller, and Tal Hassner. Fsgan: Subject agnostic face swapping and reenactment. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 7184–7193, 2019.

[161] Ye Jia, Yu Zhang, Ron Weiss, Quan Wang, Jonathan Shen, Fei Ren, Patrick Nguyen, Ruoming Pang, Ignacio Lopez Moreno, Yonghui Wu, et al. Transfer learning from speaker verification to multispeaker text-to-speech synthesis. *Advances in neural information processing systems*, 31, 2018.

[162] KR Prajwal, Rudrabha Mukhopadhyay, Vinay P Namboodiri, and CV Jawahar. A lip sync expert is all you need for speech to lip generation in the wild. In *Proceedings of the 28th ACM international conference on multimedia*, pages 484–492, 2020.

[163] Pavel Korshunov and Sébastien Marcel. Deepfakes: a new threat to face recognition? assessment and detection. *arXiv preprint arXiv:1812.08685*, 2018.

[164] Conrad Sanderson - VidTIMIT dataset — conradsanderson.id.au. `https://conradsanderson.id.au/vidtimit/`. [Accessed 20-Jul-2023].

[165] Carl Doersch. Tutorial on variational autoencoders. *arXiv preprint arXiv:1606.05908*, 2016.

[166] Bojia Zi, Minghao Chang, Jingjing Chen, Xingjun Ma, and Yu-Gang Jiang. Wilddeepfake: A challenging real-world dataset for deepfake detection. In *Proceedings of the 28th ACM international conference on multimedia*, pages 2382–2390, 2020.

[167] YouTube — youtube.com. `https://www.youtube.com/`. [Accessed 04-03-2025].

[168] Alain Hore and Djemel Ziou. Image quality metrics: Psnr vs. ssim. In *2010 20th international conference on pattern recognition*, pages 2366–2369. IEEE, 2010.

[169] Patrick Kwon, Jaeseong You, Gyuhyeon Nam, Sungwoo Park, and Gyeongsu Chae. Kodf: A large-scale korean deepfake detection dataset. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 10744–10753, 2021.

[170] Geon-Woo Park, Eun-Ju Park, and Simon S Woo. Zoom-df: a dataset for video conferencing deepfake. In *Proceedings of the 1st Workshop on Security Implications of Deepfakes and Cheapfakes*, pages 7–11, 2022.

[171] One platform to connect — Zoom — zoom.com. `https://www.zoom.com/`. [Accessed 04-03-2025].

[172] https://www.webex.com/. [Accessed 04-03-2025].

[173] Google Workspace. Google Meet: Online Web and Video Conferencing Calls — Google Workspace — meet.google.com. https://meet.google.com/landing. [Accessed 04-03-2025].

[174] Avatarify — Bring your photos to life — avatarify.ai. https://avatarify.ai/. [Accessed 04-03-2025].

[175] Aliaksandr Siarohin, Stéphane Lathuilière, Sergey Tulyakov, Elisa Ricci, and Nicu Sebe. First order motion model for image animation. *Advances in neural information processing systems*, 32, 2019.

[176] Kartik Narayan, Harsh Agarwal, Kartik Thakral, Surbhi Mittal, Mayank Vatsa, and Richa Singh. Deephy: On deepfake phylogeny. In *2022 IEEE International Joint Conference on Biometrics (IJCB)*, pages 1–10. IEEE, 2022.

[177] Tianfei Zhou, Wenguan Wang, Zhiyuan Liang, and Jianbing Shen. Face forensics in the wild. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 5778–5788, 2021.

[178] Yuan Gong, Yong Zhang, Xiaodong Cun, Fei Yin, Yanbo Fan, Xuan Wang, Baoyuan Wu, and Yujiu Yang. Toontalker: Cross-domain face reenactment. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 7690–7700, 2023.

[179] Brian Dolhansky, Joanna Bitton, Ben Pflaum, Jikuo Lu, Russ Howes, Menglin Wang, and Cristian Canton Ferrer. The deepfake detection challenge (dfdc) dataset. *arXiv preprint arXiv:2006.07397*, 2020.

[180] Thomas Wiegand, Gary J Sullivan, Gisle Bjontegaard, and Ajay Luthra. Overview of the h. 264/avc video coding standard. *IEEE Transactions on circuits and systems for video technology*, 13(7):560–576, 2003.

[181] Roberto Caldelli, Leonardo Galteri, Irene Amerini, and Alberto Del Bimbo. Optical flow based cnn for detection of unlearnt deepfake manipulations. *Pattern Recognition Letters*, 146:31–37, 2021.

[182] Denis Fortun, Patrick Bouthemy, and Charles Kervrann. Optical flow modeling and computation: A survey. *Computer Vision and Image Understanding*, 134:1–21, 2015.

[183] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 770–778, 2016.

[184] Gaojian Wang, Qian Jiang, Xin Jin, and Xiaohui Cui. Ffr_fd: Effective and fast detection of deepfakes via feature point defects. *Information Sciences*, 596:472–488, 2022.

[185] Chengcai Leng, Hai Zhang, Bo Li, Guorong Cai, Zhao Pei, and Li He. Local feature descriptor for image matching: A survey. *IEEE Access*, 7:6424–6434, 2018.

[186] Mariana Belgiu and Lucian Drăguţ. Random forest in remote sensing: A review of applications and future directions. *ISPRS journal of photogrammetry and remote sensing*, 114:24–31, 2016.

[187] Buyun Liang, Zhongyuan Wang, Baojin Huang, Qin Zou, Qian Wang, and Jingjing Liang. Depth map guided triplet network for deepfake face detection. *Neural Networks*, 159:34–42, 2023.

[188] Alican Mertan, Damien Jade Duff, and Gozde Unal. Single image depth estimation: An overview. *Digital Signal Processing*, 123:103441, 2022.

[189] Ze Liu, Yutong Lin, Yue Cao, Han Hu, Yixuan Wei, Zheng Zhang, Stephen Lin, and Baining Guo. Swin transformer: Hierarchical vision transformer using

shifted windows. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 10012–10022, 2021.

[190] Jingdong Wang, Ke Sun, Tianheng Cheng, Borui Jiang, Chaorui Deng, Yang Zhao, Dong Liu, Yadong Mu, Mingkui Tan, Xinggang Wang, et al. Deep high-resolution representation learning for visual recognition. *IEEE transactions on pattern analysis and machine intelligence*, 43(10):3349–3364, 2020.

[191] Xiangling Ding, Wenjie Zhu, and Dengyong Zhang. Deepfake videos detection via spatiotemporal inconsistency learning and interactive fusion. In *2022 19th Annual IEEE International Conference on Sensing, Communication, and Networking (SECON)*, pages 425–433. IEEE, 2022.

[192] Ann Maria John, Kiran Khanna, Ritika R Prasad, and Lakshmi G Pillai. A review on application of fourier transform in image restoration. In *2020 Fourth International Conference on I-SMAC (IoT in Social, Mobile, Analytics and Cloud)(I-SMAC)*, pages 389–397. IEEE, 2020.

[193] Yungui Cao, Jiazhen Chen, Liqing Huang, Tianqian Huang, and Feng Ye. Three-classification face manipulation detection using attention-based feature decomposition. *Computers & Security*, 125:103024, 2023.

[194] Jessica Fridrich and Jan Kodovsky. Rich models for steganalysis of digital images. *IEEE Transactions on information Forensics and Security*, 7(3):868–882, 2012.

[195] François Chollet. Xception: Deep learning with depthwise separable convolutions. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 1251–1258, 2017.

[196] Volker Blanz and Thomas Vetter. A morphable model for the synthesis of 3d faces. In *Seminal Graphics Papers: Pushing the Boundaries, Volume 2*, pages 157–164. 2023.

[197] Joon Son Chung, Arsha Nagrani, and Andrew Zisserman. Voxceleb2: Deep speaker recognition. *arXiv preprint arXiv:1806.05622*, 2018.

[198] Gen Li, Xianfeng Zhao, and Yun Cao. Forensic symmetry for deepfakes. *IEEE Transactions on Information Forensics and Security*, 18:1095–1110, 2023.

[199] Xin Liao, Yumei Wang, Tianyi Wang, Juan Hu, and Xiaoshuai Wu. Famm: Facial muscle motions for detecting compressed deepfake videos over social networks. *IEEE Transactions on Circuits and Systems for Video Technology*, 2023.

[200] Janmenjoy Nayak, Bighnaraj Naik, and HS Behera. A comprehensive survey on support vector machine in data mining tasks: applications & challenges. *International Journal of Database Theory and Application*, 8(1):169–186, 2015.

[201] Rahul Dey and Fathi M Salem. Gate-variants of gated recurrent unit (gru) neural networks. In *2017 IEEE 60th international midwest symposium on circuits and systems (MWSCAS)*, pages 1597–1600. IEEE, 2017.

[202] Zhiming Xia, Tong Qiao, Ming Xu, Ning Zheng, and Shichuang Xie. Towards deepfake video forensics based on facial textural disparities in multi-color channels. *Information Sciences*, 607:654–669, 2022.

[203] Shreyank N Gowda and Chun Yuan. Colornet: Investigating the importance of color spaces for image classification. In *Computer Vision–ACCV 2018: 14th Asian Conference on Computer Vision, Perth, Australia, December 2–6, 2018, Revised Selected Papers, Part IV 14*, pages 581–596. Springer, 2019.

[204] Li Zhang, Tong Qiao, Ming Xu, Ning Zheng, and Shichuang Xie. Unsupervised learning-based framework for deepfake video detection. *IEEE Transactions on Multimedia*, 2022.

[205] Davide Cozzolino and Luisa Verdoliva. Noiseprint: A cnn-based camera model fingerprint. *IEEE Transactions on Information Forensics and Security*, 15:144–159, 2019.

[206] Jyoti Yadav and Monika Sharma. A review of k-mean algorithm. *Int. J. Eng. Trends Technol*, 4(7):2972–2976, 2013.

[207] Video classification with a 3D convolutional neural network — TensorFlow Core — tensorflow.org. `https://www.tensorflow.org/tutorials/video/video_classification#:~:text=A%203D%20CNN%20uses%20a,for%20Action%20Recognition%20by%20D.` [Accessed 20-Jul-2023].

[208] Kensho Hara, Hirokatsu Kataoka, and Yutaka Satoh. Can spatiotemporal 3d cnns retrace the history of 2d cnns and imagenet? In *Proceedings of the IEEE conference on Computer Vision and Pattern Recognition*, pages 6546–6555, 2018.

[209] Joao Carreira and Andrew Zisserman. Quo vadis, action recognition? a new model and the kinetics dataset. In *proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 6299–6308, 2017.

[210] Han Chen, Yuezun Li, Dongdong Lin, Bin Li, and Junqiang Wu. Watching the big artifacts: Exposing deepfake videos via bi-granularity artifacts. *Pattern Recognition*, 135:109179, 2023.

[211] Ying Li, Shan Bian, Chuntao Wang, Kemal Polat, Adi Alhudhaif, and Fayadh Alenezi. Exposing low-quality deepfake videos of social network service using

spatial restored detection framework. *Expert Systems with Applications*, page 120646, 2023.

[212] Maryam Khaksar Ghalati, Ana Nunes, Hugo Ferreira, Pedro Serranho, and Rui Bernardes. Texture analysis and its applications in biomedical imaging: A survey. *IEEE Reviews in Biomedical Engineering*, 15:222–246, 2021.

[213] Anjana Samindra Perera, Ajantha S Atukorale, and Prabhash Kumarasinghe. Employing super resolution to improve low-quality deepfake detection. In *2022 22nd International Conference on Advances in ICT for Emerging Regions (ICTer)*, pages 013–018. IEEE, 2022.

[214] Saeed Anwar, Salman Khan, and Nick Barnes. A deep journey into super-resolution: A survey. *ACM computing surveys (CSUR)*, 53(3):1–34, 2020.

[215] Jianpeng Ke and Lina Wang. Df-udetector: An effective method towards robust deepfake detection via feature restoration. *Neural Networks*, 160:216–226, 2023.

[216] Indika Wickramasinghe and Harsha Kalutarage. Naive bayes: applications, variations and vulnerabilities: a review of literature with code snippets for implementation. *Soft Computing*, 25(3):2277–2293, 2021.

[217] Chao-Yuan Wu, Manzil Zaheer, Hexiang Hu, R Manmatha, Alexander J Smola, and Philipp Krähenbühl. Compressed video action recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 6026–6035, 2018.

[218] Usha Ruby, Vamsidhar Yendapalli, et al. Binary cross entropy with deep learning technique for image classification. *Int. J. Adv. Trends Comput. Sci. Eng*, 9(10), 2020.

[219] FFmpeg — ffmpeg.org. `https://ffmpeg.org/`. [Accessed 13-05-2024].

[220] Jamimamul Bakas, Ruchira Naskar, and Rahul Dixit. Detection and localization of inter-frame video forgeries based on inconsistency in correlation distribution between haralick coded frames. *Multimedia Tools and Applications*, 78:4905–4935, 2019.

[221] pypi.org. `https://pypi.org/project/dlib/`. [Accessed 13-01-2025].

[222] Connor Shorten and Taghi M Khoshgoftaar. A survey on image data augmentation for deep learning. *Journal of big data*, 6(1):1–48, 2019.

[223] Diederik P Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014.

[224] Karen Simonyan and Andrew Zisserman. Very deep convolutional networks for large-scale image recognition. *arXiv preprint arXiv:1409.1556*, 2014.

[225] Mingxing Tan and Quoc Le. Efficientnet: Rethinking model scaling for convolutional neural networks. In *International conference on machine learning*, pages 6105–6114. PMLR, 2019.

[226] Denis A Pustokhin, Irina V Pustokhina, Phuoc Nguyen Dinh, Son Van Phan, Gia Nhu Nguyen, Gyanendra Prasad Joshi, and Shankar K. An effective deep residual network based class attention layer with bidirectional lstm for diagnosis and classification of covid-19. *Journal of Applied Statistics*, 50(3):477–494, 2023.

[227] Vaishali Sharma and Neetu Singh. Deep convolutional neural network with resnet-50 learning algorithm for copy-move forgery detection. In *2021 7th International conference on signal processing and communication (ICSC)*, pages 146–150. IEEE, 2021.

[228] Tinshu Sasi, Arash Habibi Lashkari, Rongxing Lu, Pulei Xiong, and Shahrear Iqbal. An efficient self attention-based 1d-cnn-lstm network for iot attack detection and identification using network traffic. *Journal of Information and Intelligence*, 2024.

[229] Rubina Sarki, Sandra Michalska, Khandakar Ahmed, Hua Wang, and Yanchun Zhang. Convolutional neural networks for mild diabetic retinopathy detection: an experimental study. *BioRxiv*, page 763136, 2019.

[230] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. Imagenet: A large-scale hierarchical image database. In *2009 IEEE conference on computer vision and pattern recognition*, pages 248–255. Ieee, 2009.

[231] Ntivuguruzwa Jean De La Croix, Tohari Ahmad, and Fengling Han. Comprehensive survey on image steganalysis using deep learning. *Array*, page 100353, 2024.

[232] Jianwei Fei, Yunshu Dai, Peipeng Yu, Tianrun Shen, Zhihua Xia, and Jian Weng. Learning second order local anomaly for general face forgery detection. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 20270–20280, 2022.

[233] Sara Concas, Simone Maurizio La Cava, Giulia Orrù, Carlo Cuccu, Jie Gao, Xiaoyi Feng, Gian Luca Marcialis, and Fabio Roli. Analysis of score-level fusion rules for deepfake detection. *Applied Sciences*, 12(15):7365, 2022.

[234] Ahsan Bin Tufail, Yong-Kui Ma, and Qiu-Na Zhang. Binary classification of alzheimer's disease using smri imaging modality and deep learning. *Journal of digital imaging*, 33(5):1073–1090, 2020.

[235] Naveed Ur Rehman Ahmed, Afzal Badshah, Hanan Adeel, Ayesha Tajammul, Ali Duad, and Tariq Alsahfi. Visual deepfake detection: Review of techniques, tools, limitations, and future prospects. *IEEE Access*, 2024.

[236] Muhammad Zubair and Saqib Hakak. Exploring the landscape of compressed deepfakes: Generation, dataset and detection. *Neurocomputing*, page 129116, 2024.

[237] celeb-deepfakeforensics/Celeb-DF-v1 at master · yuezunli/celeb-deepfakeforensics — github.com. `https://github.com/yuezunli/celeb-deepfakeforensics/tree/master/Celeb-DF-v1`. [Accessed 13-05-2024].

[238] Andreas Rössler, Davide Cozzolino, Luisa Verdoliva, Christian Riess, Justus Thies, and Matthias Nießner. Faceforensics: A large-scale video dataset for forgery detection in human faces. *arXiv preprint arXiv:1803.09179*, 2018.

[239] YouTube recommended upload encoding settings - YouTube Help — support.google.com. `https://support.google.com/youtube/answer/1722171?hl=en#zippy=`. [Accessed 05-11-2024].

[240] developers.snap.com. `https://developers.snap.com/lens-studio/adding-content/2d/video`. [Accessed 05-11-2024].

[241] Log into Facebook — facebook.com. `https://www.facebook.com/business/help/1692533511105458`. [Accessed 05-11-2024].

[242] Sending Media — core.telegram.org. `https://core.telegram.org/blackberry/chat-media-send`. [Accessed 06-01-2025].

[243] Ticao Zhang and Shiwen Mao. An overview of emerging video coding standards. *GetMobile: Mobile Computing and Communications*, 22(4):13–20, 2019.

[244] Daniel J McDuff, Ethan B Blackford, and Justin R Estepp. The impact of video compression on remote cardiac pulse measurement using imaging photoplethysmography. In *2017 12th IEEE International Conference on Automatic Face & Gesture Recognition (FG 2017)*, pages 63–70. IEEE, 2017.

[245] Leandro Cunha, Li Zhang, Bilal Sowan, Chee Peng Lim, and Yinghui Kong. Video deepfake detection using particle swarm optimization improved deep neural networks. *Neural Computing and Applications*, 36(15):8417–8453, 2024.

[246] TensorFlow — tensorflow.org. `https://www.tensorflow.org/`. [Accessed 07-03-2025].

[247] Google Colab — colab.research.google.com. `https://colab.research.google.com/`. [Accessed 08-01-2025].

[248] Mahdi Rabbani, Jinkun Gui, Fatemeh Nejati, Zeming Zhou, Arun Kaniyamattam, Mansur Mirani, Gunjan Piya, Igor Opushnyev, Rongxing Lu, and Ali A Ghorbani. Device identification and anomaly detection in iot environments. *IEEE Internet of Things Journal*, 2024.

[249] Jerzy Wieczorek, Cole Guerin, and Thomas McMahon. K-fold cross-validation for complex sample surveys. *Stat*, 11(1):e454, 2022.

[250] Isaac Kofi Nti, Owusu Nyarko-Boateng, Justice Aning, et al. Performance of machine learning algorithms with different k values in k-fold cross-validation. *International Journal of Information Technology and Computer Science*, 13(6):61–71, 2021.

[251] Hao Li, Gopi Krishnan Rajbahadur, Dayi Lin, Cor-Paul Bezemer, and Zhen Ming Jiang. Keeping deep learning models in check: A history-based approach to mitigate overfitting. *IEEE Access*, 2024.

[252] Ramprasaath R Selvaraju, Michael Cogswell, Abhishek Das, Ramakrishna Vedantam, Devi Parikh, and Dhruv Batra. Grad-cam: Visual explanations from deep networks via gradient-based localization. In *Proceedings of the IEEE international conference on computer vision*, pages 618–626, 2017.

[253] Bo Wang, Xiaohan Wu, Yeling Tang, Yanyan Ma, Zihao Shan, and Fei Wei. Frequency domain filtered residual network for deepfake detection. *Mathematics*, 11(4):816, 2023.

Appendix A

Codes

In this appendix, some essential parts of our implementation are provided.

```
## ffmpeg
def convert_to_h264_v2(input_dir, output_dir,
    compression_factor):
    # Get the list of files in the input directory
    files = os.listdir(input_dir)

    # Create the output directory if it doesn't exist
    os.makedirs(output_dir, exist_ok=True)

    # Iterate over the files in the input directory
    for file in files:
        # Construct the input and output file paths
        input_file = os.path.join(input_dir, file)
        output_file = os.path.join(output_dir, file)
        command = f'ffmpeg -i "{input_file}" -vcodec libx264 -
            crf {compression_factor} "{output_file}"'
        os.system(command)
```

Figure A.1: Code Snippet to convert a video into H.264 and apply Compression on it

```
    import subprocess
    import cv2
    #This function takes a video, breaks it into I-Frames,
    #and saves it.
def break_into_I_Frames (input_path):
    output_dir = "IFrames_Per_Video"
    # Define the FFmpeg command to extract I-frames
    command = ['ffmpeg', '-i', input_path, '-vf',
               'select=eq(pict_type\,I)', '-vsync', 'vfr',
                 f'{output_dir}/frame%d.jpg']

    #Run the FFmpeg command using subprocess, it will
    #extract I-Frames and save them in the Folder
    #of IFrames_Per_Video
    subprocess.call(command)

    # This function will copy the I-Frames
    #from the saved path and delete them as well
    #to make way for the next video
    return read_images_from_folder((output_dir))
```

Figure A.2: Code Snippet for Extracting I-Frames from a Video

```python
# Detecting and doing alignment of face and returing it in size of 256 by 256 and image is colored RGB
def Face_Detection_and_alignment(img):
    detector = dlib.get_frontal_face_detector() # Load the face detector of dlib
    predictor = dlib.shape_predictor("Helper_Models\\shape_predictor_68_face_landmarks.dat")
    gray = img   # using gray name of variable is not good
    faces = detector(gray) # Use the face detector to detect the face in the input image,# If no face is detected, print an error message and exit the program
    if len(faces) == 0:
        print('No face detected in the input image!')
        cv2.imwrite('False data/output_image.jpg', img)
        return None
    # Get the rectangle that bounds the face
    face_rect = faces[0]
    if img is not None:  # Crop the input image to the bounds of the detected face
     face_image = img[face_rect.top():face_rect.bottom(), face_rect.left():face_rect.right()]
    if face_image is not None and face_image.size != 0:
        # Resize the cropped face image to 256x256  # resized_face_image = cv2.resize(face_image, (256, 256))
       # Detect the face in the image
     detections = detector(img, 1)
     if not detections:
        # Handle the case when no face is detected
        print('No face detected in the image.')
        return None
    # Get the landmarks for the first face, # Align the face using the landmarks
    landmarks = predictor(img, detections[0])
    aligned_face = dlib.get_face_chip(img, landmarks, size=256)
    return aligned_face
```

Figure A.3: Code Snippet for Face Detection and Resizing Image

```
1
2  # Function name is updated
3  def driver_training_ResNet152(m_Data_Dir, m_Epoches,
       m_Batch_Size, m_Model_Path, m_Model_Name, m_Results_Dir,
       m_Learning_Rate ):
4      REAL_DIR = os.path.join(m_Data_Dir, 'real')
5      FAKE_DIR = os.path.join(m_Data_Dir, 'fake')
6
7      real_images = load_images(REAL_DIR)
8      fake_images = load_images(FAKE_DIR)
9
10     X = np.concatenate((real_images, fake_images), axis=0)
11     y = np.concatenate((np.ones(len(real_images)), np.zeros(
           len(fake_images))))
12
13
14     # Record need to be saved
15
16     # 5-fold cross-validation to maintain a balance between
           real and fake, with random state = 42
17     skf = StratifiedKFold(n_splits=5, shuffle=True,
           random_state=42)
18
19     for fold, (train_index, test_index) in enumerate(skf.split
           (X, y), 1):
20         print(f"Training on Fold {fold}:")
21
22         X_train, X_test = X[train_index], X[test_index]
23         y_train, y_test = y[train_index], y[test_index]
24
25         # Creating Model
26         # Calculate steps per epoch
27         steps_per_epoch = len(X) // m_Batch_Size
28          # Learning rate scheduler
29         lr_schedule = tf.keras.optimizers.schedules.
               ExponentialDecay(
30         initial_learning_rate = m_Learning_Rate,  # Starting
               learning rate
31         decay_steps=steps_per_epoch * 20,                #
               Decay every 10 epochs
32         decay_rate=0.1,                # Decay by 0.1 (factor
               of 0.1)
33         staircase=True                 # Apply decay in
               discrete steps (not continuous)
34       )
```

Figure A.4: Code Snippet for Training of ResNet-152 on RGB data Part 1

```
1
2  with tf.device('/device:GPU:0'):
3              # Create the Adam optimizer with the learning rate
                   schedule
4              optimizer = tf.keras.optimizers.Adam(learning_rate =
                    lr_schedule)
5              ResNet152 = ResNet152Classifier(optimizer)
6              model = ResNet152.model
7              # Data augmentation for training, some augmentation
                   did not work well
8              train_datagen = ImageDataGenerator(
9                rotation_range = 20,
10               zoom_range = 0.2,
11               horizontal_flip = True,
12              )
13             test_datagen = ImageDataGenerator()
14             train_generator = train_datagen.flow(
15               X_train, y_train,
16               batch_size = m_Batch_Size
17               )
18             test_generator = test_datagen.flow(
19               X_test, y_test,
20               batch_size = m_Batch_Size
21           )
22           #Early stopping, the patience is important
23           early_stopping = EarlyStopping(
24               monitor='val_loss',
25               patience=15,
26               restore_best_weights=True,
27           )
```

Figure A.5: Code Snippet for Training of ResNet-152 on RGB data Part 2

```
# Model history
        history = model.fit(
            train_generator,
            epochs = m_Epoches,
            validation_data = test_generator,
            callbacks = [early_stopping],
        )
        # Saving of plots and graphs
        model_save_path = os.path.join(m_Model_Path, f"{
           m_Model_Name}_fold{fold}.h5")
        model.save(model_save_path)
        plt.figure()
        # Plot training accuracy
        plt.subplot(2, 1, 1)  # 2 rows, 1 column, plot in the
           first position
        plt.plot(history.history['accuracy'], label='Training
           accuracy')
        plt.plot(history.history['val_accuracy'], label='
           Validation accuracy')
        plt.legend()
        plt.xlabel('Epoch')
        plt.ylabel('Accuracy')

        # Plot training loss
        plt.subplot(2, 1, 2)  # 2 rows, 1 column, plot in the
           second position
        plt.plot(history.history['loss'], label='Training loss
           ')
        plt.plot(history.history['val_loss'], label='
           Validation loss')
        plt.legend()
        plt.xlabel('Epoch')
        plt.ylabel('Loss')

        plt.tight_layout()  # Adjust layout to prevent overlap
        plt.savefig(os.path.join(m_Results_Dir, f"
           training_plot_fold{fold}.png"))
        plt.close()



    print("Training completed for all folds.")
```

Figure A.6: Code Snippet for Training of ResNet-152 for RGB data Part 3

```
1
2
3
4  #Run the training loop
5  connect_to_storage()  # Ensure the storage is connected once
6
7  # Example usage:
8  # change from 500 to 1 to check if the pipeline runs correctly
       , fixed parameters
9  Epoches = 500
10
11 # ***********Batch Size need to be checked
12 Batch_Size = 8
13 ****************** Learning rate need to be updated
14 learning_rate = 0.00001
15
16
17 # Loop through dataset array. change calling function name
18 for dataset in datasets:
19     print(f"Starting training for dataset: {dataset['DATA_DIR
           ']}")
20     driver_training_ResNet152(
21         m_Data_Dir = dataset["DATA_DIR"],
22         m_Epoches = Epoches,
23         m_Batch_Size = Batch_Size,
24         m_Model_Path = dataset["Model_Path"],
25         m_Model_Name = dataset["Model_Name"],
26         m_Results_Dir = dataset["Results_DIR"],
27         m_Learning_Rate = learning_rate
28     )
29     print(f"Finished training for dataset: {dataset['DATA_DIR
           ']}")
```

Figure A.7: Code Snippet for Calling Function to Train ResNet-152 for Different DataSets on RGB data PART 1

```
1
2  # Main function to test the model
3  def test_data_set_model(data_set_name_model,
       data_set_name_folder_distribution, model_architecture,
       model_architecture_model, base_path, fold_number,
       thresh_hold, max_length):
4      model_file_name = f"{model_architecture_model}_{
           data_set_name_model}_fold{fold_number}.h5"
5      csv_file_path = f"/{base_path}/
           Data_Distribution_Information/
           Final_Updated_Folder_Distribution_Train_Test/{
           data_set_name_folder_distribution}"
6      model_file_path = f"/{base_path}/Tested Deep Learning
           Architectures/Final_Model_Collections/RGB/RGB_80_20/
           Augmentation/{model_architecture}/{data_set_name_model
           }/Models/{model_file_name}"
7      confusion_matrix_file_path = f"/{base_path}/Tested Deep
           Learning Architectures/Final_Model_Collections/RGB/
           RGB_80_20/Augmentation/{model_architecture}/{
           data_set_name_model}/Results/{model_architecture}
           _with_augmentation_confusion_matrix{fold_number}.png"
8      metrics_csv_file_path = f"/{base_path}/Tested Deep
           Learning Architectures/Final_Model_Collections/RGB/
           RGB_80_20/Augmentation/{model_architecture}/{
           data_set_name_model}/Results/{model_architecture}
           _with_augmentation_metrics_results{fold_number}.csv"
9
10     # Read CSV file into a pandas DataFrame
11     df = pd.read_csv(csv_file_path)
12
13     # Check if the DataFrame is not empty
14     if not df.empty:
15         # Filter rows where 'Train_Test' column contains the
               value 'Test'
16         test_df = df[df['Train_Test'] == 'test']
17         real_count = test_df[test_df['Label'] == 1].shape[0]
               # Real class (1)
18         fake_count = test_df[test_df['Label'] == 0].shape[0]
               # Fake class (0)
19
20
21         # Load the pre-trained model
22         model = load_model(model_file_path)
```

Figure A.8: Code Snippet for Calling Function to Test ResNet-152 for Different DataSets on RGB data Part 2

```
# Convert the lists to numpy arrays for calculation
        actual_labels = np.array(actual_labels)
        predicted_labels = np.array(predicted_labels)

        # Calculate performance metrics
        accuracy = accuracy_score(actual_labels,
           predicted_labels)
        precision = precision_score(actual_labels,
           predicted_labels)
        recall = recall_score(actual_labels, predicted_labels)
        f1 = f1_score(actual_labels, predicted_labels)
        conf_matrix = confusion_matrix(actual_labels,
           predicted_labels)
        # Format metrics as percentages
        accuracy_percent = f"{accuracy * 100:.2f}%"
        precision_percent = f"{precision * 100:.2f}%"
        recall_percent = f"{recall * 100:.2f}%"
        f1_percent = f"{f1 * 100:.2f}%"
        # Save formatted metrics to a CSV file
        metrics_df = pd.DataFrame({
            'Metric': ['Accuracy', 'Precision', 'Recall', 'F1
               Score'],'Value': [accuracy_percent,
               precision_percent, recall_percent, f1_percent]
        })
       metrics_df.to_csv(metrics_csv_file_path, index=False)
       # Display and save confusion matrix if it's not empty
        if conf_matrix.size > 0:
            plt.figure(figsize=(8, 6))
            sns.heatmap(conf_matrix, annot=True, fmt='d', cmap
               ='Blues', xticklabels=['Fake', 'Real'],
               yticklabels=['Fake', 'Real'])
            plt.xlabel('Predicted Label')
            plt.ylabel('Actual Label')
            plt.title('Confusion Matrix')
            plt.savefig(confusion_matrix_file_path)
            plt.show()
        else:
            print("Confusion Matrix is empty.")
        print(f"Metrics saved to {metrics_csv_file_path}")
        print(f"Confusion Matrix saved to {
           confusion_matrix_file_path}")
    else:
        print("DataFrame is empty. Check your CSV file or
           filtering conditions.")
```

Figure A.9: Code Snippet for Calling Function to Test ResNet-152 for Different DataSets on RGB data Part 3

```
import tensorflow as tf
import matplotlib.pyplot as plt

def initialize_filters_and_Q(num_filters=27):  # ---------------- May be need to change ----------
    Based on needs
    # Define filters (make sure they are tensors)
    filter1 = tf.constant([[0, 0, 0, 0, 0],
                           [0, -1, 2, -1, 0],
                           [0, 2, -4, 2, 0],
                           [0, -1, 2, -1, 0],
                           [0, 0, 0, 0, 0]], dtype=tf.float32)

    filter2 = tf.constant([[-1, 2, -2, 2, -1],
                           [2, -6, 8, -6, 2],
                           [-2, 8, -12, 8, -2],
                           [2, -6, 8, -6, 2],
                           [-1, 2, -2, 2, -1]], dtype=tf.
                              float32)

    filter3 = tf.constant([[0, 0, 0, 0, 0],
                           [0, 0, 0, 0, 0],
                           [0, 1, -2, 1, 0],
                           [0, 0, 0, 0, 0],
                           [0, 0, 0, 0, 0]], dtype=tf.float32)

    filter4 = tf.constant([[0, 0, 0, 0, 0],
                            [0, 1, 0, 0, 0],
                            [0, 0, -2, 0, 0],
                            [0, 0, 0, 1, 0],
                            [0, 0, 0, 0, 0]], dtype=tf.
                               float32)

    # updated here class 1
    filter5 = tf.constant       ([[0, 0, 0, 0, 0],
                                  [0, 0, 0, 0, 0],
                                  [0, 0, -1, 1, 0],
                                  [0, 0, 0, 0, 0],
                                  [0, 0, 0, 0, 0]], dtype=tf.
                                     float32)
         # Edge 5*5
    filter6 = tf.constant        ([[-1,2, -2,2, -1],
                                  [2, -6,8,-6 ,2],
                                  [-2, 8, -12, 8, -2],
                                  [0, 0, 0, 0, 0],
                                  [0, 0, 0, 0, 0]], dtype=tf.
                                     float32)
```

Figure A.10: Code Snippet for Defining SRM Layer Part 1

```
filter7 = tf.constant       filter7 = tf.constant
                                ([[0, 0,  0,0, 0],
                                 [0, 0, 0, 0, 0],
                                 [-2,8,-12 ,8, -2],
                                 [2, -6,8,-6, 2],
                                 [-1, 2, -2, 2, -1]], dtype=tf.
                                    float32)
                                 filter8 = tf.constant
                                    ([[0, 0,-1,0, 0],
                                 [0, 0,  3, 0, 0],
                                 [0, 0, -3, 0, 0],
                                 [0, 0,  1, 0, 0],
                                 [0, 0, 0, 0, -0]
                                 ], dtype=tf.float32)
                                 filter9 = tf.constant
                                 ([[0, 0, 0, 0, -1],
                                 [0, 0, 0, 3, 0],
                                 [0, 0,-3, 0, 0],
                                 [0, 1, 0, 0, 0],
                                 [0, 0, 0, 0, 0]
                                 ], dtype=tf.float32)
                                 filter10 =
                                 tf.constant([[0,0,0,0,0],
                                  [0,0,0,0,0],
                                  [0,0,-1,0,0],
                                  [0,0,1,0,0],
                                  [0,0,0,0,0]],
                                  dtype=tf.float32)
                                  filter11 = tf.constant
                                  ([[0,0,0,0,0],
                                  [0,0,0,0,0],
                                  [0,0,-1,0,0],
                                  [0,1,0,0,0],
                                  [0,0,0,0,0]],dtype=tf.float32
                                     )
```

Figure A.11: Code Snippet for Defining SRM Layer Part 2

```
1
2  filter13 = tf.constant([
3     [0,0,0,0,0],
4     [0,1,0,0,0],
5     [0,0,-1,0,0],
6     [0,0,0,0,0],
7     [0,0,0,0,0]
8     ],dtype=tf.float32)
9     filter14 = tf.constant([
10    [0,0,0,0,0],
11    [0,0,1,0,0],
12    [0,0,-2,0,0],
13    [0,0,1,0,0],
14    [0,0,0,0,0]
15    ],dtype=tf.float32)
16    filter15 = tf.constant([
17    [0,0,0,0,0],
18    [0,0,0,1,0],
19    [0,0,-2,0,0],
20    [0,1,0,0,0],
21    [0,0,0,0,0]
22    ],dtype=tf.float32)
23   filter16 = tf.constant([
24    [0,0,0,0,0],
25    [0,0,0,0,0],
26    [0,1,-3,3,-1],
27    [0,0,0,0,0],
28    [0,0,0,0,0]
29    ],dtype=tf.float32)
30    filter17 = tf.constant([
31    [0,0,0,0,0],
32    [0,1,0,0,0],
33    [0,0,-3,0,0],
34    [0,0,0,3,0],
35    [0,0,0,0,-1]
36    ],dtype=tf.float32)
37   filter18 = tf.constant([
38    [0,0,0,0,0],
39    [0,0,1,0,0],
40    [0,0,-3,0,0],
41    [0,0,3,0,0],
42    [0,0,-1,0,0]
43    ],dtype=tf.float32)
```

Figure A.12: Code Snippet for Defining SRM Layer Part 3

```
filter19 = tf.constant([
   [0,0,0,0,0],
   [0,0,0,1,0],
   [0,0,-3,0,0],
   [0,3,0,0,0],
   [-1,0,0,0,0]
   ],dtype=tf.float32)
  filter20 = tf.constant([
   [0,0,0,0,0],
   [0,0,0,0,0],
   [-1,3,-3,1,0],
   [0,0,0,0,0],
   [0,0,0,0,0]
   ],dtype=tf.float32)
  filter21 = tf.constant([
   [-1,0,0,0,0],
   [0,3,0,0,0],
   [0,0,-3,0,0],
   [0,0,0,1,0],
   [0,0,0,0,0]
   ],dtype=tf.float32)
   filter22 = tf.constant([
   [0,0,0,0,0],
   [0,-1,2,-1,0],
   [0,2,-4,2,0],
   [0,0,0,0,0],
   [0,0,0,0,0]
   ],dtype=tf.float32)
   filter23 = tf.constant([
   [0,0,0,0,0],
   [0,0,2,-1,0],
   [0,0,-4,2,0],
   [0,0,2,-1,0],
   [0,0,0,0,0]
   ],dtype=tf.float32)

   filter24 = tf.constant([
   [0,0,0,0,0],
   [0,0,0,0,0],
   [0,2,-4,2,0],
   [0,-1,2,-1,0],
   [0,0,0,0,0]
   ],dtype=tf.float32)
```

Figure A.13: Code Snippet for Defining SRM Layer Part 4

```
1
2  filter25 = tf.constant([
3      [0,0,0,0,0],
4      [0,-1,2,0,0],
5      [0,2,-4,0,0],
6      [0,-1,2,0,0],
7      [0,0,0,0,0]
8      ],dtype=tf.float32)
9
10
11     filter26 = tf.constant([
12     [0,0,-2,2,-1],
13     [0,0,8,-6,2],
14     [0,0,-12,8,-2],
15     [0,0,8,-6,2],
16     [0,0,-2,2,-1]
17     ],dtype=tf.float32)
18
19     filter27 = tf.constant([
20     [-1,2,-2,0,0],
21     [2,-6,8,0,0],
22     [-2,8,-12,0,0],
23     [2,-6,8,0,0],
24     [-1,2,-2,0,0]
25     ],dtype=tf.float32)
26
27 # Stack filters into a 3D tensor of shape (3, 5, 5)
28     filters = tf.stack([filter1, filter2, filter3, filter4,
           filter5, filter6, filter7, filter8, filter9,
29                         filter10, filter11, filter12, filter13
                               , filter14, filter15, filter16,
                               filter17, filter18,
30                         filter19, filter20, filter21, filter22
                               , filter23, filter24, filter25,
                               filter26, filter27
31                       ],axis=0)  # Shape: (3, 5, 5)
                            //------------ may ber need to chnage
                            -------------------------
32
33     # Repeat along axis 1 to create the 3x3 pattern for each
           filter, for different channels
34     filters = tf.stack([filters] * 3, axis=1)
35
36     print("Filters shape:", filters.shape)
37     return filters
```

Figure A.14: Code Snippet for Defining SRM Layer Part 5

```
1
2  # Define LearnableSRM layer
3  class LearnableSRM(tf.keras.layers.Layer):
4      def __init__(self, num_filters, **kwargs):
5          super(LearnableSRM, self).__init__(**kwargs)
6          self.num_filters = num_filters
7          self.filters = initialize_filters_and_Q()  # Initialize filters
8           print("SRM CALLED AGAIN")
9
10         # Initialize learnable matrices for Q, 5x5 matrices
11         self.Q = [
12             self.add_weight(name='Q1', shape=(5, 5), initializer=tf.keras.initializers.Constant(value=4.0), trainable=True),
13             self.add_weight(name='Q2', shape=(5, 5), initializer=tf.keras.initializers.Constant(value=12.0), trainable=True),
14             self.add_weight(name='Q3', shape=(5, 5), initializer=tf.keras.initializers.Constant(value=2.0), trainable=True),
15             #---------------------- update here ---------------------------------
16             self.add_weight(name='Q4',shape=(5, 5), initializer=tf.keras.initializers.Constant(value=2.0), trainable=True),
17             self.add_weight(name='Q5',shape=(5, 5), initializer=tf.keras.initializers.Constant(value=1.0), trainable=True),
18             self.add_weight(name='Q6',shape=(5, 5), initializer=tf.keras.initializers.Constant(value=12.0), trainable=True),
19            self.add_weight(name='Q7',shape=(5, 5), initializer=tf.keras.initializers.Constant(value=12.0), trainable=True),
20             self.add_weight(name='Q8',shape=(5, 5), initializer=tf.keras.initializers.Constant(value=3.0), trainable=True),
21             self.add_weight(name='Q9',shape=(5, 5), initializer=tf.keras.initializers.Constant(value=3.0), trainable=True),
22              self.add_weight(name='Q10',shape=(5, 5), initializer=tf.keras.initializers.Constant(value=1), trainable=True),
23             self.add_weight(name='Q11',shape=(5, 5), initializer=tf.keras.initializers.Constant(value=1), trainable=True),
```

Figure A.15: Code Snippet for Defining SRM Layer Part 6

```
1
2               self.add_weight(name='Q12',shape=(5, 5),
                   initializer=tf.keras.initializers.Constant(
                   value=1), trainable=True),self.add_weight(name=
                   'Q13',shape=(5, 5), initializer=tf.keras.
                   initializers.Constant(value=1), trainable=True)
                   ,
3           self.add_weight(name='Q14',shape=(5, 5), initializer=
               tf.keras.initializers.Constant(value=2), trainable
               =True), self.add_weight(name='Q15',shape=(5, 5),
               initializer=tf.keras.initializers.Constant(value
               =2), trainable=True),
4               self.add_weight(name='Q16',shape=(5, 5),
                   initializer=tf.keras.initializers.Constant(
                   value=3), trainable=True),self.add_weight(name
                   ='Q17',shape=(5, 5), initializer=tf.keras.
                   initializers.Constant(value=3), trainable=True
                   ),
5               self.add_weight(name='Q18',shape=(5, 5),
                   initializer=tf.keras.initializers.Constant(
                   value=3), trainable=True),self.add_weight(name=
                   'Q19',shape=(5, 5), initializer=tf.keras.
                   initializers.Constant(value=3), trainable=True)
                   ,
6               self.add_weight(name='Q20',shape=(5, 5),
                   initializer=tf.keras.initializers.Constant(
                   value=3), trainable=True),self.add_weight(name=
                   'Q21',shape=(5, 5), initializer=tf.keras.
                   initializers.Constant(value=3), trainable=True)
                   ,self.add_weight(name='Q22',shape=(5, 5),
                   initializer=tf.keras.initializers.Constant(
                   value=4), trainable=True),self.add_weight(name=
                   'Q23',shape=(5, 5), initializer=tf.keras.
                   initializers.Constant(value=4), trainable=True)
                   ,self.add_weight(name='Q24',shape=(5, 5),
                   initializer=tf.keras.initializers.Constant(
                   value=4), trainable=True),self.add_weight(name=
                   'Q25',shape=(5, 5), initializer=tf.keras.
                   initializers.Constant(value=4), trainable=True)
                   ,
7               self.add_weight(name='Q26',shape=(5, 5),
                   initializer=tf.keras.initializers.Constant(
                   value=12), trainable=True),self.add_weight(name
                   ='Q27',shape=(5, 5), initializer=tf.keras.
                   initializers.Constant(value=12), trainable=True
                   ),
8   ]
```

Figure A.16: Code Snippet for Defining SRM Layer Part 7

```
1
2
3  def visualize_filters(self, epoch):
4          fig, axs = plt.subplots(self.num_filters, 3, figsize
               =(15, 5 * self.num_filters))
5          for i, (filter, Q) in enumerate(zip(self.filters, self
               .Q)):
6              normalized_filter = tf.divide(filter, Q)
7              axs[i, 0].imshow(normalized_filter.numpy(), cmap='
                   gray')
8              axs[i, 0].set_title(f"Filter {i+1} (Original)")
9              axs[i, 1].imshow(Q.numpy(), cmap='gray')
10             axs[i, 1].set_title(f"Q {i+1}")
11             axs[i, 2].imshow(normalized_filter.numpy(), cmap='
                   gray')
12             axs[i, 2].set_title(f"Normalized Filter {i+1}")
13         plt.tight_layout()
14         plt.savefig(f'filters_epoch{epoch}.png')
15         plt.close()
16   def call(self, inputs):
17         # Convert Q weights to tensors before using tf.stack
18         Q = [tf.convert_to_tensor(q) for q in self.Q]
19        # Stack the filters and Q tensors to use with map_fn
20         filters = tf.stack(self.filters, axis=0)  # Shape: (3,
               5, 5)
21         Q = tf.stack(Q, axis=0)  # Shape: (3, 5, 5)
22        # Use tf.map_fn to normalize filters element-wise using
               Q
23         normalized_filters = tf.map_fn(lambda filter_Q: tf.
               divide(filter_Q[0], filter_Q[1]),
24                                        (filters, Q), dtype=tf.
                                             float32)
25         # Convert normalized_filters to the required shape for
               convolution (5, 5, 3, 3)
26         #Update here for number of filters
27         normalized_filters = tf.reshape(normalized_filters,
               [5, 5, 3, 27])  # Shape: (5, 5, 3, 3)
28       # Print the shape of the normalized filters
29         print("Normalized Filters shape:", normalized_filters.
               shape)
30         # Ensure inputs are in the right shape (batch_size,
               height, width, channels)
31         batch_size = tf.shape(inputs)[0]
32         height = tf.shape(inputs)[1]
33         width = tf.shape(inputs)[2]
34         # Apply convolution using all normalized filters
35         conv_output = tf.nn.conv2d(inputs, normalized_filters,
               strides=[1, 1, 1, 1], padding='SAME')
36         return conv_output
```


Figure A.17: Code Snippet for Defining SRM Layer Part 8

```
def get_config(self):
        config = super(LearnableSRM, self).get_config()
        config.update({
            'num_filters': self.num_filters,                    #
               may be need to change here
            'Q1': self.Q[0].numpy(),
            'Q2': self.Q[1].numpy(),
            'Q3': self.Q[2].numpy(),
            # updated here
           'Q4': self.Q[3].numpy(),
           'Q5': self.Q[4].numpy(),
           'Q6': self.Q[5].numpy(),
           'Q7': self.Q[6].numpy(),
           'Q8': self.Q[7].numpy(),
           'Q9': self.Q[8].numpy(),
        'Q10': self.Q[9].numpy(),
        'Q11': self.Q[10].numpy(),
        'Q12': self.Q[11].numpy(),
        'Q13': self.Q[12].numpy(),
        'Q14': self.Q[13].numpy(),
        'Q15': self.Q[14].numpy(),
        'Q16': self.Q[15].numpy(),
        'Q17': self.Q[16].numpy(),
        'Q18': self.Q[17].numpy(),
        'Q19': self.Q[18].numpy(),
        'Q20': self.Q[19].numpy(),
        'Q21': self.Q[20].numpy(),
        'Q22': self.Q[21].numpy(),
        'Q23': self.Q[22].numpy(),
        'Q24': self.Q[23].numpy(),
        'Q25': self.Q[24].numpy(),
        'Q26': self.Q[25].numpy(),
        'Q27': self.Q[26].numpy(),
        })
        return config
```

Figure A.18: Code Snippet for Defining SRM Layer Part 9

```
@classmethod
    def from_config(cls, config):
        layer = cls(num_filters=config['num_filters'])
                        # may be need to change here
        layer.Q[0].assign(config['Q1'])
        layer.Q[1].assign(config['Q2'])
        layer.Q[2].assign(config['Q3'])
        #updated here
        layer.Q[3].assign(config['Q4'])
        layer.Q[4].assign(config['Q5'])
        layer.Q[5].assign(config['Q6'])
        layer.Q[6].assign(config['Q7'])
        layer.Q[7].assign(config['Q8'])
        layer.Q[8].assign(config['Q9'])

        # Add here

        # updated here
        layer.Q[9].assign(config['Q10'])
        layer.Q[10].assign(config['Q11'])
        layer.Q[11].assign(config['Q12'])
        layer.Q[12].assign(config['Q13'])
        layer.Q[13].assign(config['Q14'])
        layer.Q[14].assign(config['Q15'])


        layer.Q[15].assign(config['Q16'])
        layer.Q[16].assign(config['Q17'])
        layer.Q[17].assign(config['Q18'])
        layer.Q[18].assign(config['Q19'])
        layer.Q[19].assign(config['Q20'])
        layer.Q[20].assign(config['Q21'])


        layer.Q[21].assign(config['Q22'])
        layer.Q[22].assign(config['Q23'])
        layer.Q[23].assign(config['Q24'])
        layer.Q[24].assign(config['Q25'])
        layer.Q[25].assign(config['Q26'])
        layer.Q[26].assign(config['Q27'])



        return layer
```

Figure A.19: Code Snippet for Defining SRM Layer Part 10

```
from tensorflow.keras.layers import Input,
    GlobalAveragePooling2D, Dropout, Dense
from tensorflow.keras.applications import ResNet152
from tensorflow.keras.models import Model
from tensorflow.keras.utils import get_custom_objects
# Register the custom layer (LearnableSRM) so it is recognized
    during model loading
get_custom_objects().update({'LearnableSRM': LearnableSRM})
image_dimensions = {'height': 256, 'width': 256, 'channels':
    3}
# update here for filters
def modify_model(input_shape=(256, 256, 27)):
        base_model = ResNet152(weights='imagenet', include_top
            =False)
        # Modify the input layer to accept input with 27
            channels
        new_input = Input(shape=input_shape)
        x = Conv2D(3, (1, 1), padding='same')(new_input)  #
            Reduce 27 channels to 3
        x = base_model(x)
        # Create the modified model
        modified_model = tf.keras.Model(inputs=new_input,
            outputs=x)
        return modified_model
# Base class
class Classifier:
    def __init__(self):
        self.model = 0
    def predict(self, x):
        return self.model.predict(x)
    def fit(self, x, y):
        return self.model.train_on_batch(x, y)
    def get_accuracy(self, x, y):
        return self.model.test_on_batch(x, y)
    def load(self, path):
        self.model.load_weights(path)
```

Figure A.20: Code Snippet for Defining ResNet-152 with SRM Layer Part 1

```
class ResNet152Classifier(Classifier):
    def __init__(self, optimizer):
        self.model = self.init_model()
     # Compile the model
        self.model.compile(
            optimizer=optimizer,
            loss='binary_crossentropy',  # Binary
               classification task
            metrics=['accuracy']          # Monitor accuracy
               during training
        )
def init_model(self):
        # Input layer
        Input_layer = Input(shape=(image_dimensions['height'],
                          image_dimensions['width'],
                          image_dimensions['channels']))
        # ------------Apply the LearnableSRM layer
        srm_layer = LearnableSRM(num_filters=27)(Input_layer)
                 # update here   --------------
       base_model = modify_model(input_shape=(256, 256, 27))
          # ResNet is now expecting 27 channels
        base_model = base_model(srm_layer)
        # For Fine Tuning
        base_model.trainable = True
        # Add custom classification head
        y = GlobalAveragePooling2D()(base_model)
        y = Dropout(0.5)(y)  # Dropout to reduce overfitting
        y = Dense(16, activation='relu')(y)
        y = Dropout(0.5)(y)
        y = Dense(1, activation='sigmoid')(y)  # Binary
           classification output
        # Build the model
        return Model(inputs=Input_layer , outputs=y)
```

Figure A.21: Code Snippet for Defining ResNet-152 with SRM Layer Part 2

Vita

Candidate's full name: Muhammad Zubair

University attended:
Bachelor of Computer Science
National University of Computer and Emerging Science (FAST-NUCES)
2015-2019

Publications:
Muhammad Zubair, Saqib Hakak: **"Deep Authenticator: A Method for Compressed DeepFake Detection"**: – Under Review, Elsevier Expert Systems With Applications

Muhammad Zubair, Saqib Hakak: **"Exploring the Landscape of Compressed DeepFakes: Generation, Dataset and Detection"**: – Elsevier NeuroComputing

Conference Poster Presentations:
Muhammad Zubair, Saqib Hakak; **"DeepFakes"**
Industry Summit of Annual International Conference on Privacy, Security & Trust; October 2024; Fredericton, New Brunswick, Canada.

Muhammad Zubair, Saqib Hakak; **"Look Beyond the pixels- Unmask the deception"**
UNB Research Expo; April 2024; Fredericton, New Brunswick, Canada.

Muhammad Zubair, Saqib Hakak; **"Look Beyond the pixels- Unmask the deception"**
Industry Summit of International Conference on Privacy, Security & Trust; October 2023; Fredericton, New Brunswick, Canada.

Muhammad Zubair, Saqib Hakak; **"DeepFake Analysis in Compressed Video"**
UNB Research Expo; April 2023; Fredericton, New Brunswick, Canada.